Table of contents

MEDSCRIPTAI - Zero-to-Elite Lab Manual (October 2026)

PART 0 - Start Here

0.1 What MedScriptAI is
0.2 The parts of this manual
0.3 What you will be able to do at the end
0.4 Environment map (every place you will work)
0.5 What each piece does
0.6 What it costs (be honest with yourself before you start)

PART 1 - Prerequisites

1.1 Checklist
1.2 Refreshers and 1-minute checks for every MUST item
1.3 Accounts and installs (do these now; some need a day to be approved)

PART 2 - Set Up Your Workspace

SECTION 2.1 - Create an SSH key on your laptop
SECTION 2.2 - Hugging Face token and LLM machine
SECTION 2.3 - Hyperstack account, billing, environment and key
SECTION 2.4 - Deploy the GPU virtual machine
SECTION 2.5 - First SSH login from PowerShell
SECTION 2.6 - Connect VS Code to the VM (Remote-SSH)
SECTION 2.7 - Connect WinSCP to the VM (file transfer)
SECTION 2.8 - Prepare the VM: folders, caches, code and Python environments
SECTION 2.9 - Tiny test: does PyTorch see the GPU?
SECTION 2.10 - Install FakeCloud (free AWS) on the VM, and the AWS CLI
SECTION 2.11 - (Real AWS only) Request GPU quota now
SECTION 2.12 - Stop paying when you stop working (do this EVERY time)

PART 3 - The Core Idea

3.1 One analogy
3.2 The 20% of ideas that explain 80% of the practice

PART 4 - BASICS: your first medical fine-tune (data → .jsonl → QLoRA SFT → test → publish)

4.0 Concept
SECTION 4.1 - Download the datasets
SECTION 4.2 - Clean the data and convert it to chat .jsonl
SECTION 4.3 - Validate every .jsonl file before paying for GPU time
SECTION 4.4 - Train your first adapter (QLoRA SFT on 2,000 QA examples)
SECTION 4.5 - Compare the base model and your adapter
SECTION 4.6 - Publish the adapter on the Hugging Face Hub
SECTION 4.7 - Your first cloud step: store the adapter in object storage (S3)
4.8 Common mistake at this level

PART 5 - INTERMEDIATE: synthetic data, three adapters, real evaluation

5.0 Concept
SECTION 5.1 - Start the teacher model (vLLM, port 8001)
SECTION 5.2 - Generate synthetic explanations with distilabel
SECTION 5.3 - Free the GPU and train the three adapters
SECTION 5.4 - Serve all adapters with vLLM (port 8000) and the judge (port 8001)
SECTION 5.5 - Generate test-set answers for base and fine-tuned models
SECTION 5.6 - Automatic metrics: accuracy, ROUGE-L, BERTScore, JSON validity
SECTION 5.7 - LLM-as-a-judge (score mode)
SECTION 5.8 - Publish your datasets
SECTION 5.9 - Cloud step: reports to S3 and your token into Secrets Manager
5.10 Common mistake at this level

PART 6 - ADVANCED: DPO, multi-LoRA serving, production API and cloud deployment

6.0 Concept
SECTION 6.1 - Build DPO preference pairs with distilabel
SECTION 6.2 - Train DPO on top of the SFT adapter
SECTION 6.3 - vLLM multi-LoRA: one base model, five adapters, hot-swapping
SECTION 6.4 - Evaluate base vs SFT vs DPO
SECTION 6.5 - The FastAPI service
SECTION 6.6 - LangSmith tracing
SECTION 6.7 - Package the API with Docker and run the whole stack with Compose
SECTION 6.8 - Deploy step by step: registry, storage, secrets, IAM, logs, network, GPU machine
SECTION 6.9 - (Optional) SageMaker endpoint: the managed alternative
6.10 Common mistake at this level

PART 7 - ELITE: how top labs and practitioners do it

7.0 Concept
7.1 Who defines the top level today
SECTION 7.2 - Prove there is no test leakage (near-duplicate contamination check)
SECTION 7.3 - Prove a gain is real (paired bootstrap confidence interval)
SECTION 7.4 - Prove the judge is trustworthy (agreement with your own labels)
SECTION 7.5 - Criterion-level (rubric) scoring for structured summaries
SECTION 7.6 - Zero-downtime adapter rollout and rollback (canary)
SECTION 7.7 - Report and model card
7.8 What elite teams would do next (pick one as your stretch goal)
7.9 Common mistake at this level

PART 8 - Tools and Resources Map (checked October 2026)

PART 9 - Keep It Working

PART 10 - Troubleshooting

PART 11 - Practice Projects (goals and success criteria only, no solutions)

11.1 Easy - a fourth adapter: patient-friendly explanations
11.2 Medium - RL with verifiable rewards for QA
11.3 Hard - production hardening on AWS

PART 12 - Cheat Sheet

Part 2 - workspace
Part 4 - data and first SFT (.venv-train, folder scripts)
Part 5 - synthetic data, three adapters, evaluation
Part 6 - DPO, serving, API, Docker, deployment
Part 7 - elite checks

MedScriptAI

Zero-to-Elite Lab Manual: medical LLM post-training (QLoRA SFT + DPO) and production deployment

Llama-3.1-8B-Instruct · Unsloth · TRL · PEFT · distilabel · vLLM multi-LoRA · FastAPI · LangSmith · Docker · FakeCloud / AWS

Version: October 2026

Key numbers and settings

PART 13 - Self-Test

PART 14 - Sources

ANSWER KEY

MEDSCRIPTAI - Zero-to-Elite Lab Manual (October 2026)

Assumptions (change anything that is wrong for you): you know basic Python but have never fine-tuned a model; your laptop is Windows 11 with no usable GPU; your goal is a portfolio project you can build, show and defend in AI-engineer interviews; budget is pay-as-you-go GPU time on Hyperstack (about US$25-40 for the whole manual), FakeCloud is free, and real AWS is optional (about US$3-10 if you follow the teardown steps).

How to read this manual: every practical section has WHERE (where you work), GOAL, numbered steps with ONE action each, exact text to type in grey boxes, "You should see", "Watch out" and a CHECKPOINT. Cloud steps are shown twice in a table: the white column is FakeCloud (free practice), the light-blue column is the same step in real AWS.

All code in this manual is also in the GitHub repo https://github.com/kashyapphassan2000-netizen/-MEDSCRIPTAI (branch claude/determined-carson-9p7115). The data, synthetic-data, evaluation, API, Docker and FakeCloud parts were run end to end on a CPU machine before publishing; the GPU-only parts (Unsloth training, vLLM serving) use the exact library versions that were resolved and smoke-tested, but you are the first person to run them on a GPU, so read every "You should see".

PART 0 - Start Here

0.1 What MedScriptAI is

MedScriptAI is your own medical assistant built on Meta's Llama-3.1-8B-Instruct. You take a general chat model and post-train it on open, benchmarked medical datasets so it gets better at three jobs: answering medical exam questions with an explanation (qa), turning a doctor-patient conversation into a clinical note (notes), and turning a clinical note into structured JSON (summary).

Post-training = everything done to a model after its expensive pre-training: supervised fine-tuning (SFT) teaches it the format and the domain from examples; preference optimisation (DPO) teaches it which of two answers is better. Companies do exactly this to turn base models into products (OpenAI, Meta, Google, AI2's Tulu 3, Hugging Face's Zephyr).

Where it is used in the real world: clinical documentation ("AI scribes" such as Nuance DAX, Abridge, Suki), medical education and exam prep, structured extraction from health records, and internal copilots at hospitals and insurers. Important: real clinical deployment needs regulatory approval and clinical validation; this project is an educational portfolio system, never a medical device.

0.2 The parts of this manual

Part	What you will do	Time estimate
0 Start Here	Understand the project, tools and costs	20 min
1 Prerequisites	Check skills, create accounts	1-2 h
2 Workspace	Hyperstack GPU VM, SSH key, VS Code, WinSCP, Python envs, FakeCloud, AWS CLI	2-3 h
3 Core Idea	The 20% of ideas that explain 80% of post-training	30 min
4 Basics	Data → .jsonl → first QLoRA SFT adapter (qa) → test → Hugging Face → S3	3-4 h (about 1.5 GPU-h)
5 Intermediate	Teacher model + distilabel synthetic data, three adapters, ROUGE-L / BERTScore / LLM judge	1 day (about 6 GPU-h)
6 Advanced	DPO, vLLM multi-LoRA hot-swap, FastAPI + LangSmith, Docker, ECR, EC2 (FakeCloud and AWS)	1-2 days (about 5 GPU-h)
7 Elite	Contamination check, significance tests, judge calibration, runtime adapter swaps, model card	4-6 h
8-14 Reference	Tools map, maintenance, troubleshooting, projects, cheat sheet, self-test, sources	reference

0.3 What you will be able to do at the end

- Collect open medical datasets, clean them, de-duplicate them and convert them to chat-format .jsonl that TRL accepts.
- Fine-tune Llama-3.1-8B-Instruct with QLoRA (Unsloth + TRL + PEFT) and then align it with DPO, and explain every hyper-parameter.
- Generate synthetic training data and preference pairs with distilabel and a teacher LLM, with quality filters.
- Evaluate honestly: accuracy, ROUGE-L, BERTScore, LLM-as-a-judge with position swap, bootstrap confidence intervals, contamination checks.
- Serve one base model with several hot-swappable LoRA adapters in vLLM behind a FastAPI service traced in LangSmith.
- Package it with Docker, push to (Fake)ECR and run it on a GPU machine on AWS, then tear everything down so the bill stops.

0.4 Environment map (every place you will work)

Place	What it is	What you do there
Your laptop: PowerShell	Windows terminal	create the SSH key, first SSH login
Your laptop: VS Code + Remote-SSH	editor that runs on the VM	edit files, run every command in its Terminal
Your laptop: WinSCP	drag-and-drop file transfer (SFTP)	download reports/adapters, upload your own data
Your laptop: web browser	websites	Hyperstack console, Hugging Face, LangSmith, AWS console
Hyperstack GPU VM (Ubuntu 24.04 + NVIDIA A100)	rented cloud computer	ALL training, serving, Docker, FakeCloud
FakeCloud (on the VM, port 4566)	free local AWS emulator	practise every AWS command at no cost
Real AWS (us-east-1)	Amazon's cloud	ECR registry, S3, IAM, Secrets Manager, CloudWatch, EC2 g6.xlarge

Hugging Face Hub	model/dataset hosting	download Llama and data, publish adapters and datasets
LangSmith	tracing website	see every API request, latency and output

Flow in one line: laptop (VS Code / WinSCP over SSH) → Hyperstack VM (data → SFT → DPO → eval → vLLM + FastAPI in Docker) → Hugging Face (artifacts) → FakeCloud for practice → AWS ECR + EC2 for the real deployment.

0.5 What each piece does

Component	Its job
Llama-3.1-8B-Instruct	the base model you post-train (8 billion parameters, 128k vocabulary, chat template with special header tokens)
Hugging Face datasets	downloads MedQA, MedMCQA, PubMedQA and clinical notes
.jsonl	one JSON object per line; the standard input format for TRL trainers
distilabel	builds synthetic-data pipelines (generate with a teacher LLM, rate with a judge LLM)
Qwen2.5-14B-Instruct	the teacher (writes explanations) and the judge (rates answers); Apache-2.0 licence
bitsandbytes (4-bit NF4)	compresses the frozen base weights to 4 bits: the "Q" in QLoRA
PEFT (LoRA)	adds small trainable matrices; only ~0.5% of parameters are trained
Unsloth	speeds up QLoRA training about 2x and cuts VRAM; patches Transformers/TRL
TRL (SFTTrainer, DPOTrainer)	Hugging Face's post-training library
vLLM	fast inference server with PagedAttention; serves many LoRA adapters on one base model
FastAPI	your public HTTP API (/v1/generate) in front of vLLM
LangSmith	records a trace for every request (inputs, outputs, latency)
Docker / Docker Compose	packages the API; runs vLLM + API together
FakeCloud	open-source AWS emulator (AGPL-3.0); same aws commands, endpoint http://localhost:4566
AWS ECR / S3 / IAM / Secrets Manager / CloudWatch / EC2	registry for images, storage for adapters, permissions, secrets, logs/alarms, the GPU server

0.6 What it costs (be honest with yourself before you start)

Item	Price (October 2026)	How much you need	Approx. cost
Hyperstack 1x A100-80G-PCIe	about US$1.35 per hour on demand [1] (verify on the pricing page)	about 13-15 hours over Parts 4-7	US$18-21
Hyperstack storage while hibernated	about US$0.10 per TB per hour for persistent storage [3]	a few days	under US$3
FakeCloud	free, open source [5]	always	US$0
AWS g6.xlarge (1x NVIDIA L4 24 GB)	about US$0.805 per hour in us-east-1 [12]	2-3 hours in Part 6	about US$2-3
AWS ECR, S3, Secrets Manager, CloudWatch	cents for this project	Part 6	under US$2
Hugging Face, LangSmith Developer plan	free (LangSmith Developer: about 5k traces per month [11])	always	US$0

> **Watch out:** Hyperstack keeps billing a VM in SHUTOFF state. Only HIBERNATED or deleted VMs stop GPU billing [3]. You will learn how in Section 2.12.

PART 1 - Prerequisites

1.1 Checklist

Item	MUST or NICE-TO-HAVE
Python basics: variables, lists, dicts, functions, f-strings, reading a traceback	MUST
JSON: objects, arrays, strings in double quotes	MUST
Linux terminal basics: cd, ls, pwd, cat, mkdir, nano or VS Code	MUST
What a GPU and VRAM are (memory on the graphics card)	MUST
HTTP APIs: URL, POST, JSON body, status codes	MUST
Git basics: clone, pull, commit	NICE-TO-HAVE
Docker basics: image vs container, ports	NICE-TO-HAVE (taught in Part 6)
Basic statistics: mean, confidence interval	NICE-TO-HAVE (taught in Part 7)
Medical vocabulary	NICE-TO-HAVE (the data teaches you)

1.2 Refreshers and 1-minute checks for every MUST item

Python basics. A dict maps keys to values; a list keeps order; a function groups steps. Check: without running it, what does this print?

```
row = {"task": "qa", "messages": [1, 2, 3]}
print(len(row["messages"]), row.get("source", "none"))
```

Answer: 3 none. If you got it right, you are ready.

JSON. JSON is text: objects use {}, arrays use [], strings MUST use double quotes. A .jsonl file has exactly one JSON object per line. Check: which line is invalid JSON?

```
{"role": "user", "content": "hi"}
{'role': 'user', 'content': 'hi'}
```

Answer: the second line (single quotes are Python, not JSON).

Linux terminal. pwd shows where you are, ls lists files, cd folder moves, cat file prints a file, mkdir -p a/b creates folders. Check: in a terminal type pwd, then mkdir -p test/one, then ls test. You should see one.

GPU and VRAM. A GPU runs the matrix maths of a neural network in parallel; VRAM is its memory. An 8B model in 16-bit needs about 16 GB just for weights (8 billion x 2 bytes); in 4-bit it needs about 5-6 GB. Check: how much VRAM do 14 billion parameters need in bfloat16? Answer: about 28 GB.

HTTP APIs. A client sends a request (method + URL + JSON body); the server answers with a status code (200 OK, 401 unauthorised, 422 invalid input, 500 server error) and a JSON body. Check: what status code should an API return when the API key is missing? Answer: 401.

1.3 Accounts and installs (do these now; some need a day to be approved)

What	Where (official link)	Cost	Which option to choose
Hugging Face account + access token	https://huggingface.co/join and https://huggingface.co/settings/tokens	free	token type: Write (you will upload adapters)
Llama 3.1 licence	https://huggingface.co/meta-llama/Llama-3.1-8B-Instruct	free	click the licence form and submit; approval is manual and can take hours
Hyperstack account	https://console.hyperstack.cloud	pay as you go	activate billing, add US$30-40 credit
LangSmith account	https://smith.langchain.com	free Developer plan	create an API key (Settings > API Keys)
AWS account (optional, for the blue column)	https://aws.amazon.com	pay as you go	region us-east-1; you will request GPU quota in Section 2.11
VS Code	https://code.visualstudio.com	free	Windows User Installer (x64)
VS Code extension Remote - SSH	inside VS Code: Extensions, search "Remote - SSH" (publisher Microsoft)	free	install the Microsoft one
WinSCP	https://winscp.net/eng/download.php	free	the installation package

OpenSSH client	built into Windows 11	free	nothing to install; test with ssh -V in PowerShell

Watch out: the Llama licence requires that any model you publish that was fine-tuned from Llama has a name starting with "Llama" and that you show "Built with Llama" [15]. This manual names your adapters Llama-3.1-8B-MedScriptAI-... for that reason.

PART 2 - Set Up Your Workspace

SECTION 2.1 - Create an SSH key on your laptop

WHERE: Windows 11 laptop > Start menu > type PowerShell > open Windows PowerShell

GOAL: one private key that stays on your laptop and one public key you give to Hyperstack.

Step 1. Type exactly this and press Enter:

```
ssh-keygen -t ed25519 -f $env:USERPROFILE\.ssh\hyperstack_ed25519 -C "medscriptai"
```

Step 2. When asked "Enter passphrase", press Enter twice (empty passphrase is simplest for learning).

You should see: "Your identification has been saved in C:\Users\YOUR_NAME\.ssh\hyperstack_ed25519".

Step 3. Show the public key:

```
Get-Content $env:USERPROFILE\.ssh\hyperstack_ed25519.pub
```

You should see one line that starts with ssh-ed25519 and ends with medscriptai.

Step 4. Select that whole line with the mouse and press Ctrl+C to copy it.

Watch out: never share the file WITHOUT .pub (the private key). Only the .pub line goes to Hyperstack.

CHECKPOINT: two files exist in C:\Users\YOUR_NAME\.ssh: hyperstack_ed25519 and hyperstack_ed25519.pub.

If not: if PowerShell says "ssh-keygen is not recognized", open Settings > System > Optional features > Add a feature > OpenSSH Client > Install, then repeat Step 1.

SECTION 2.2 - Hugging Face token and Llama licence

WHERE: web browser

GOAL: a Write token for uploads, and access to Llama 3.1.

Step 1. Open https://huggingface.co/settings/tokens.

Step 2. Click Create new token.

Step 3. Click the Write tab.

Step 4. In Token name type medscriptai.

Step 5. Click Create token.

Step 6. Click Copy and paste the token into a private note (it starts with hf_). You will not see it again.

Step 7. Open https://huggingface.co/meta-llama/Llama-3.1-8B-Instruct.

Step 8. Fill in the licence form at the top of the page and click Submit.

You should see: a message that your request is pending; later the page shows "You have been granted access".

Watch out: this manual downloads the ungated mirror unsloth/Llama-3.1-8B-Instruct so you are not blocked while waiting, but the Llama licence still applies to you; accepting it on Meta's page is the honest, licence-compliant step.

CHECKPOINT: you have an hf_ token saved and the licence request is submitted.

SECTION 2.3 - Hyperstack account, billing, environment and key

WHERE: web browser > https://console.hyperstack.cloud

GOAL: a funded account with an environment that holds your SSH key.

Step 1. Open https://authui.hyperstack.cloud/register and create the account.

Step 2. Open https://console.hyperstack.cloud/billing/activation and activate billing.

Step 3. Open https://console.hyperstack.cloud/billing and add credit (US$30-40 is enough for this manual).

Step 4. In the left menu click Virtual Machines.

Step 5. Click Deploy New Virtual Machine.

Step 6. In the Environment section click Create New Environment.

Step 7. Type the name medscriptai-env and save (keep the region it suggests).

Step 8. In the Environment section click Create New SSH Key.

Step 9. Click the Import SSH key tab.

Step 10. Select the environment medscriptai-env.

Step 11. Paste the public key line you copied in Section 2.1 into the key box.

Step 12. Type the name laptop-ed25519.

Step 13. Click Import.

You should see: the key laptop-ed25519 listed for medscriptai-env.

Watch out: Hyperstack warns that a fake look-alike website exists; only use hyperstack.cloud addresses [1].

CHECKPOINT: billing is active, credit is visible, and the key is imported into medscriptai-env.

SECTION 2.4 - Deploy the GPU virtual machine

WHERE: Hyperstack console > Virtual Machines > Deploy New Virtual Machine

GOAL: one Ubuntu 24.04 VM with an NVIDIA A100 80 GB, Docker and the NVIDIA driver pre-installed.

Step 1. Fill the form with the values in this table, one field at a time.

Field	Value	Why
Flavor	1x A100-80G-PCIe (or 1x L40 48 GB if A100 is sold out)	80 GB lets the teacher (14B) and the student (8B) run together in Parts 5-6
Environment	medscriptai-env	holds your key
Select OS Image	Server 24.04 LTS R570 CUDA 12.8 with Docker	newest widely available image; Docker + NVIDIA container toolkit included [4]
SSH key	laptop-ed25519	lets you log in
SSH Access	Enabled	opens TCP port 22 in the firewall
Public IP Address	Assign Public IP: on	you connect over the internet
VM name	medscriptai-gpu	easy to recognise

Step 2. Click Deploy.

Step 3. Wait until the Status column shows ACTIVE (usually a few minutes).

Step 4. Copy the address in the PUBLIC IP column into your note as VM_IP.

You should see: medscriptai-gpu, ACTIVE, and an IP address such as 38.80.123.45.

Watch out: billing starts when the VM becomes ACTIVE [2]. If you stop working for the day, go to Section 2.12.

If you see...	It means...	Do this next
The A100 flavor is greyed out	sold out in that region	choose 1x L40, or create another environment in a different region and import the key again
Only Server 24.04 LTS R595 CUDA 13.2 with Docker is offered	newer driver image	use it; later set VLLM_IMAGE=vllm/vllm-openai:v0.32.0 and VLLM_ENABLE_CUDA_COMPATIBILITY=0

CHECKPOINT: the VM is ACTIVE and you have VM_IP.

If not: if it stays in BUILD for more than 15 minutes, delete it and deploy again in another region.

SECTION 2.5 - First SSH login from PowerShell

WHERE: Windows PowerShell on your laptop

GOAL: prove the key, the firewall and the GPU all work.

Step 1. Type (replace VM_IP):

```
ssh -i $env:USERPROFILE\.ssh\hyperstack_ed25519 ubuntu@VM_IP
```

Step 2. When asked "Are you sure you want to continue connecting", type yes and press Enter.

You should see: a prompt like ubuntu@medscriptai-gpu:~$.

Step 3. Type:

```
nvidia-smi
```

You should see: a table with "NVIDIA A100 80GB PCIe", "Driver Version: 570.xx" and "CUDA Version: 12.8".

Step 4. Type:

```
df -h / /ephemeral
```

You should see: two lines: the root disk (/, about 100 GB) and the large ephemeral disk mounted at /ephemeral [6].

Step 5. Type exit and press Enter.

Placeholder	Replace with	Example
VM_IP	the PUBLIC IP from Section 2.4	38.80.123.45
YOUR_NAME	your Windows user folder name	rahul

Watch out: /ephemeral is fast but is WIPED when you hibernate or delete the VM [6]. This manual stores only re-downloadable caches there; your code, data and adapters live in /home/ubuntu (root disk) and on the Hugging Face Hub.

CHECKPOINT: nvidia-smi shows the GPU and the driver line shows 570 or newer.

If not: "Permission denied (publickey)" means the wrong key file or user: the user is ubuntu and the key is the file without .pub. "Connection timed out" means SSH Access was not enabled: open the VM > Firewall tab > enable SSH access.

SECTION 2.6 - Connect VS Code to the VM (Remote-SSH)

WHERE: VS Code on your laptop

GOAL: edit files and run terminals ON the VM from your laptop.

Step 1. Press Ctrl+Shift+X to open Extensions.

Step 2. Search Remote - SSH and click Install on the one published by Microsoft.

Step 3. Press F1.

Step 4. Type Remote-SSH: Open SSH Configuration File and press Enter.

Step 5. Choose C:\Users\YOUR_NAME\.ssh\config.

Step 6. Paste this at the end of the file (replace VM_IP and YOUR_NAME):

```
Host medscriptai
    HostName VM_IP
    User ubuntu
    IdentityFile C:\Users\YOUR_NAME\.ssh\hyperstack_ed25519
    ServerAliveInterval 60
```

Step 7. Press Ctrl+S to save.

Step 8. Press F1, type Remote-SSH: Connect to Host and press Enter.

Step 9. Click medscriptai.

Step 10. If asked for the platform, click Linux.

Step 11. Open the menu Terminal > New Terminal.

You should see: the bottom-left green box says SSH: medscriptai and the terminal prompt is ubuntu@medscriptai-gpu.

Watch out: from now on, "VS Code Terminal" in this manual means this remote terminal on the VM, not your laptop.

CHECKPOINT: typing hostname in the VS Code Terminal prints medscriptai-gpu.

If not: if VS Code hangs on "Setting up SSH host", press F1 > Remote-SSH: Kill VS Code Server on Host, then connect again.

SECTION 2.7 - Connect WinSCP to the VM (file transfer)

WHERE: WinSCP on your laptop

GOAL: drag files between your laptop and the VM.

Step 1. Open WinSCP. The Login window appears.

Step 2. Click New Site.

Step 3. Fill the fields:

Field	Value	Why
File protocol	SFTP	file transfer over your SSH connection

Host name	VM_IP	your VM
Port number	22	SSH port
User name	ubuntu	default user on Hyperstack Ubuntu images
Password	leave empty	you use a key

Step 4. Click Advanced....

Step 5. In the left list click SSH > Authentication.

Step 6. Next to Private key file click ... and choose C:\Users\YOUR_NAME\.ssh\hyperstack_ed25519 (set the file filter to All Files).

Step 7. When WinSCP asks to convert the key to PuTTY format, click OK, then Save (it saves hyperstack_ed25519.ppk).

Step 8. Click OK to close Advanced.

Step 9. Click Save, type medscriptai as the site name, click OK.

Step 10. Click Login.

You should see: your laptop files on the left, /home/ubuntu on the right.

CHECKPOINT: you can see /home/ubuntu on the right panel.

If not: "Server refused our key" = wrong key file; choose the .ppk WinSCP created.

SECTION 2.8 - Prepare the VM: folders, caches, code and Python environments

WHERE: VS Code Terminal (connected to the VM)

GOAL: the project code on the VM and three isolated Python environments.

Step 1. Point the Hugging Face cache to the big ephemeral disk:

```
sudo mkdir -p /ephemeral/hf-cache && sudo chown ubuntu:ubuntu /ephemeral/hf-cache
```

Step 2. Make that setting permanent for every new terminal:

```
echo 'export HF_HOME=/ephemeral/hf-cache' >> ~/.bashrc
```

Step 3. Save your Hugging Face token (replace hf_YOUR_TOKEN):

```
echo 'export HF_TOKEN=hf_YOUR_TOKEN' >> ~/.bashrc && source ~/.bashrc
```

Step 4. Install uv (a very fast Python package manager from Astral):

```
curl -LsSf https://astral.sh/uv/install.sh | sh && source ~/.bashrc
```

Step 5. Install tmux (keeps long jobs running if your laptop disconnects):

```
sudo apt-get update && sudo apt-get install -y tmux git unzip jq
```

Step 6. Download the project code:

```
cd ~ && git clone -b claude/determined-carson-9p7115 https://github.com/kashyapphassan2000-netizen/-MEDSCRIPTAI.git
```

Step 7. Enter the project folder:

```
cd ~/-MEDSCRIPTAI
```

Step 8. Create the training environment (Python 3.12):

```
uv venv .venv-train --python 3.12
```

Step 9. Activate it:

```
source .venv-train/bin/activate
```

Step 10. Install Unsloth and its exact matching libraries (takes 3-6 minutes):

```
uv pip install -r requirements/train.txt --torch-backend cu126
```

You should see: "Installed ... packages" including torch==2.14.1+cu126, unsloth==2026.10.7, trl==1.15.0, transformers==5.17.0.

Step 11. Deactivate it:

```
deactivate
```

Step 12. Create and fill the synthetic-data environment:

```
uv venv .venv-synth --python 3.12 && uv pip install --python .venv-synth -r requirements/synth.txt
```

Step 13. Create and fill the evaluation/API environment:

```
uv venv .venv-eval --python 3.12 && uv pip install --python .venv-eval -r requirements/eval.txt --torch-backend cu126
```

Line / piece	What it does
HF_HOME	where Hugging Face stores downloaded models and datasets (about 60 GB by Part 6)
uv venv .venv-train --python 3.12	creates an isolated Python 3.12 folder so packages of different tools never clash
--torch-backend cu126	picks the PyTorch build for CUDA 12.6, which runs on your R570 driver (CUDA 12.8); PyTorch 2.14 has no cu128 build
three environments	Unsloth pins transformers<=5.17 and trl<=1.15; distilabel 1.5.3 (last release Jan 2025) needs datasets<4 and openai<2; evaluation uses the newest transformers. One environment cannot satisfy all three

Watch out: always check which environment is active: the prompt starts with (.venv-train), (.venv-synth) or (.venv-eval). Running a script in the wrong one is the number-one beginner error in this project.

CHECKPOINT: ls -a shows .venv-train, .venv-synth and .venv-eval in ~/-MEDSCRIPTAI.

If not: if Step 10 fails with "No solution found", run uv --version (needs 0.8 or newer) and run uv self update, then repeat.

SECTION 2.9 - Tiny test: does PyTorch see the GPU?

WHERE: VS Code Terminal, folder ~/-MEDSCRIPTAI

GOAL: prove the GPU, CUDA and bfloat16 work before any real job.

Step 1. Activate the training environment:

```
source .venv-train/bin/activate
```

Step 2. Run this one-line check:

```
python -c "import torch; print(torch.__version__, torch.cuda.is_available(), torch.cuda.get_device_name(0),
torch.cuda.is_bf16_supported(), round(torch.cuda.get_device_properties(0).total_memory/1e9), 'GB')"
```

You should see: 2.14.1+cu126 True NVIDIA A100 80GB PCIe True 85 GB

Step 3. Run the data validator on the (still empty) data folder to check the scripts import correctly:

```
cd scripts && python -c "import common; print(common.BASE_MODEL)" && cd ..
```

You should see: unsloth/Llama-3.1-8B-Instruct

Piece	What it does
torch.cuda.is_available()	True means PyTorch can talk to the NVIDIA driver
get_device_name(0)	the name of GPU number 0
is_bf16_supported()	bfloat16 = 16-bit format with float32's range; A100/L40/H100 support it, older T4/V100 do not
total_memory/1e9	VRAM in GB (80 GB cards report about 85 billion bytes)

CHECKPOINT: the line ends with True ... True 85 GB (or 48 GB on an L40).

If not: False for cuda means you installed the CPU build; run uv pip install -r requirements/train.txt --torch-backend cu126 --reinstall.

SECTION 2.10 - Install FakeCloud (free AWS) on the VM, and the AWS CLI

WHERE: VS Code Terminal, folder ~/-MEDSCRIPTAI

GOAL: the aws command works against FakeCloud now, and against real AWS when you want.

Step 1. Install the AWS CLI version 2 (used for BOTH columns):

```
cd /tmp && curl -s "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o awscliv2.zip && unzip -q awscliv2.zip
&& sudo ./aws/install && cd ~/-MEDSCRIPTAI
```

Step 2. Check it:

```
aws --version
```

You should see: aws-cli/2.x.x Python/3.x Linux/...

Step 3. Install the FakeCloud binary (pinned version; the installer's "latest" lookup can hit GitHub API limits):

```
curl -fsSL https://fakecloud.dev/install.sh | bash -s -- --version v0.51.0
```

Step 4. Start FakeCloud in the background, saving its state to disk:

```
mkdir -p ~/fakecloud-data && nohup fakecloud --storage-mode persistent --data-path ~/fakecloud-data > ~/fakecloud.log
2>&1 &
```

Step 5. Check it is healthy:

```
curl -s http://localhost:4566/_fakecloud/health | head -c 120
```

You should see: {"status":"ok","version":"0.51.0","services":[...

Step 6. Do the setup for each column:

Step	FakeCloud (free practice)	Real AWS (same step)
Credentials	`export AWS_ACCESS_KEY_ID=test AWS_SECRET_ACCESS_KEY=test` (any values work)	AWS console > IAM > Users > Create user medscriptai-admin > attach AdministratorAccess > Security credentials > Create access key (Command Line Interface). Then run `aws configure` and paste the two keys
Region	`export AWS_DEFAULT_REGION=us-east-1`	in `aws configure` type us-east-1
Endpoint	`export AWS_ENDPOINT_URL=http://localhost:4566` (every aws command now goes to FakeCloud)	no endpoint: run `unset AWS_ENDPOINT_URL` so commands go to real AWS
Test	`aws sts get-caller-identity`	`aws sts get-caller-identity`
You should see	"Account": "123456789012" (FakeCloud's fake account)	your real 12-digit account number
Safety	nothing to do	AWS console > Billing and Cost Management > Budgets > Create budget > Zero spend or Monthly US$10 with e-mail alerts. Also turn on MFA for the root user

Watch out: the only difference between the two columns is the AWS_ENDPOINT_URL variable and the credentials. That is the whole idea of FakeCloud: same commands, no bill. Before any real-AWS step, run echo $AWS_ENDPOINT_URL; it must print an empty line.

Watch out: never use your AWS root user's keys. Never paste access keys into code or Git.

CHECKPOINT: aws sts get-caller-identity returns 123456789012 with the FakeCloud variables set.

If not: "Could not connect to the endpoint URL" means FakeCloud is not running; repeat Step 4 and check tail ~/fakecloud.log.

SECTION 2.11 - (Real AWS only) Request GPU quota now

WHERE: web browser > AWS console (region us-east-1)

GOAL: permission to start one GPU machine in Part 6. New accounts have a GPU quota of 0, and approval can take hours to days.

Step 1. Open https://console.aws.amazon.com/servicequotas.

Step 2. Click AWS services, then search and click Amazon Elastic Compute Cloud (Amazon EC2).

Step 3. Search for Running On-Demand G and VT instances.

Step 4. Click it, then click Request increase at account level.

Step 5. Type 8 (vCPUs; a g6.xlarge uses 4) and click Request.

You should see: a request with status Pending.

CHECKPOINT: the request exists. Continue with Part 3 and 4 while AWS reviews it.

SECTION 2.12 - Stop paying when you stop working (do this EVERY time)

WHERE: Hyperstack console > Virtual Machines

GOAL: GPU billing stops; your root disk is kept.

Step 1. In the VS Code Terminal, make sure no job is running (tmux ls shows nothing important).

Step 2. Push anything valuable to the Hugging Face Hub (adapters, datasets) — Part 4 shows how.

Step 3. In the console, click the three dots next to medscriptai-gpu.

Step 4. Click Hibernate.

Step 5. Next time, click Restore (the public IP may change: update HostName in your SSH config and WinSCP).

VM state	GPU billed?	Data kept?
ACTIVE	yes	yes
SHUTOFF	YES (resources stay reserved) [3]	yes
HIBERNATED	no (only storage is billed) [3]	root disk yes, /ephemeral NO [6]
Deleted	no	no

Watch out: after Restore, /ephemeral is empty: the Hugging Face cache re-downloads automatically (about 10 minutes); run mkdir -p /ephemeral/hf-cache first. Restart FakeCloud with Step 4 of Section 2.10.

CHECKPOINT: the VM shows HIBERNATED when you are not working.

PART 3 - The Core Idea

3.1 One analogy

Llama-3.1-8B-Instruct is a smart new graduate doctor. SFT is residency: you show thousands of worked examples ("here is a question, here is a correct answer in our house style") and it imitates them. DPO is the senior consultant reviewing two of the resident's notes and saying "this one is better than that one"; the resident shifts towards the better habits without forgetting everything else. LoRA means you do not rewrite the doctor's whole brain; you give them a slim specialty notebook (an adapter) that changes how they answer. QLoRA compresses the frozen textbook to fit in your backpack (4-bit) while the notebook stays full precision. vLLM multi-LoRA is one doctor carrying three specialty notebooks and picking the right one for each patient in milliseconds.

3.2 The 20% of ideas that explain 80% of the practice

- **Tokens and the chat template.** Models read token IDs, not text. Llama-3.1 wraps each turn as <|start_header_id|>role<|end_header_id|> ... <|eot_id|>. Training and serving MUST use the same template, or quality silently drops.
- **Next-token loss with masking.** SFT minimises cross-entropy on the next token, but only on the assistant's answer; system and user tokens are masked to -100 so the model does not learn to write questions.
- **LoRA.** A frozen weight W gets a trainable low-rank update B x A of rank r (we use r=16). Effective change = (alpha/r) x B x A. You train about 42 million parameters instead of 8 billion.
- **QLoRA.** The frozen base is stored in 4-bit NF4; LoRA weights stay in 16-bit. Memory falls from about 16 GB to about 6 GB for weights, so an 8B model trains on one GPU.
- **Learning rate and epochs.** LoRA SFT works at about 2e-4 for 1-3 epochs; DPO needs about 5e-6 (40x smaller). Too high = loss spikes and broken grammar; too many epochs = memorisation (validation loss goes up while training loss keeps falling).
- **Data quality beats data size.** De-duplicate, filter invalid rows (25% of the clinical summaries are broken JSON), and never let test questions leak into training (contamination).
- **Preference optimisation.** DPO needs (prompt, chosen, rejected). Its loss starts at ln 2 = 0.693 and falls as the model prefers chosen answers. beta (0.1) controls how far it may drift from the reference (SFT) model.
- **Evaluation is a system, not a number.** Exact accuracy for multiple choice, overlap (ROUGE-L) and meaning (BERTScore) for text, an LLM judge for quality, and statistics to know whether a difference is real.
- **Serving.** vLLM keeps the 16 GB base model once in GPU memory and adds the small adapter matrices per request, so three adapters cost almost the same as one model. Clients choose an adapter through the model field of an OpenAI-style request.
- **Production.** A stateless API container in front of the model server, secrets outside the code, logs and traces, an image registry (ECR), a GPU host (EC2), least-privilege IAM, and a teardown plan.

PART 4 - BASICS: your first medical fine-tune (data → .jsonl → QLoRA SFT → test → publish)

4.0 Concept

What: download four open medical datasets, clean them, convert them into chat-format .jsonl, and train your first LoRA adapter (qa) on 2,000 exam questions with Unsloth QLoRA.

Why: this is the smallest complete loop of post-training. Every later part reuses these exact files and scripts at larger scale.

When: always start a new domain project this way: a small, fast run proves the data format, the template and the GPU setup before you spend hours of GPU money.

Cost and time: about 1.5 GPU-hours on 1x A100 (about US$2). Training itself takes roughly 10-15 minutes (estimate; your log shows the real speed).

SECTION 4.1 - Download the datasets

WHERE: VS Code Terminal on the VM, folder ~/-MEDSCRIPTAI

GOAL: seven raw .jsonl files in data/raw, untouched copies of the sources.

Step 1. Start a tmux session so the job survives a disconnect:

```
tmux new -s data
```

Step 2. Activate the training environment:

```
cd ~/-MEDSCRIPTAI && source .venv-train/bin/activate
```

Step 3. Move into the scripts folder:

```
cd scripts
```

Step 4. Run the download script:

```
python 01_download_data.py
```

You should see (after 2-5 minutes) seven lines like: medqa_train 10178 rows ← GBaker/MedQA-USMLE-4-options [train], medmcqa_train 20000 rows, medmcqa_dpo 3000 rows, medmcqa_val 4183 rows, pubmedqa 1000 rows, clinical_notes 12000 rows.

Step 5. Detach from tmux (the job keeps running): press Ctrl+B, release, then press D. Re-attach later with tmux attach -t data.

FILE scripts/common.py (shared settings used by every script):

```
"""Shared settings and helpers used by every MedScriptAI script."""
import json                     # read/write JSON text
import os                       # read environment variables
from pathlib import Path        # build file paths that work on any OS

# Project folders. ROOT is the repo folder (one level above scripts/).
ROOT = Path(__file__).resolve().parent.parent
DATA = ROOT / "data"            # all datasets live here
RAW = DATA / "raw"              # untouched downloads
SFT = DATA / "sft"              # chat-format .jsonl for SFT
PREF = DATA / "pref"            # preference pairs for DPO
OUT = ROOT / "outputs"          # adapters, predictions, reports

# Base model. The unsloth mirror is ungated; you still must accept Meta's licence.
BASE_MODEL = os.getenv("BASE_MODEL", "unsloth/Llama-3.1-8B-Instruct")
BASE_MODEL_4BIT = os.getenv("BASE_MODEL_4BIT", "unsloth/Llama-3.1-8B-Instruct-bnb-4bit")

# One system prompt per task = one LoRA adapter per task.
SYSTEM = {
    "qa": (
        "You are MedScriptAI, a careful medical education assistant. "
        "Answer the multiple-choice question. First line: 'Answer: <letter>) <option>'. "
        "Then give a short explanation. Educational use only, not medical advice."
    ),
    "notes": (
        "You are MedScriptAI, a clinical documentation assistant. "
```

```
        "Write a concise clinical case note from the doctor-patient conversation. "
        "Use only facts stated in the conversation. Do not invent findings."
    ),
    "summary": (
        "You are MedScriptAI, a clinical information extractor. "
        "Return ONLY valid JSON that summarises the clinical note. "
        "Include only fields that are stated in the note."
    ),
}


def write_jsonl(path, rows):
    """Write a list of dicts to a .jsonl file (one JSON object per line)."""
    path = Path(path)
    path.parent.mkdir(parents=True, exist_ok=True)       # create folders if missing
    with path.open("w", encoding="utf-8") as f:
        for row in rows:
            f.write(json.dumps(row, ensure_ascii=False) + "\n")
    print(f"wrote {len(rows):>6} rows -> {path.relative_to(ROOT)}")


def read_jsonl(path):
    """Read a .jsonl file back into a list of dicts."""
    with Path(path).open(encoding="utf-8") as f:
        return [json.loads(line) for line in f if line.strip()]


def chat(task, user, assistant=None):
    """Build the 'messages' list in the chat format TRL and vLLM understand."""
    msgs = [{"role": "system", "content": SYSTEM[task]},
            {"role": "user", "content": user}]
    if assistant is not None:
        msgs.append({"role": "assistant", "content": assistant})
    return msgs
```

Line	What it does (plain English)
`import json`, `import os`, `from pathlib import Path`	standard Python tools: JSON text, environment variables, file paths that work on every OS
`ROOT = Path(__file__).resolve().parent.parent`	the repo folder = two levels above this file (scripts/common.py → repo)
`DATA`, `RAW`, `SFT`, `PREF`, `OUT`	one variable per folder so no script hard-codes paths
`BASE_MODEL = os.getenv("BASE_MODEL", "unsloth/Llama-3.1-8B-Instruct")`	the bf16 base model; you can override it with an environment variable without editing code
`BASE_MODEL_4BIT = ...-bnb-4bit`	the same model already quantised to 4-bit NF4 by Unsloth (5.7 GB download instead of 16 GB)
`SYSTEM = {...}`	one system prompt per task. Each task = one adapter. The API (Part 6) imports this SAME dict, so training and serving prompts can never differ
`def write_jsonl(path, rows)`	writes a list of dicts, one JSON object per line; `mkdir(parents=True, exist_ok=True)` creates missing folders
`json.dumps(row, ensure_ascii=False)`	turns a dict into JSON text; ensure_ascii=False keeps symbols like °C readable
`print(f"wrote {len(rows):>6} rows -> ...")`	`:>6` right-aligns the number in 6 characters so the log lines up
`def read_jsonl(path)`	reads the file back; `if line.strip()` skips empty lines
`def chat(task, user, assistant=None)`	builds the messages list: system, user, and (for training rows) the assistant answer. This is the conversational format TRL and vLLM understand

FILE scripts/01_download_data.py:

```
"""Step 1: download the open medical datasets and save them untouched as .jsonl."""
import argparse                                  # read command-line options
from datasets import load_dataset                # Hugging Face dataset loader
from common import RAW                           # data/raw folder

# name on the Hub, config, split, file we save, rows to keep as (start, end) after a seeded shuffle
SOURCES = [
    ("GBaker/MedQA-USMLE-4-options", None, "train", "medqa_train", None),
    ("GBaker/MedQA-USMLE-4-options", None, "test", "medqa_test", None),
    ("openlifescienceai/medmcqa", None, "train", "medmcqa_train", (0, 20000)),
```

```
    ("openlifescienceai/medmcqa", None, "train", "medmcqa_dpo", (20000, 23000)),  # unseen prompts for DPO
    ("openlifescienceai/medmcqa", None, "validation", "medmcqa_val", None),
    ("qiaojin/PubMedQA", "pqa_labeled", "train", "pubmedqa", None),
    ("AGBonnet/augmented-clinical-notes", None, "train", "clinical_notes", (0, 12000)),
]

parser = argparse.ArgumentParser()
parser.add_argument("--limit", type=int, default=None, help="tiny test run: keep N rows per file")
args = parser.parse_args()

RAW.mkdir(parents=True, exist_ok=True)            # make data/raw if it does not exist
for name, config, split, out_name, rows in SOURCES:
    ds = load_dataset(name, config, split=split)  # downloads once, then uses the local cache
    if rows:                                      # same seed = same order, so slices never overlap
        ds = ds.shuffle(seed=42).select(range(rows[0], min(rows[1], len(ds))))
    if args.limit:                                # --limit: tiny test run
        ds = ds.select(range(min(args.limit, len(ds))))
    path = RAW / f"{out_name}.jsonl"
    ds.to_json(str(path), lines=True, force_ascii=False)  # one JSON object per line
    print(f"{out_name:16} {len(ds):>6} rows  <- {name} [{split}]")
```

Line	What it does (plain English)
`import argparse`	lets you pass options such as --limit on the command line
`from datasets import load_dataset`	Hugging Face's loader: downloads once, caches in HF_HOME, returns a table
`from common import RAW`	the data/raw folder from common.py
`SOURCES = [...]`	the shopping list: Hub name, config, split, output file name, and which rows to keep
`("GBaker/MedQA-USMLE-4-options", None, "train", "medqa_train", None)`	US medical licensing exam questions with 4 options (CC-BY-4.0); None = keep all rows
`("openlifescienceai/medmcqa", ..., (0, 20000))`	Indian medical entrance exam questions with explanations (Apache-2.0); keep rows 0-19,999 after shuffling
`(..., "medmcqa_dpo", (20000, 23000))`	3,000 DIFFERENT MedMCQA questions kept aside for DPO in Part 6, so DPO prompts are new to the SFT model
`("qiaojin/PubMedQA", "pqa_labeled", ...)`	1,000 research-abstract questions with yes/no/maybe answers (MIT)
`("AGBonnet/augmented-clinical-notes", ..., (0, 12000))`	12,000 clinical notes with a doctor-patient conversation and a JSON summary (MIT)
`parser.add_argument("--limit", ...)`	optional tiny run for testing: keep only N rows per file
`RAW.mkdir(parents=True, exist_ok=True)`	create data/raw if needed; no error if it already exists
`ds = load_dataset(name, config, split=split)`	download (first time) or load from cache
`ds.shuffle(seed=42).select(range(rows[0], min(rows[1], len(ds))))`	shuffle with a fixed seed so the order is identical every run, then take a slice. Same seed = same order = the SFT slice and the DPO slice never overlap
`if args.limit: ds = ds.select(range(...))`	only used for quick tests
`ds.to_json(str(path), lines=True, force_ascii=False)`	save as .jsonl (lines=True means one object per line)
`print(f"{out_name:16} ...")`	progress line; `:16` pads the name to 16 characters

Watch out: if you see "429 Too Many Requests", you are being rate-limited by the Hub: make sure HF_TOKEN is set (echo $HF_TOKEN) and run the script again; finished files are cached.

CHECKPOINT: ls ../data/raw shows clinical_notes.jsonl, medmcqa_dpo.jsonl, medmcqa_train.jsonl, medmcqa_val.jsonl, medqa_test.jsonl, medqa_train.jsonl, pubmedqa.jsonl.

If not: re-attach with tmux attach -t data and read the last error line.

SECTION 4.2 - Clean the data and convert it to chat .jsonl

WHERE: VS Code Terminal, folder ~/-MEDSCRIPTAI/scripts, (.venv-train) active

GOAL: train/val/test .jsonl files for qa, notes and summary, plus held-out DPO prompts.

Step 1. Run:

```
python 02_build_sft_jsonl.py
```

You should see lines such as: wrote 300 rows → data/sft/qa_val.jsonl, wrote ~30000 rows → data/sft/qa_train.jsonl, wrote 500 rows → data/sft/qa_test.jsonl, wrote ~2950 rows → data/pref/qa_dpo.jsonl, then notes/summary test 200, val 200, dpo 1000 and train ~7000 each (exact numbers depend on how many rows pass the filters).

Step 2. Look at one training line in a readable way:

```
head -n 1 ../data/sft/qa_train.jsonl | python -m json.tool
```

You should see a JSON object with "task": "qa", "source", "answer_letter", "question_key" and "messages" holding three dicts with roles system, user and assistant. The assistant content starts with Answer: and a letter.

FILE scripts/02_build_sft_jsonl.py:

```
"""Step 2: clean the raw data and convert it to chat-format .jsonl (one file per task and split)."""
import json                                     # parse the JSON summaries
import random                                   # repeatable shuffling
import re                                       # simple text clean-up
from common import RAW, SFT, PREF, chat, read_jsonl, write_jsonl

random.seed(42)                                 # same split every time you run it
MAX_CHARS = 7000                                # ~1,750 tokens: fits max_seq_length 2048
LETTERS = "ABCD"


def clean(text):
    """Collapse repeated spaces/newlines and strip the ends."""
    return re.sub(r"[ \t]+", " ", re.sub(r"\n{3,}", "\n\n", str(text or ""))).strip()


def key(text):
    """Normalised text used to find duplicates (lower-case, letters and digits only)."""
    return re.sub(r"[^a-z0-9]", "", text.lower())[:300]


def mcq_prompt(question, options):
    """Question + lettered options, the exact user message format for the qa adapter."""
    lines = [f"{letter}) {text}" for letter, text in options]
    return f"{clean(question)}\n\nOptions:\n" + "\n".join(lines)


def short(text, limit=900):
    """Cut long explanations at the last full stop before `limit` characters."""
    text = clean(text)
    if len(text) <= limit:
        return text
    cut = text[:limit].rsplit(".", 1)[0]
    return cut + "."


# ---------- QA task: MedQA + MedMCQA + PubMedQA ----------
def from_medqa(row):
    opts = sorted(row["options"].items())          # [("A", "..."), ("B", "..."), ...]
    letter = row["answer_idx"]
    answer = f"Answer: {letter}) {clean(row['answer'])}"   # no explanation in MedQA (Part 5 adds one)
    return {"task": "qa", "source": "medqa", "answer_letter": letter,
            "question_key": key(row["question"]),
            "messages": chat("qa", mcq_prompt(row["question"], opts), answer)}


def from_medmcqa(row):
    if row.get("choice_type") != "single" or row.get("cop") not in (0, 1, 2, 3):
        return None                                # skip multi-answer or broken rows
    opts = list(zip(LETTERS, [row["opa"], row["opb"], row["opc"], row["opd"]]))
    letter = LETTERS[row["cop"]]
    answer = f"Answer: {letter}) {clean(opts[row['cop']][1])}"
    if row.get("exp") and len(clean(row["exp"])) > 40:
        answer += f"\nExplanation: {short(row['exp'])}"
    return {"task": "qa", "source": "medmcqa", "answer_letter": letter,
            "question_key": key(row["question"]),
            "messages": chat("qa", mcq_prompt(row["question"], opts), answer)}


def from_pubmedqa(row):
    context = clean(" ".join(row["context"]["contexts"]))
```

```
    opts = [("A", "yes"), ("B", "no"), ("C", "maybe")]
    letter = {"yes": "A", "no": "B", "maybe": "C"}[row["final_decision"]]
    question = f"Context: {context}\n\nQuestion: {row['question']}"
    answer = f"Answer: {letter}) {row['final_decision']}\nExplanation: {short(row['long_answer'])}"
    return {"task": "qa", "source": "pubmedqa", "answer_letter": letter,
            "question_key": key(row["question"]),
            "messages": chat("qa", mcq_prompt(question, opts), answer)}


# ---------- notes + summary tasks: augmented clinical notes ----------
def compact(value):
    """Drop empty fields ("None", "", [], {}) so the target JSON is short and factual."""
    if isinstance(value, dict):
        out = {k: compact(v) for k, v in value.items()}
        return {k: v for k, v in out.items() if v not in (None, "", [], {})}
    if isinstance(value, list):
        out = [compact(v) for v in value]
        return [v for v in out if v not in (None, "", [], {})]
    if isinstance(value, str) and value.strip().lower() in ("none", "n/a", ""):
        return None
    return value


def from_clinical(row):
    """One raw row gives one 'notes' example and one 'summary' example (or None if broken)."""
    conversation, note = clean(row["conversation"]), clean(row["note"])
    try:
        summary = compact(json.loads(row["summary"]))   # ~25% of summaries are invalid JSON
    except (json.JSONDecodeError, TypeError):
        return None
    if not conversation or not note or not summary:
        return None
    summary_text = json.dumps(summary, ensure_ascii=False)
    notes_ex = {"task": "notes", "source": "clinical_notes", "id": row["idx"],
                "messages": chat("notes", conversation, note)}
    summary_ex = {"task": "summary", "source": "clinical_notes", "id": row["idx"],
                  "messages": chat("summary", note, summary_text)}
    return notes_ex, summary_ex


def fits(example):
    """Keep only examples short enough for max_seq_length=2048."""
    return sum(len(m["content"]) for m in example["messages"]) <= MAX_CHARS


def build(rows, fn):
    """Apply a converter, drop None/too-long rows."""
    out = [fn(r) for r in rows]
    return [x for x in out if x is not None]


# ---- QA: build, de-duplicate, split ----
def sample(rows, n):
    """Random n rows (or all of them if there are fewer than n)."""
    return random.sample(rows, min(n, len(rows)))


qa_test = (sample(build(read_jsonl(RAW / "medqa_test.jsonl"), from_medqa), 200)
           + sample(build(read_jsonl(RAW / "medmcqa_val.jsonl"), from_medmcqa), 200))
pubmed = build(read_jsonl(RAW / "pubmedqa.jsonl"), from_pubmedqa)
random.shuffle(pubmed)
qa_test += pubmed[:100]                                  # 100 PubMedQA rows only for testing
qa_pool = (build(read_jsonl(RAW / "medqa_train.jsonl"), from_medqa)
           + build(read_jsonl(RAW / "medmcqa_train.jsonl"), from_medmcqa)
           + pubmed[100:])

test_keys = {x["question_key"] for x in qa_test}         # decontamination: never train on test questions
seen, qa_clean = set(), []
for ex in qa_pool:
    k = ex["question_key"]
    if k in seen or k in test_keys or not fits(ex):
        continue                                         # duplicate, leaked, or too long
    seen.add(k)
    qa_clean.append(ex)
random.shuffle(qa_clean)
write_jsonl(SFT / "qa_val.jsonl", qa_clean[:300])
```

```
write_jsonl(SFT / "qa_train.jsonl", qa_clean[300:])
write_jsonl(SFT / "qa_test.jsonl", qa_test)

# DPO prompts: 3,000 MedMCQA questions the SFT model never saw (and not in the test set)
dpo_qa = [x for x in build(read_jsonl(RAW / "medmcqa_dpo.jsonl"), from_medmcqa)
          if x["question_key"] not in seen and x["question_key"] not in test_keys and fits(x)]
write_jsonl(PREF / "qa_dpo.jsonl", dpo_qa)

# ---- notes + summary: one shared split so a test note is never a training note ----
pairs = [p for p in build(read_jsonl(RAW / "clinical_notes.jsonl"), from_clinical)
         if fits(p[0]) and fits(p[1])]
random.shuffle(pairs)
splits = {"test": pairs[:200], "val": pairs[200:400], "dpo": pairs[400:1400], "train": pairs[1400:]}
for split, rows in splits.items():
    folder = PREF if split == "dpo" else SFT              # DPO prompts are held out from SFT training
    write_jsonl(folder / f"notes_{split}.jsonl", [p[0] for p in rows])
    write_jsonl(folder / f"summary_{split}.jsonl", [p[1] for p in rows])
```

Line	What it does (plain English)
`import json, random, re`	JSON parsing, repeatable random sampling, regular expressions for text clean-up
`from common import RAW, SFT, PREF, chat, read_jsonl, write_jsonl`	folders and helpers from common.py
`random.seed(42)`	fixes the random generator, so the train/test split is identical every time (reproducibility)
`MAX_CHARS = 7000`	about 1,750 tokens (rule of thumb: 1 token is about 4 characters of English); keeps every example under max_seq_length 2048
`LETTERS = "ABCD"`	maps answer index 0-3 to a letter
`def clean(text)`	turns None into "", squeezes 3+ newlines into 2 and repeated spaces/tabs into 1, trims the ends
`def key(text)`	a fingerprint for de-duplication: lower-case, keep only letters and digits, first 300 characters. "What is X?" and "what is x" get the same key
`def mcq_prompt(question, options)`	builds the user message: question, blank line, "Options:", then lines like "A) Ampicillin". The API expects users to send this same shape
`def short(text, limit=900)`	shortens long explanations at the last full stop before 900 characters, so targets stay focused
`def from_medqa(row)`	converts one MedQA row
`opts = sorted(row["options"].items())`	the options dict {"A": ..., "B": ...} becomes a sorted list of (letter, text) pairs
`answer = f"Answer: {letter}) {clean(row['answer'])}"`	the target answer, e.g. Answer: D) Nitrofurantoin. MedQA has no explanation; Part 5 adds one with a teacher model
`return {"task": ..., "answer_letter": ..., "question_key": ..., "messages": chat(...)}`	one training example; answer_letter is kept for scoring accuracy later, question_key for de-duplication
`def from_medmcqa(row)`	converts one MedMCQA row
`if row.get("choice_type") != "single" or row.get("cop") not in (0, 1, 2, 3): return None`	drops multi-answer questions and rows with a missing correct option (cop = correct option index)
`opts = list(zip(LETTERS, [row["opa"], ...]))`	pairs A-D with option texts opa-opd
`if row.get("exp") and len(clean(row["exp"])) > 40`	adds the expert explanation only when it is real text (more than 40 characters)
`def from_pubmedqa(row)`	converts PubMedQA: the abstract becomes "Context:", the answer options are A) yes B) no C) maybe, and the long_answer becomes the explanation
`def compact(value)`	recursively removes empty fields ("None", "", [], {}) from the summary JSON. The raw summaries are about 70% "None" values; removing them makes targets short and teaches the model to output only facts that exist
`def from_clinical(row)`	one clinical row gives TWO examples: notes (conversation → note) and summary (note → JSON)
`summary = compact(json.loads(row["summary"]))` inside try/except	about 25% of the raw summaries are not valid JSON; those rows are skipped (you measured this; never train on broken targets)

`json.dumps(summary, ensure_ascii=False)`	the cleaned summary as one compact JSON string (the assistant target)
`def fits(example)`	True if the total characters of all messages are at most MAX_CHARS
`def build(rows, fn)`	applies a converter to every row and drops the None results
`def sample(rows, n)`	random n rows, or all rows if there are fewer
`qa_test = (sample(...medqa_test..., 200) + sample(...medmcqa_val..., 200))`	the QA test set comes from the OFFICIAL test/validation splits only: 200 MedQA + 200 MedMCQA
`pubmed = build(...); random.shuffle(pubmed); qa_test += pubmed[:100]`	100 PubMedQA rows go to test; the other 900 can be used for training
`qa_pool = (...)`	every training candidate: MedQA train + MedMCQA train + 900 PubMedQA
`test_keys = {x["question_key"] for x in qa_test}`	fingerprints of every test question
`for ex in qa_pool: if k in seen or k in test_keys or not fits(ex): continue`	skip exact duplicates, skip anything that matches a TEST question (decontamination), skip examples that are too long
`write_jsonl(SFT / "qa_val.jsonl", qa_clean[:300])`	300 rows for validation loss during training; the rest is the training set
`dpo_qa = [...] write_jsonl(PREF / "qa_dpo.jsonl", dpo_qa)`	3,000 unseen MedMCQA prompts for DPO, also checked against train and test
`pairs = [p for p in build(... from_clinical) if fits(p[0]) and fits(p[1])]`	(notes, summary) pairs that are valid and short enough
`splits = {"test": pairs[:200], "val": pairs[200:400], "dpo": pairs[400:1400], "train": pairs[1400:]}`	ONE shared split for both tasks, so a note that is in the summary test set is never a notes training example
`folder = PREF if split == "dpo" else SFT`	DPO prompts go to data/pref, everything else to data/sft

Watch out: the assistant answer format ("Answer: X) text") is a CONTRACT. The evaluation script and the API parse it. If you change it here, change it everywhere.

CHECKPOINT: ls ../data/sft shows nine files (qa, notes, summary x train, val, test) and ls ../data/pref shows qa_dpo.jsonl, notes_dpo.jsonl, summary_dpo.jsonl.

If not: "FileNotFoundError: data/raw/..." means Section 4.1 did not finish; re-run it.

SECTION 4.3 - Validate every .jsonl file before paying for GPU time

WHERE: VS Code Terminal, folder ~/-MEDSCRIPTAI/scripts, (.venv-train) active

GOAL: prove every line is valid chat data and fits the 2048-token window.

Step 1. Run:

```
python 03_validate_jsonl.py
```

You should see one line per file such as qa_train.jsonl rows= 30000 tokens median=314 max=934, notes_train.jsonl ... median=1130 max=1520, and finally ALL_OK.

FILE scripts/03_validate_jsonl.py:

```
"""Step 3: prove every .jsonl file is valid chat data before you pay for GPU time."""
import json                                     # parse each line
import statistics                               # median / max token counts
import sys                                      # exit code for pass/fail
from pathlib import Path
from common import SFT, BASE_MODEL

ROLES = ["system", "user", "assistant"]
errors = 0

try:                                            # token counts need the Llama tokenizer
    from transformers import AutoTokenizer
    tok = AutoTokenizer.from_pretrained(BASE_MODEL)
except Exception as exc:                        # still validate structure without it
    tok = None
    print(f"(tokenizer not loaded, skipping token counts: {exc.__class__.__name__})")
```

```
for path in sorted(Path(SFT).glob("*.jsonl")):
    lengths = []
    for n, line in enumerate(path.open(encoding="utf-8"), start=1):
        try:
            row = json.loads(line)                # 1) each line must be valid JSON
            roles = [m["role"] for m in row["messages"]]
            assert roles == ROLES, f"roles {roles}"  # 2) exactly system -> user -> assistant
            assert all(m["content"].strip() for m in row["messages"]), "empty message"
            if row["task"] == "summary":          # 3) summary targets must be valid JSON
                json.loads(row["messages"][-1]["content"])
            if tok:                               # 4) length in tokens after the chat template
                ids = tok.apply_chat_template(row["messages"], tokenize=True)
                ids = ids["input_ids"] if isinstance(ids, dict) or hasattr(ids, "keys") else ids
                lengths.append(len(ids))
        except Exception as exc:
            errors += 1
            print(f"ERROR {path.name}:{n}: {exc}")
    stats = f" tokens median={statistics.median(lengths):.0f} max={max(lengths)}" if lengths else ""
    print(f"{path.name:22} rows={n:>6}{stats}")

print("ALL_OK" if errors == 0 else f"FAILED with {errors} errors")
sys.exit(1 if errors else 0)
```

Line	What it does (plain English)
`ROLES = ["system", "user", "assistant"]`	the only allowed message order
`from transformers import AutoTokenizer` inside try	loads Llama-3.1's real tokenizer to count tokens; if it cannot load, structure is still checked
`for path in sorted(Path(SFT).glob("*.jsonl"))`	every .jsonl file in data/sft
`for n, line in enumerate(path.open(...), start=1)`	read line by line with a 1-based line number for error messages
`row = json.loads(line)`	check 1: valid JSON
`assert roles == ROLES`	check 2: exactly system → user → assistant
`assert all(m["content"].strip() ...)`	check 3: no empty message
`if row["task"] == "summary": json.loads(...)`	check 4: summary targets are valid JSON
`ids = tok.apply_chat_template(row["messages"], tokenize=True)`	check 5: the exact token count after Llama's chat template (what the trainer will see)
`ids = ids["input_ids"] if ... else ids`	newer transformers return a dict, older return a list; this line handles both
`stats = f" tokens median=... max=..."`	median and maximum length per file
`print("ALL_OK" ...); sys.exit(1 if errors else 0)`	exit code 0 = success, 1 = failure (useful in scripts and CI)

Watch out: "apply_chat_template requires jinja2" means jinja2 is missing; run uv pip install jinja2 (it is in requirements/train.txt, so this only happens in a hand-made environment).

CHECKPOINT: the last line is ALL_OK and every max is below 2048.

If not: the ERROR line names the file and line number; open it with sed -n 'LINEp' ../data/sft/FILE.

SECTION 4.4 - Train your first adapter (QLoRA SFT on 2,000 QA examples)

WHERE: VS Code Terminal, new tmux session, folder ~/-MEDSCRIPTAI/scripts, (.venv-train) active

GOAL: outputs/adapters/sft-qa containing a trained LoRA adapter.

Step 1. Start a new tmux session:

```
tmux new -s train
```

Step 2. Activate the environment and go to the scripts folder:

```
cd ~/-MEDSCRIPTAI && source .venv-train/bin/activate && cd scripts
```

Step 3. Start training on the first 2,000 examples:

```
python 10_train_sft.py --task qa --max-train 2000
```

You should see, in order: the Unsloth banner with your GPU name; "Unsloth: Will patch your computer"; a line like trainable params: 41,943,040 || all params: 8,072,204,288 || trainable%: 0.5196; one formatted example starting with

<|start_header_id|>system<|end_header_id|>; then a progress bar with about 125 steps and lines like {'loss': 1.9, ...} every 10 steps.

Step 4. In a SECOND VS Code Terminal (click + in the terminal panel), watch the GPU:

```
watch -n 5 nvidia-smi
```

You should see: python using about 10-16 GB of the 80 GB and GPU-Util at 90-100%. Press Ctrl+C to stop watching.

Step 5. When training ends, read the last line:

You should see: DONE adapter=/home/ubuntu/-MEDSCRIPTAI/outputs/adapters/sft-qa train_loss=0.8xxx (your number will differ; a falling loss from about 1.5-2.5 down to about 0.6-1.0 is healthy).

FILE scripts/10_train_sft.py:

```
"""Step 4: QLoRA supervised fine-tuning (SFT) of Llama-3.1-8B-Instruct with Unsloth + TRL + PEFT.

One run = one task = one LoRA adapter, e.g.  python scripts/10_train_sft.py --task qa
"""
from unsloth import FastLanguageModel                    # import Unsloth FIRST so it can patch TRL/transformers
from unsloth.chat_templates import train_on_responses_only

import argparse                                          # command-line options
import json                                              # save the training log
from datasets import load_dataset                        # read our .jsonl files
from trl import SFTConfig, SFTTrainer                    # Hugging Face TRL trainer
from common import SFT, OUT, BASE_MODEL_4BIT

p = argparse.ArgumentParser()
p.add_argument("--task", required=True, choices=["qa", "notes", "summary"])
p.add_argument("--max-train", type=int, default=None, help="use only the first N training rows")
p.add_argument("--epochs", type=float, default=1.0)
p.add_argument("--max-steps", type=int, default=-1, help="stop after N steps (-1 = use epochs)")
p.add_argument("--lr", type=float, default=2e-4)        # LoRA learns fast; 2e-4 is the usual QLoRA rate
p.add_argument("--rank", type=int, default=16)          # LoRA rank r: size of the adapter matrices
p.add_argument("--batch", type=int, default=8)          # examples per GPU step
p.add_argument("--grad-accum", type=int, default=2)     # effective batch = batch x grad-accum = 16
p.add_argument("--train-file", default=None, help="override the training .jsonl (Part 5 uses this)")
p.add_argument("--run-name", default=None)
a = p.parse_args()
run = a.run_name or f"sft-{a.task}"
out_dir = OUT / "adapters" / run

# 1) Load the base model already quantised to 4-bit (this is the "Q" in QLoRA).
model, tokenizer = FastLanguageModel.from_pretrained(
    model_name=BASE_MODEL_4BIT,     # 4-bit NF4 weights: ~5.7 GB on disk instead of ~16 GB
    max_seq_length=2048,            # longest example we keep is ~1,600 tokens
    load_in_4bit=True,
    dtype=None,                     # None = auto: bfloat16 on A100/L40/H100
)

# 2) Attach small trainable LoRA matrices to every attention and MLP projection.
model = FastLanguageModel.get_peft_model(
    model,
    r=a.rank,
    lora_alpha=a.rank,              # scaling = alpha / r = 1.0
    lora_dropout=0.0,               # 0 is fastest in Unsloth and works well
    target_modules=["q_proj", "k_proj", "v_proj", "o_proj", "gate_proj", "up_proj", "down_proj"],
    use_gradient_checkpointing="unsloth",   # recompute activations to save VRAM
    random_state=3407,
)
model.print_trainable_parameters()  # expect ~42M trainable of ~8B (about 0.5%)


# 3) Turn each {"messages": [...]} row into one string using Llama-3.1's own chat template.
#    The template starts with <|begin_of_text|>; the trainer adds it again when tokenising,
#    so we remove it here to end up with exactly ONE begin token (double BOS hurts quality).
def to_text(batch):
    texts = [tokenizer.apply_chat_template(m, tokenize=False).removeprefix(tokenizer.bos_token)
             for m in batch["messages"]]
    return {"text": texts}


train_file = a.train_file or str(SFT / f"{a.task}_train.jsonl")
train = load_dataset("json", data_files=train_file, split="train")
if a.max_train:
```

```
    train = train.select(range(min(a.max_train, len(train))))
val = load_dataset("json", data_files=str(SFT / f"{a.task}_val.jsonl"), split="train").select(range(100))
train = train.map(to_text, batched=True, remove_columns=train.column_names)
val = val.map(to_text, batched=True, remove_columns=val.column_names)
print(train[0]["text"][:600])       # eyeball one formatted example before spending GPU time

# 4) Training settings.
cfg = SFTConfig(
    output_dir=str(OUT / "checkpoints" / run),
    dataset_text_field="text",
    max_length=2048,
    per_device_train_batch_size=a.batch,
    gradient_accumulation_steps=a.grad_accum,
    num_train_epochs=a.epochs,
    max_steps=a.max_steps,
    learning_rate=a.lr,
    lr_scheduler_type="cosine",     # smooth decay to ~0 by the end
    warmup_steps=10,                # ramp the LR up for the first 10 steps
    optim="adamw_8bit",             # 8-bit optimiser states: big VRAM saving
    weight_decay=0.01,
    bf16=True,
    logging_steps=10,
    eval_strategy="steps",
    eval_steps=100,
    save_steps=200,
    save_total_limit=2,             # keep only the 2 newest checkpoints on disk
    report_to="none",
    seed=3407,
)
trainer = SFTTrainer(model=model, processing_class=tokenizer, train_dataset=train,
                     eval_dataset=val, args=cfg)

# 5) Only learn from the assistant's answer, not from the system/user text.
trainer = train_on_responses_only(
    trainer,
    instruction_part="<|start_header_id|>user<|end_header_id|>\n\n",
    response_part="<|start_header_id|>assistant<|end_header_id|>\n\n",
)

stats = trainer.train()

# 6) Save ONLY the adapter (~80-170 MB at r=16), not the 16 GB base model.
model.save_pretrained(str(out_dir))
tokenizer.save_pretrained(str(out_dir))
(out_dir / "train_log.json").write_text(json.dumps(trainer.state.log_history, indent=1))
print(f"DONE adapter={out_dir} train_loss={stats.training_loss:.4f}")
```

Line	What it does (plain English)
`from unsloth import FastLanguageModel` (first import)	Unsloth must be imported BEFORE trl/transformers so it can patch them with its fast kernels
`from unsloth.chat_templates import train_on_responses_only`	the helper that masks system/user tokens so loss is computed only on the answer
`from trl import SFTConfig, SFTTrainer`	Hugging Face TRL's supervised fine-tuning trainer and its settings
`p.add_argument("--task", required=True, choices=[...])`	which adapter to train; choices prevents typos
`--max-train`	use only the first N training rows (2,000 in Part 4)
`--epochs 1.0`	one pass over the data; more passes risk memorisation
`--max-steps -1`	-1 = stop by epochs; a number = stop after that many optimizer steps (handy for tests)
`--lr 2e-4`	learning rate; 2e-4 is the standard starting point for QLoRA (the QLoRA paper and Unsloth defaults)
`--rank 16`	LoRA rank r: width of the adapter matrices. 8-16 for focused tasks, 32-64 for big behaviour changes
`--batch 8`, `--grad-accum 2`	8 examples per GPU step, gradients summed over 2 steps = effective batch 16
`--train-file`	train on another file (Part 5 uses qa_train_synth.jsonl)
`run = a.run_name or f"sft-{a.task}"`	folder name, e.g. sft-qa

`FastLanguageModel.from_pretrained(model_name=BASE_MODEL_4BIT, ...)`	downloads (once) and loads Llama-3.1-8B-Instruct in 4-bit NF4 plus its tokenizer
`max_seq_length=2048`	longest sequence the model will be fed; our longest example is about 1,600 tokens
`load_in_4bit=True`	QLoRA: the frozen base weights stay 4-bit, saving about 10 GB of VRAM
`dtype=None`	auto-detect: bfloat16 on A100/L40/H100
`FastLanguageModel.get_peft_model(model, ...)`	wraps the model with LoRA adapters (PEFT)
`r=a.rank`, `lora_alpha=a.rank`	rank 16 and alpha 16: the update is scaled by alpha/r = 1.0
`lora_dropout=0.0`	no dropout inside LoRA; Unsloth is fastest at 0 and short runs rarely overfit through LoRA dropout
`target_modules=["q_proj", ..., "down_proj"]`	put adapters on all 4 attention projections and all 3 MLP projections; adapting all linear layers beats attention-only (QLoRA paper)
`use_gradient_checkpointing="unsloth"`	do not store all activations; recompute them in the backward pass. Saves VRAM, costs about 20% speed
`random_state=3407`	seed for the adapter's random initialisation
`model.print_trainable_parameters()`	prints about 42M trainable of 8B: proof that only LoRA is trained
`def to_text(batch)`	turns each messages list into one string with Llama's chat template
`tokenizer.apply_chat_template(m, tokenize=False)`	produces <|begin_of_text|><|start_header_id|>system<|end_header_id|> ... <|eot_id|> text
`.removeprefix(tokenizer.bos_token)`	removes the leading <|begin_of_text|> because the trainer adds it again during tokenisation. Without this you get TWO begin tokens (we caught this bug while testing)
`load_dataset("json", data_files=train_file, split="train")`	reads the .jsonl into a Dataset ("train" is just the default split name for a single file)
`train.select(range(min(a.max_train, len(train))))`	keep the first N rows
`val = ... .select(range(100))`	100 validation rows are enough to watch eval loss and keep evaluation fast
`train.map(to_text, batched=True, remove_columns=train.column_names)`	add the "text" column and drop the original columns (task, source, ...) the trainer does not need
`print(train[0]["text"][:600])`	eyeball one example: the template must look right BEFORE you spend GPU time
`SFTConfig(output_dir=...)`	where checkpoints are written (outputs/checkpoints/sft-qa)
`dataset_text_field="text"`	the column that holds the full formatted conversation
`max_length=2048`	truncate anything longer (nothing is, thanks to the validator)
`per_device_train_batch_size=a.batch`, `gradient_accumulation_steps=a.grad_accum`	batch 8 × 2 = 16 examples per optimizer step; 2,000 / 16 = 125 steps
`num_train_epochs`, `max_steps`	stop condition
`learning_rate=a.lr`	2e-4
`lr_scheduler_type="cosine"`	LR follows half a cosine wave down to about 0 at the end: smooth, stable finish
`warmup_steps=10`	the LR ramps up for the first 10 steps so early random updates do not wreck the model
`optim="adamw_8bit"`	AdamW with optimizer states stored in 8-bit (bitsandbytes): big memory saving, same quality
`weight_decay=0.01`	a small pull of weights towards zero: mild regularisation
`bf16=True`	compute in bfloat16
`logging_steps=10`	print loss every 10 steps
`eval_strategy="steps", eval_steps=100`	compute validation loss every 100 steps: if it rises while train loss falls, you are overfitting
`save_steps=200, save_total_limit=2`	checkpoint every 200 steps, keep only the newest 2 (disk safety)
`report_to="none"`	do not send metrics to W&B/Trackio (add later if you like)
`seed=3407`	data order seed
`SFTTrainer(model=..., processing_class=tokenizer, ...)`	processing_class is the new name for the tokenizer argument in TRL 1.x

`train_on_responses_only(trainer, instruction_part=..., response_part=...)`	finds the user header and the assistant header in each tokenised example and sets labels to -100 for everything except the assistant's answer
`" <|start_header_id|>assistant<|end_header_id|> "`	Llama-3.1's exact marker where the answer starts; for another model family these strings are different
`stats = trainer.train()`	the training loop: forward pass, loss, backward pass, optimizer step, repeat
`model.save_pretrained(str(out_dir))`	saves ONLY adapter_model.safetensors (~80-170 MB) and adapter_config.json, which records the base model name
`tokenizer.save_pretrained(str(out_dir))`	saves the tokenizer next to the adapter so the folder is self-contained
`(out_dir / "train_log.json").write_text(...)`	every logged loss/LR value, for your report and plots

Watch out: "CUDA out of memory" means batch too large for your GPU (for example on a 24 GB card): add --batch 4 --grad-accum 4 (same effective batch 16).

Watch out: if the eval loss goes UP for two evaluations in a row while train loss goes down, stop: you are overfitting. Use fewer epochs or more data.

CHECKPOINT: ls ../outputs/adapters/sft-qa shows adapter_config.json, adapter_model.safetensors, tokenizer files and train_log.json.

If not: re-attach with tmux attach -t train and read the error; the Troubleshooting part (Part 10) lists the common ones.

SECTION 4.5 - Compare the base model and your adapter

WHERE: VS Code Terminal, folder ~/-MEDSCRIPTAI/scripts, (.venv-train) active

GOAL: see with your own eyes what fine-tuning changed.

Step 1. Run:

```
python 11_chat_test.py --task qa --n 3
```

You should see three blocks with USER, BASE, FINE-TUNED and REFERENCE. Typically BASE writes a long free-form answer and often does not start with "Answer:"; FINE-TUNED starts with "Answer: <letter>) <option>" exactly like the training data.

FILE scripts/11_chat_test.py:

```
"""Step 5: ask the same test questions to the base model and to base + your adapter, side by side."""
from unsloth import FastLanguageModel                    # Unsloth first, as always

import argparse
import torch
from common import SFT, OUT, read_jsonl

p = argparse.ArgumentParser()
p.add_argument("--task", default="qa", choices=["qa", "notes", "summary"])
p.add_argument("--adapter", default=None, help="adapter folder (default outputs/adapters/sft-<task>)")
p.add_argument("--n", type=int, default=3, help="how many test examples to show")
a = p.parse_args()
adapter = a.adapter or str(OUT / "adapters" / f"sft-{a.task}")

# Loading the ADAPTER folder makes Unsloth read adapter_config.json, load the 4-bit base, then attach the LoRA.
model, tokenizer = FastLanguageModel.from_pretrained(adapter, max_seq_length=2048, load_in_4bit=True)
FastLanguageModel.for_inference(model)                   # switch on Unsloth's fast generation path


def answer(messages):
    """Greedy-decode one reply for a list of chat messages (system + user)."""
    ids = tokenizer.apply_chat_template(messages, add_generation_prompt=True, return_tensors="pt",
                                        return_dict=True).to("cuda")
    out = model.generate(**ids, max_new_tokens=400, do_sample=False)   # do_sample=False = deterministic
    return tokenizer.decode(out[0][ids["input_ids"].shape[1]:], skip_special_tokens=True).strip()


for row in read_jsonl(SFT / f"{a.task}_test.jsonl")[: a.n]:
    prompt = row["messages"][:2]                          # system + user only; hide the reference answer
```

```
    with model.disable_adapter():                          # temporarily turn LoRA OFF = plain base model
        base = answer(prompt)
    tuned = answer(prompt)                                 # LoRA back ON
    print("=" * 80, "\nUSER:", prompt[1]["content"][:500])
    print("\n--- BASE ---\n", base[:700])
    print("\n--- FINE-TUNED ---\n", tuned[:700])
    print("\n--- REFERENCE ---\n", row["messages"][2]["content"][:700])
    torch.cuda.empty_cache()
```

Line	What it does (plain English)
`FastLanguageModel.from_pretrained(adapter, ...)`	pointing at an ADAPTER folder: Unsloth reads adapter_config.json, loads the 4-bit base model, and attaches your LoRA
`FastLanguageModel.for_inference(model)`	switches Unsloth to its faster generation mode (about 2x)
`def answer(messages)`	generates one reply
`apply_chat_template(messages, add_generation_prompt=True, return_tensors="pt", return_dict=True)`	formats system + user and ends with the assistant header so the model starts answering; returns PyTorch tensors
`.to("cuda")`	moves the tensors to the GPU
`model.generate(**ids, max_new_tokens=400, do_sample=False)`	greedy decoding: always the most likely token, so results are repeatable
`out[0][ids["input_ids"].shape[1]:]`	cut away the prompt tokens, keep only the new answer
`skip_special_tokens=True`	hide <|eot_id|> and similar tokens
`prompt = row["messages"][:2]`	system + user only; the reference answer is hidden from the model
`with model.disable_adapter():`	temporarily switches LoRA OFF: the same object behaves like the plain base model
`tuned = answer(prompt)`	outside the with-block LoRA is ON again
`torch.cuda.empty_cache()`	frees cached GPU memory between examples

Watch out: three examples prove nothing statistically; they only prove the adapter loads and changed behaviour. Real measurement comes in Part 5.

CHECKPOINT: the FINE-TUNED answers follow the "Answer: X) ..." format.

If not: if FINE-TUNED looks identical to BASE, check that the adapter path is correct and that train_loss went down.

SECTION 4.6 - Publish the adapter on the Hugging Face Hub

WHERE: VS Code Terminal, folder ~/-MEDSCRIPTAI/scripts, (.venv-train) active

GOAL: a private model repo holding your adapter (your first production artifact).

Step 1. Run (replace YOUR_HF_USERNAME):

```
python 12_push_to_hub.py --folder ../outputs/adapters/sft-qa --repo YOUR_HF_USERNAME/Llama-3.1-8B-MedScriptAI-qa-lora
```

You should see: a URL like https://huggingface.co/YOUR_HF_USERNAME/Llama-3.1-8B-MedScriptAI-qa-lora

Step 2. Open the URL in your browser.

You should see: adapter_config.json and adapter_model.safetensors (about 80-170 MB), and the repo marked Private.

FILE scripts/12_push_to_hub.py:

```
"""Step 6: publish an adapter or a dataset folder to the Hugging Face Hub (private by default)."""
import argparse
from huggingface_hub import HfApi                  # official Hub client; reads HF_TOKEN from the environment

p = argparse.ArgumentParser()
p.add_argument("--folder", required=True, help="local folder, e.g. outputs/adapters/sft-qa or data/sft")
p.add_argument("--repo", required=True, help="YOUR_HF_USERNAME/Llama-3.1-8B-MedScriptAI-qa-lora")
p.add_argument("--type", default="model", choices=["model", "dataset"])
p.add_argument("--public", action="store_true", help="make the repo public (default private)")
a = p.parse_args()

api = HfApi()
api.create_repo(a.repo, repo_type=a.type, private=not a.public, exist_ok=True)   # no error if it exists
api.upload_folder(folder_path=a.folder, repo_id=a.repo, repo_type=a.type,
```

```
                  ignore_patterns=["checkpoint-*", "*.pt", "optimizer*"])          # skip training leftovers
print(f"https://huggingface.co/{'datasets/' if a.type == 'dataset' else ''}{a.repo}")
```

Line	What it does (plain English)
`from huggingface_hub import HfApi`	the official Hub client; it reads HF_TOKEN from your environment
`--folder`, `--repo`, `--type`, `--public`	what to upload, where, model or dataset, and visibility (private by default)
`api.create_repo(a.repo, repo_type=a.type, private=not a.public, exist_ok=True)`	creates the repo; exist_ok=True means no error if it already exists
`api.upload_folder(..., ignore_patterns=["checkpoint-*", "*.pt", "optimizer*"])`	uploads the folder but skips training leftovers such as optimizer states
`print(f"https://huggingface.co/...")`	prints the repo address

Placeholders:

Placeholder	Replace with	Example
YOUR_HF_USERNAME	your Hugging Face user name	rahulml

Watch out: the repo name must start with "Llama" (Llama 3.1 licence) [15].

CHECKPOINT: the Hub page shows the adapter files.

If not: "401 Unauthorized" = HF_TOKEN missing or read-only; create a Write token (Section 2.2) and run source ~/.bashrc.

SECTION 4.7 - Your first cloud step: store the adapter in object storage (S3)

WHERE: VS Code Terminal, folder ~/-MEDSCRIPTAI

GOAL: the adapter copied to a bucket, practised free on FakeCloud, then (optionally) on real AWS.

Step 1. Follow the column you are working in, one row at a time.

Step	FakeCloud (free practice)	Real AWS (same step)
1 Point the CLI	`export AWS_ENDPOINT_URL=http://localhost:4566 AWS_ACCESS_KEY_ID=test AWS_SECRET_ACCESS_KEY=test AWS_DEFAULT_REGION=us-east-1`	`unset AWS_ENDPOINT_URL` (your keys from `aws configure` are used)
2 Make a unique bucket name	`export BUCKET=medscriptai-artifacts-$(aws sts get-caller-identity --query Account --output text)-us-east-1`	same command (your real account number makes the name globally unique)
3 Create the bucket	`aws s3 mb s3://$BUCKET`	`aws s3 mb s3://$BUCKET --region us-east-1`
4 Upload the adapter	`aws s3 sync outputs/adapters/sft-qa s3://$BUCKET/adapters/sft-qa`	same command
5 List what is there	`aws s3 ls s3://$BUCKET --recursive --human-readable`	same command
You should see	adapters/sft-qa/adapter_model.safetensors with its size	the same list; also visible in the AWS console > S3 > your bucket
Cost	US$0	about US$0.023 per GB-month in us-east-1 (verify); a 170 MB adapter costs well under 1 cent per month

What each command piece means:

Command piece	What it does
`aws s3 mb`	"make bucket"
`aws s3 sync SRC DST`	copies only new or changed files (re-running is cheap and safe)
`--query Account --output text`	asks the CLI for just one field, as plain text
`$(...)`	runs the command inside and pastes its output into the outer command

Watch out: bucket names are global across all AWS customers; that is why the account number is in the name.

CHECKPOINT: aws s3 ls shows the adapter in the bucket for the column you used.

If not: "NoSuchBucket" = Step 3 failed or BUCKET is empty in this terminal (echo $BUCKET).

4.8 Common mistake at this level

The mistake: training on text that is formatted differently from what the server sends at inference. Examples: two <|begin_of_text|> tokens (fixed in 10_train_sft.py), a different system prompt in the API, or a missing chat template. The model then underperforms and you blame the hyper-parameters.

How to spot it: print one formatted training example (10_train_sft.py does this), print the exact messages your API sends, and compare token by token. The repo avoids the problem by importing SYSTEM from one file everywhere.

PART 5 - INTERMEDIATE: synthetic data, three adapters, real evaluation

5.0 Concept

What: serve a stronger open model (Qwen2.5-14B-Instruct) as a teacher, use distilabel to write explanations for the 10,000 MedQA answers that have none, train all three adapters (qa, notes, summary) on the full data, and evaluate base vs fine-tuned with accuracy, ROUGE-L, BERTScore and an LLM judge.

Why: Part 4 proved the loop; Part 5 is what a real team does next: improve the data (synthetic augmentation with quality filters), scale training, and measure. "It looks better" is not evidence; numbers on a held-out test set are.

When: whenever the labels you have are thinner than the behaviour you want (here: letters without reasoning), and before any claim about quality.

Harder than Part 4 because: two model servers, a pipeline framework, three training runs, and metrics you must interpret correctly. Budget: about 6 GPU-hours (about US$8).

SECTION 5.1 - Start the teacher model (vLLM, port 8001)

WHERE: VS Code Terminal on the VM, folder ~/-MEDSCRIPTAI

GOAL: Qwen2.5-14B-Instruct answering OpenAI-style requests under the name "teacher".

Step 1. Start it:

```
bash serve/start_teacher.sh
```

Step 2. Follow the start-up log (the first start downloads about 30 GB, 5-10 minutes):

```
docker logs -f medscriptai-teacher
```

You should see, at the end: "Application startup complete." and "Uvicorn running on http://0.0.0.0:8000". Press Ctrl+C to stop following (the server keeps running).

Step 3. Ask it a test question:

```
curl -s http://localhost:8001/v1/chat/completions -H "Content-Type: application/json" -d '{"model":"teacher","messages":
[{"role":"user","content":"In one sentence: why is nitrofurantoin used for cystitis in pregnancy?"}],"max_tokens":80}' |
jq -r '.choices[0].message.content'
```

You should see: one sensible medical sentence.

FILE serve/start_teacher.sh:

```
#!/usr/bin/env bash
# Serve the TEACHER / JUDGE model (Qwen2.5-14B-Instruct) on port 8001 under the name "teacher".
# Used by distilabel (Part 5/6) and by the LLM judge. Stop it when you are done: docker rm -f medscriptai-teacher
set -euo pipefail

VLLM_IMAGE="${VLLM_IMAGE:-vllm/vllm-openai:v0.32.0-cu129}"
GPU_UTIL="${GPU_UTIL:-0.85}"                        # 0.85 alone on the GPU; 0.50 next to start_vllm.sh (0.35)

docker rm -f medscriptai-teacher 2>/dev/null || true
docker run -d --name medscriptai-teacher \
  --gpus all --ipc=host \
  -p 8001:8000 \
  -e VLLM_ENABLE_CUDA_COMPATIBILITY="${VLLM_ENABLE_CUDA_COMPATIBILITY:-1}" \
  -v "${HF_HOME:-$HOME/.cache/huggingface}":/root/.cache/huggingface \
  "$VLLM_IMAGE" \
  --model Qwen/Qwen2.5-14B-Instruct \
  --served-model-name teacher \
  --dtype bfloat16 \
  --max-model-len 8192 \
  --gpu-memory-utilization "$GPU_UTIL"

echo "Started. Follow the logs with:  docker logs -f medscriptai-teacher"
```

Line	What it does (plain English)
`set -euo pipefail`	stop at the first error (-e), treat unset variables as errors (-u), fail a pipeline if any part fails

`VLLM_IMAGE="${VLLM_IMAGE:-vllm/vllm-openai:v0.32.0-cu129}"`	the official vLLM server image, CUDA 12.9 build. `${X:-default}` = use X if set, else the default
`GPU_UTIL="${GPU_UTIL:-0.85}"`	fraction of the GPU's memory this server may take (weights + KV cache)
`docker rm -f medscriptai-teacher ... || true`	remove an old container with the same name; `|| true` ignores "no such container"
`docker run -d --name medscriptai-teacher`	start a container in the background (-d) with a fixed name
`--gpus all --ipc=host`	give the container the GPU; share host memory for fast inter-process communication (vLLM needs it)
`-p 8001:8000`	the server listens on 8000 inside the container; you reach it on 8001 on the VM
`-e VLLM_ENABLE_CUDA_COMPATIBILITY=1`	your VM driver (R570) supports CUDA 12.8 but the image uses 12.9; NVIDIA's forward-compatibility libraries inside the image bridge the gap on data-centre GPUs such as A100/L40/H100
`-v ${HF_HOME}:/root/.cache/huggingface`	share the model cache, so the 30 GB download happens once
`--model Qwen/Qwen2.5-14B-Instruct`	the teacher: 14.7B parameters, Apache-2.0, strong at medical reasoning for its size
`--served-model-name teacher`	clients ask for model "teacher" instead of the long Hub name
`--dtype bfloat16`	16-bit weights: about 29.5 GB
`--max-model-len 8192`	longest prompt+answer in tokens; smaller = more room for parallel requests
`--gpu-memory-utilization "$GPU_UTIL"`	vLLM pre-allocates this fraction and fills the rest of it with KV cache (PagedAttention)

If something goes wrong:

If you see...	It means...	Do this next
"CUDA driver version is insufficient"	the compatibility layer was not used	check the -e VLLM_ENABLE_CUDA_COMPATIBILITY=1 line, or use the R595 OS image with VLLM_IMAGE=vllm/vllm-openai:v0.32.0
"No available memory for the cache blocks"	GPU memory is taken by something else	run nvidia-smi; stop other containers or training jobs
curl: (7) Failed to connect	the server is still loading	wait for "Application startup complete"

CHECKPOINT: curl http://localhost:8001/v1/models lists "teacher".

If not: docker logs medscriptai-teacher | tail -40 shows the reason.

SECTION 5.2 - Generate synthetic explanations with distilabel

WHERE: VS Code Terminal, tmux session, folder ~/-MEDSCRIPTAI/scripts, (.venv-synth) active

GOAL: data/sft/qa_train_synth.jsonl where MedQA answers also have a teacher explanation.

Step 1. Start tmux:

```
tmux new -s synth
```

Step 2. Activate the synthetic-data environment:

```
cd ~/-MEDSCRIPTAI && source .venv-synth/bin/activate && cd scripts
```

Step 3. Dry run on 50 rows first (1-2 minutes):

```
python 20_synth_explanations.py --limit 50
```

You should see: distilabel's pipeline log, then kept 4x/50 explanations after the quality filter (most should pass).

Step 4. Look at three random explanations:

```
shuf -n 3 ../data/synth/medqa_explanations.jsonl | python -m json.tool
```

You should see: medically sensible explanations that mention the correct option. If they are wrong or rambling, fix the prompt before the full run.

Step 5. Full run (about 10,000 rows; roughly 15-30 minutes on an A100, estimate):

```
python 20_synth_explanations.py
```

You should see: kept ~9xxx/10xxx explanations after the quality filter, and wrote ... rows → data/sft/qa_train_synth.jsonl.

FILE scripts/20_synth_explanations.py:

```
"""Part 5: use distilabel + a teacher LLM to write explanations for MedQA answers (synthetic SFT data).

MedQA only gives the correct letter. A model trained on letters alone learns to guess, not to reason.
The teacher (Qwen2.5-14B-Instruct served by vLLM as "teacher") explains WHY the gold answer is right.
"""
import argparse
import os
import re
from distilabel.models import OpenAILLM                  # talks to any OpenAI-compatible server (vLLM)
from distilabel.pipeline import Pipeline
from distilabel.steps import LoadDataFromDicts
from distilabel.steps.tasks import TextGeneration
from common import DATA, SFT, read_jsonl, write_jsonl

p = argparse.ArgumentParser()
p.add_argument("--limit", type=int, default=None, help="only explain the first N MedQA rows (test run)")
a = p.parse_args()

TEACHER_URL = os.getenv("TEACHER_URL", "http://localhost:8001/v1")   # vLLM teacher endpoint
TEACHER = os.getenv("TEACHER_MODEL", "teacher")                       # --served-model-name in vLLM
TEACHER_SYSTEM = ("You are a senior physician and medical educator. Explain clearly and accurately "
                  "in at most 120 words. Do not change the given correct answer.")

rows = read_jsonl(SFT / "qa_train.jsonl")
todo = [r for r in rows if r["source"] == "medqa" and "Explanation:" not in r["messages"][2]["content"]]
todo = todo[: a.limit] if a.limit else todo
inputs = [{
    "question_key": r["question_key"],
    "gold": r["messages"][2]["content"],                          # e.g. "Answer: D) Nitrofurantoin"
    "instruction": (f"{r['messages'][1]['content']}\n\nThe correct answer is: "
                    f"{r['messages'][2]['content'].removeprefix('Answer: ')}\n"
                    "Explain step by step why it is correct and briefly why the other options are wrong."),
} for r in todo]
print(f"{len(inputs)} MedQA rows need an explanation")

with Pipeline(name="medscriptai-explanations") as pipeline:
    load = LoadDataFromDicts(data=inputs, batch_size=64)          # step 1: feed our rows in
    explain = TextGeneration(                                       # step 2: ask the teacher
        llm=OpenAILLM(model=TEACHER, base_url=TEACHER_URL, api_key="EMPTY",
                      generation_kwargs={"temperature": 0.3, "max_new_tokens": 300}),
        system_prompt=TEACHER_SYSTEM,
        input_batch_size=32,                                          # 32 requests in flight at a time
    )
    load >> explain                                                   # ">>" connects the steps

distiset = pipeline.run(use_cache=False)                            # returns a Hugging Face DatasetDict
generated = distiset["default"]["train"]


def good(text, gold):
    """Quality filter: non-empty, sane length, and it mentions the gold option."""
    if not text or not (80 <= len(text) <= 1500):
        return False
    option = gold.split(") ", 1)[-1].lower()
    letter = gold.removeprefix("Answer: ")[0]
    return option[:25] in text.lower() or f"{letter})" in text


explanations = {g["question_key"]: g["generation"].strip()
                for g in generated if good(g["generation"], g["gold"])}
write_jsonl(DATA / "synth" / "medqa_explanations.jsonl",
            [{"question_key": k, "explanation": v} for k, v in explanations.items()])

for r in rows:                                                      # merge back into the SFT data
    if r["question_key"] in explanations:
        r["messages"][2]["content"] += f"\nExplanation: {explanations[r['question_key']]}"
write_jsonl(SFT / "qa_train_synth.jsonl", rows)
print(f"kept {len(explanations)}/{len(inputs)} explanations after the quality filter")
```

Line	What it does (plain English)
`from distilabel.models import OpenAILLM`	distilabel's client for any OpenAI-compatible server; vLLM is one
`from distilabel.pipeline import Pipeline`	a pipeline = a graph of steps that pass batches of rows to each other
`from distilabel.steps import LoadDataFromDicts`	first step: feeds a Python list of dicts into the pipeline
`from distilabel.steps.tasks import TextGeneration`	a task step: sends each row's "instruction" to the LLM, writes the reply to "generation"
`TEACHER_URL = os.getenv("TEACHER_URL", "http://localhost:8001/v1")`	where the teacher listens (Section 5.1)
`TEACHER = os.getenv("TEACHER_MODEL", "teacher")`	the served model name
`TEACHER_SYSTEM = (...)`	instructions for the teacher: expert tone, at most 120 words, never change the given answer
`todo = [r for r in rows if r["source"] == "medqa" and "Explanation:" not in ...]`	only MedQA rows that have no explanation yet (MedMCQA and PubMedQA already have one)
`"gold": r["messages"][2]["content"]`	the correct answer line, kept for the quality filter
`"instruction": (... "The correct answer is: " ...)`	the teacher sees the question, the options and the GOLD answer. It explains a known answer instead of guessing one: this is "rationale generation", much more reliable than letting it answer
`with Pipeline(name="medscriptai-explanations") as pipeline:`	steps created inside this block belong to the pipeline
`LoadDataFromDicts(data=inputs, batch_size=64)`	sends rows in batches of 64
`OpenAILLM(model=TEACHER, base_url=TEACHER_URL, api_key="EMPTY", generation_kwargs={...})`	vLLM ignores the API key; temperature 0.3 = focused but not robotic; max 300 new tokens
`system_prompt=TEACHER_SYSTEM`	added as the system message of every request
`input_batch_size=32`	32 requests in flight: vLLM batches them on the GPU, so this is much faster than one by one
`load >> explain`	connects the steps: output of load is input of explain
`distiset = pipeline.run(use_cache=False)`	runs everything; use_cache=False forces a fresh run instead of re-using old results
`distiset["default"]["train"]`	distilabel returns a DatasetDict; "default" is the leaf step's output, "train" the split
`def good(text, gold)`	the quality filter: non-empty, 80-1,500 characters, and it must mention the gold option text or letter
`explanations = {...}`	question_key → explanation, only for rows that passed
`write_jsonl(DATA / "synth" / "medqa_explanations.jsonl", ...)`	keep the raw synthetic data separately (you will publish it as a dataset)
`r["messages"][2]["content"] += f" Explanation: {...}"`	append the explanation to the assistant target
`write_jsonl(SFT / "qa_train_synth.jsonl", rows)`	the augmented training file; the original qa_train.jsonl stays untouched for comparison

Watch out: "No module named 'requests'" or strange datasets errors mean you are in the wrong environment. distilabel 1.5.3 only works with the pins in requirements/synth.txt (we found and fixed this while testing).

Watch out: synthetic data inherits the teacher's mistakes. A filter catches format errors, not subtle medical errors. Read 20 random explanations yourself before training (Step 4). In an interview, say exactly this.

CHECKPOINT: wc -l ../data/sft/qa_train_synth.jsonl equals wc -l ../data/sft/qa_train.jsonl, and grep -c "Explanation:" ../data/sft/qa_train_synth.jsonl is much larger than in qa_train.jsonl.

If not: if almost nothing passes the filter, print a few raw generations from distiset and adjust the prompt or the filter.

SECTION 5.3 - Free the GPU and train the three adapters

WHERE: VS Code Terminal, tmux session train, folder ~/-MEDSCRIPTAI/scripts, (.venv-train) active

GOAL: outputs/adapters/sft-qa (re-trained on synthetic data), sft-notes and sft-summary.

Step 1. Stop the teacher (training needs the GPU memory):

```
docker rm -f medscriptai-teacher
```

Step 2. Open the training tmux session:

```
tmux new -s train3
```

Step 3. Activate and move:

```
cd ~/-MEDSCRIPTAI && source .venv-train/bin/activate && cd scripts
```

Step 4. Train all three, one after another (about 2.5-3.5 hours in total on an A100, estimate):

```
python 10_train_sft.py --task qa --train-file ../data/sft/qa_train_synth.jsonl --max-train 12000 && python
10_train_sft.py --task notes && python 10_train_sft.py --task summary
```

Step 5. Detach with Ctrl+B then D, and come back later with tmux attach -t train3.

Setting	qa	notes	summary	Why
training rows	12,000 (of ~30,000)	~7,000	~7,000	12k QA rows gives most of the gain at 40% of the cost
median tokens	~330	~1,130	~1,090	longer examples = more time per step
steps (batch 16)	750	~440	~440	rows / 16
rough time on A100	45-70 min	40-60 min	40-60 min	estimates; read the progress bar for the truth

You should see: three DONE adapter=... lines. Each train_log.json ends with a lower loss than it started.

Watch out: the && between commands means "run the next only if the previous succeeded". If qa fails, notes and summary do not start; fix and re-run only the missing ones.

CHECKPOINT: ls ../outputs/adapters shows sft-qa, sft-notes, sft-summary, each with adapter_model.safetensors.

If not: tmux attach -t train3 and read the error.

SECTION 5.4 - Serve all adapters with vLLM (port 8000) and the judge (port 8001)

WHERE: VS Code Terminal, folder ~/-MEDSCRIPTAI

GOAL: one vLLM server with base + 3 adapters, and the teacher back as the judge, both on the same A100.

Step 1. Start the student server with 35% of the GPU:

```
GPU_UTIL=0.35 bash serve/start_vllm.sh
```

You should see: Adapters: qa=/adapters/sft-qa notes=/adapters/sft-notes summary=/adapters/sft-summary

Step 2. Wait until it is ready:

```
until curl -s localhost:8000/v1/models > /dev/null; do sleep 10; done; curl -s localhost:8000/v1/models | jq -r
'.data[].id'
```

You should see: base, qa, notes, summary (one per line).

Step 3. Start the judge with 50% of the GPU:

```
GPU_UTIL=0.50 bash serve/start_teacher.sh
```

Step 4. Wait for it:

```
until curl -s localhost:8001/v1/models > /dev/null; do sleep 10; done; echo judge ready
```

Watch out: the two GPU_UTIL values must add up to less than about 0.9, or the second server fails with "insufficient GPU memory". The full explanation of start_vllm.sh is in Section 6.4.

CHECKPOINT: both curl commands answer.

SECTION 5.5 - Generate test-set answers for base and fine-tuned models

WHERE: VS Code Terminal, folder ~/-MEDSCRIPTAI/scripts, (.venv-eval) active

GOAL: one predictions file per task and model in outputs/preds.

Step 1. Activate the evaluation environment:

```
cd ~/-MEDSCRIPTAI && source .venv-eval/bin/activate && cd scripts
```

Step 2. Generate (about 5-10 minutes in total):

```
python 30_generate_preds.py --task qa --models base qa && python 30_generate_preds.py --task notes --models base notes
&& python 30_generate_preds.py --task summary --models base summary
```

You should see: six lines like wrote 500 rows → outputs/preds/qa/base.jsonl.

FILE scripts/30_generate_preds.py:

```
"""Evaluation step A: send every test prompt to vLLM for each model/adapter and save the answers.

python scripts/30_generate_preds.py --task qa --models base qa qa-dpo
"base" = Llama-3.1-8B-Instruct with no adapter; "qa" etc. = LoRA adapter names given to vLLM.
"""
import argparse
import os
from concurrent.futures import ThreadPoolExecutor   # many requests at once = vLLM batches them on the GPU
from openai import OpenAI                            # vLLM speaks the OpenAI API
from common import SFT, OUT, read_jsonl, write_jsonl

p = argparse.ArgumentParser()
p.add_argument("--task", required=True, choices=["qa", "notes", "summary"])
p.add_argument("--models", nargs="+", required=True)
p.add_argument("--n", type=int, default=None, help="only the first N test rows")
p.add_argument("--url", default=os.getenv("VLLM_URL", "http://localhost:8000/v1"))
a = p.parse_args()

client = OpenAI(base_url=a.url, api_key="EMPTY")    # vLLM ignores the key, the SDK needs one
rows = read_jsonl(SFT / f"{a.task}_test.jsonl")[: a.n]
max_tokens = 400 if a.task == "qa" else 900


def ask(model, row):
    resp = client.chat.completions.create(
        model=model,
        messages=row["messages"][:2],                # system + user; never send the reference
        temperature=0.0,                             # greedy = reproducible scores
        max_tokens=max_tokens,
    )
    return resp.choices[0].message.content


for model in a.models:
    with ThreadPoolExecutor(max_workers=16) as pool:
        preds = list(pool.map(lambda r: ask(model, r), rows))
    out = [{"i": i, "source": r.get("source"), "answer_letter": r.get("answer_letter"),
            "prompt": r["messages"][1]["content"], "reference": r["messages"][2]["content"],
            "prediction": pred} for i, (r, pred) in enumerate(zip(rows, preds))]
    write_jsonl(OUT / "preds" / a.task / f"{model}.jsonl", out)
```

Line	What it does (plain English)
`from concurrent.futures import ThreadPoolExecutor`	send many requests at the same time; vLLM batches them on the GPU (continuous batching)
`from openai import OpenAI`	the official OpenAI Python client; vLLM implements the same API
`--models`	list of model names known to vLLM: "base" = no adapter, "qa" = base + qa adapter
`--url`	the vLLM address (default http://localhost:8000/v1)
`client = OpenAI(base_url=a.url, api_key="EMPTY")`	the client talks to vLLM; the key is required by the SDK but ignored by vLLM
`rows = read_jsonl(SFT / f"{a.task}_test.jsonl")[: a.n]`	the held-out TEST set (never used for training)
`max_tokens = 400 if a.task == "qa" else 900`	answer length limit per task
`def ask(model, row)`	one request
`messages=row["messages"][:2]`	only system + user; the reference answer is never sent
`temperature=0.0`	greedy decoding: the same input gives the same output, so scores are reproducible

`pool.map(lambda r: ask(model, r), rows)`	16 parallel workers, results in the original order
`out = [{"i": i, "source": ..., "answer_letter": ..., "prompt": ..., "reference": ..., "prediction": pred} ...]`	everything the metrics and the judge need in one row
`write_jsonl(OUT / "preds" / a.task / f"{model}.jsonl", out)`	e.g. outputs/preds/qa/base.jsonl

CHECKPOINT: ls ../outputs/preds/* shows base.jsonl and the adapter file for each task.

SECTION 5.6 - Automatic metrics: accuracy, ROUGE-L, BERTScore, JSON validity

WHERE: VS Code Terminal, folder ~/-MEDSCRIPTAI/scripts, (.venv-eval) active

GOAL: outputs/reports/metrics.json with every metric for every model.

Step 1. Run (the first run downloads roberta-large, 1.4 GB):

```
python 31_eval_metrics.py
```

You should see: one line per task/model such as qa base {'n': 500, 'rougeL': ..., 'bertscore_f1': ..., 'accuracy': ..., 'format_ok': ...}.

FILE scripts/31_eval_metrics.py:

```
"""Evaluation step B: automatic metrics for every saved prediction file.

qa      -> accuracy (letter match), ROUGE-L, BERTScore
notes   -> ROUGE-L, BERTScore
summary -> JSON-valid rate, ROUGE-L, BERTScore
"""
import argparse
import json
import os
from pathlib import Path
from rouge_score import rouge_scorer                # Google's reference ROUGE implementation
from bert_score import score as bert_score          # BERTScore (Zhang et al., 2020)
from common import OUT, read_jsonl

p = argparse.ArgumentParser()
p.add_argument("--tasks", nargs="+", default=["qa", "notes", "summary"])
p.add_argument("--bert-model", default=os.getenv("BERTSCORE_MODEL", "roberta-large"))
a = p.parse_args()

rouge = rouge_scorer.RougeScorer(["rougeL"], use_stemmer=True)   # stemmer: "infections" ~ "infection"


def letter(text):
    """'Answer: C) ...' -> 'C' (None if the model did not follow the format)."""
    if text and "Answer:" in text:
        tail = text.split("Answer:", 1)[1].strip()
        return tail[:1].upper() or None
    return None


def is_json(text):
    try:
        json.loads(text.strip().removeprefix("```json").removesuffix("```").strip())
        return True
    except (json.JSONDecodeError, AttributeError):
        return False


results = {}
for task in a.tasks:
    for path in sorted(Path(OUT / "preds" / task).glob("*.jsonl")):
        rows = read_jsonl(path)
        preds = [r["prediction"] or "" for r in rows]
        refs = [r["reference"] for r in rows]
        m = {"n": len(rows)}
        m["rougeL"] = sum(rouge.score(ref, pred)["rougeL"].fmeasure for ref, pred in zip(refs, preds)) / len(rows)
        _, _, f1 = bert_score(preds, refs, model_type=a.bert_model, lang="en", verbose=False)
        m["bertscore_f1"] = f1.mean().item()
        if task == "qa":
            m["accuracy"] = sum(letter(p_) == r["answer_letter"] for p_, r in zip(preds, rows)) / len(rows)
            m["format_ok"] = sum(letter(p_) is not None for p_ in preds) / len(rows)
```

```
        if task == "summary":
            m["json_valid"] = sum(is_json(p_) for p_ in preds) / len(rows)
        results[f"{task}/{path.stem}"] = {k: round(v, 4) if isinstance(v, float) else v for k, v in m.items()}
        print(f"{task:8} {path.stem:12} {results[f'{task}/{path.stem}']}")

report = OUT / "reports" / "metrics.json"
report.parent.mkdir(parents=True, exist_ok=True)
report.write_text(json.dumps(results, indent=1))
print(f"saved {report}")
```

Line	What it does (plain English)
`from rouge_score import rouge_scorer`	Google's reference ROUGE implementation
`from bert_score import score as bert_score`	BERTScore: compares answers by meaning using contextual embeddings
`--bert-model roberta-large`	the standard BERTScore backbone for English
`RougeScorer(["rougeL"], use_stemmer=True)`	ROUGE-L = longest common subsequence of words; the stemmer treats "infections" and "infection" as equal
`def letter(text)`	reads the letter after "Answer:"
`def is_json(text)`	True if the text parses as JSON (also accepts a ```json fenced block)
`rouge.score(ref, pred)["rougeL"].fmeasure`	F1 of LCS-precision and LCS-recall, averaged over rows
`bert_score(preds, refs, model_type=..., lang="en")`	returns precision, recall and F1 tensors; we keep the mean F1
`m["accuracy"] = sum(letter(p_) == r["answer_letter"] ...)`	the most honest QA metric: fraction of correct letters
`m["format_ok"]`	fraction of answers that followed the "Answer: X)" format at all
`m["json_valid"]`	fraction of summary outputs that are valid JSON
`report.write_text(json.dumps(results, indent=1))`	saves all numbers for the report and the API docs

How to read the numbers:

Metric	Good for	Trap
accuracy (qa)	multiple choice: exact and meaningful	needs the format to be parseable; check format_ok first
ROUGE-L	"did it use the reference's words?"	a correct note with different wording scores low; a fluent wrong note can score high
BERTScore F1	meaning overlap	values cluster high (0.80-0.90 is common); compare models with each other, never to a fixed "good" value
json_valid	structured output	valid JSON can still contain invented facts: the judge checks that

Watch out: do not report BERTScore as "% correct". It is a similarity score.

CHECKPOINT: cat ../outputs/reports/metrics.json shows qa/base, qa/qa, notes/base, notes/notes, summary/base, summary/summary.

SECTION 5.7 - LLM-as-a-judge (score mode)

WHERE: VS Code Terminal, folder ~/-MEDSCRIPTAI/scripts, (.venv-eval) active

GOAL: a 1-5 quality score per model from the judge, on 100 test items per task.

Step 1. Run:

```
python 32_judge.py --task notes --mode score --models base notes --n 100 && python 32_judge.py --task summary --mode score --models base summary --n 100
```

You should see: JSON such as {"base": {"mean_score": ..., "judged": 100}, "notes": {...}}.

FILE scripts/32_judge.py:

```
"""Evaluation step C: LLM-as-a-judge.

--mode score     one answer at a time, 1-5 rubric score            (Part 5)
--mode pairwise  model A vs model B, run twice with order swapped    (Part 6)
                 -> removes position bias: a "win" must win in BOTH orders
```

```
"""
import argparse
import json
import os
import re
from concurrent.futures import ThreadPoolExecutor
from openai import OpenAI
from common import OUT, read_jsonl

p = argparse.ArgumentParser()
p.add_argument("--task", required=True, choices=["qa", "notes", "summary"])
p.add_argument("--mode", default="score", choices=["score", "pairwise"])
p.add_argument("--models", nargs="+", required=True, help="score: any number; pairwise: exactly two (A B)")
p.add_argument("--n", type=int, default=100)
a = p.parse_args()

judge = OpenAI(base_url=os.getenv("JUDGE_URL", "http://localhost:8001/v1"), api_key="EMPTY")
JUDGE_MODEL = os.getenv("JUDGE_MODEL", "teacher")
RUBRIC = {
    "qa": "correct final answer (vs the reference), sound medical reasoning, no unsafe advice",
    "notes": "faithful to the conversation (no invented facts), complete key findings, clear clinical style",
    "summary": "valid JSON, every value supported by the note, no important fact missing",
}


def ask_json(prompt):
    """Call the judge at temperature 0 and pull the first {...} object out of the reply."""
    text = judge.chat.completions.create(
        model=JUDGE_MODEL, temperature=0.0, max_tokens=300,
        messages=[{"role": "user", "content": prompt}]).choices[0].message.content
    match = re.search(r"\{.*\}", text, re.DOTALL)
    try:
        return json.loads(match.group(0)) if match else {}
    except json.JSONDecodeError:
        return {}


def score_prompt(row):
    return (f"You are a strict medical expert grader. Criteria: {RUBRIC[a.task]}.\n\n"
            f"TASK INPUT:\n{row['prompt'][:4000]}\n\nREFERENCE ANSWER:\n{row['reference'][:3000]}\n\n"
            f"MODEL ANSWER:\n{row['prediction'][:3000]}\n\n"
            'Return ONLY a JSON object: {"score": <1-5>, "reason": "<one sentence>"}')


def pair_prompt(row, first, second):
    return (f"You are a strict medical expert judge. Criteria: {RUBRIC[a.task]}.\n\n"
            f"TASK INPUT:\n{row['prompt'][:4000]}\n\nREFERENCE ANSWER:\n{row['reference'][:3000]}\n\n"
            f"ANSWER A:\n{first[:3000]}\n\nANSWER B:\n{second[:3000]}\n\n"
            'Which answer is better? Return ONLY a JSON object: {"winner": "A" or "B" or "tie", "reason": "..."}')


preds = {m: read_jsonl(OUT / "preds" / a.task / f"{m}.jsonl")[: a.n] for m in a.models}
report = {}
with ThreadPoolExecutor(max_workers=16) as pool:
    if a.mode == "score":
        for m, rows in preds.items():
            scores = [r.get("score") for r in pool.map(lambda row: ask_json(score_prompt(row)), rows)]
            valid = [s for s in scores if isinstance(s, (int, float))]
            report[m] = {"mean_score": round(sum(valid) / max(len(valid), 1), 3), "judged": len(valid)}
    else:
        ma, mb = a.models
        rows_a, rows_b = preds[ma], preds[mb]

        def duel(i):
            ab = ask_json(pair_prompt(rows_a[i], rows_a[i]["prediction"], rows_b[i]["prediction"]))
            ba = ask_json(pair_prompt(rows_a[i], rows_b[i]["prediction"], rows_a[i]["prediction"]))
            first, second = ab.get("winner"), ba.get("winner")
            if first == "A" and second == "B":
                return ma                                   # A won in both orders
            if first == "B" and second == "A":
                return mb
            return "tie"                                    # disagreement across orders = tie

        outcomes = list(pool.map(duel, range(min(len(rows_a), len(rows_b)))))
        n = len(outcomes)
        report = {f"{ma}_wins": round(outcomes.count(ma) / n, 3),
```

```
                  f"{mb}_wins": round(outcomes.count(mb) / n, 3),
                  "ties": round(outcomes.count("tie") / n, 3), "n": n,
                  "per_item": outcomes}                    # kept for judge-vs-human agreement (Part 7)

print(json.dumps({k: v for k, v in report.items() if k != "per_item"}, indent=1))
path = OUT / "reports" / f"judge_{a.task}_{a.mode}_{'_vs_'.join(a.models)}.json"
path.parent.mkdir(parents=True, exist_ok=True)
path.write_text(json.dumps(report, indent=1))
```

Line	What it does (plain English)
`judge = OpenAI(base_url=os.getenv("JUDGE_URL", "http://localhost:8001/v1"), ...)`	the judge is the teacher server on port 8001
`RUBRIC = {...}`	what "good" means per task: for notes, faithfulness (no invented facts) first
`def ask_json(prompt)`	asks at temperature 0 and extracts the first {...} with a regex; returns {} if the judge broke the format
`def score_prompt(row)`	task input + reference + model answer + "Return ONLY a JSON object {score, reason}"; [:4000] cuts very long inputs
`def pair_prompt(row, first, second)`	the pairwise version: answer A vs answer B (used in Part 6)
`preds = {m: read_jsonl(...)[: a.n] for m in a.models}`	the first n predictions of each model
`if a.mode == "score":`	grade each answer alone, then average the valid scores
`def duel(i)`	pairwise: ask twice, once A-then-B and once B-then-A
`if first == "A" and second == "B": return ma`	a model wins only if the judge prefers it in BOTH orders; otherwise it is a tie. This cancels position bias, a well-documented judge weakness (Zheng et al., 2023)
`report = {...wins, ties, n, "per_item": outcomes}`	win rates plus each verdict (Part 7 checks the judge against your own labels)
`print(json.dumps({... if k != "per_item"}))`	prints the summary only
`path = OUT / "reports" / f"judge_{task}_{mode}_{models}.json"`	one report file per run

Watch out: here the judge is also the teacher that wrote the synthetic explanations, so it may favour answers in its own style (self-preference bias). Say this in interviews. The Elite part shows how to measure judge reliability.

CHECKPOINT: outputs/reports has judge_notes_score_base_notes.json and judge_summary_score_base_summary.json.

SECTION 5.8 - Publish your datasets

WHERE: VS Code Terminal, folder ~/-MEDSCRIPTAI/scripts, (.venv-train) active

GOAL: two private dataset repos on the Hub.

Step 1. Publish the SFT data:

```
python 12_push_to_hub.py --type dataset --folder ../data/sft --repo YOUR_HF_USERNAME/medscriptai-sft
```

Step 2. Publish the synthetic explanations:

```
python 12_push_to_hub.py --type dataset --folder ../data/synth --repo YOUR_HF_USERNAME/medscriptai-medqa-explanations
```

Step 3. On each dataset page click Edit dataset card and write: sources and licences (MedQA CC-BY-4.0, MedMCQA Apache-2.0, PubMedQA MIT, augmented-clinical-notes MIT), how you cleaned the data, that explanations are synthetic from Qwen2.5-14B-Instruct, and "not for clinical use".

Watch out: CC-BY-4.0 requires attribution: credit the MedQA authors in the card.

CHECKPOINT: both dataset pages load and show your files.

SECTION 5.9 - Cloud step: reports to S3 and your token into Secrets Manager

WHERE: VS Code Terminal, folder ~/-MEDSCRIPTAI

GOAL: practise storing artifacts and secrets the way production systems do.

Step	FakeCloud (free practice)	Real AWS (same step)

1 Point the CLI	`export AWS_ENDPOINT_URL=http://localhost:4566 AWS_ACCESS_KEY_ID=test AWS_SECRET_ACCESS_KEY=test AWS_DEFAULT_REGION=us-east-1`	`unset AWS_ENDPOINT_URL`
2 Bucket name	`export BUCKET=medscriptai-artifacts-$(aws sts get-caller-identity --query Account --output text)-us-east-1`	same
3 Upload reports	`aws s3 sync outputs/reports s3://$BUCKET/reports`	same
4 Upload all adapters	`aws s3 sync outputs/adapters s3://$BUCKET/adapters --exclude "*/checkpoint-*"`	same
5 Store the HF token	`aws secretsmanager create-secret --name medscriptai/HF_TOKEN --secret-string "$HF_TOKEN"`	same (costs about US$0.40 per secret per month (verify); delete it in teardown)
6 Read it back	`aws secretsmanager get-secret-value --secret-id medscriptai/HF_TOKEN --query SecretString --output text`	same
You should see	your hf_ token printed	your hf_ token printed; also visible (masked) in AWS console > Secrets Manager

What each command piece means:

Piece	What it does
`--exclude "*/checkpoint-*"`	skip training checkpoints (large, not needed for serving)
`create-secret --name medscriptai/HF_TOKEN`	stores an encrypted secret; the slash is just a naming convention for grouping
`--query SecretString --output text`	print only the secret value

Watch out: printing a secret is fine for this exercise in your own terminal; never print secrets in logs or CI.

CHECKPOINT: aws s3 ls s3://$BUCKET/adapters/ lists sft-qa, sft-notes, sft-summary.

5.10 Common mistake at this level

The mistake: celebrating a metric that does not measure what you think. Examples: ROUGE-L goes up because the fine-tuned model copies the reference style while facts get worse; accuracy goes up because test questions leaked into training; the judge prefers the fine-tuned model because it writes like the judge.

How to spot it: always read 10 random predictions next to the references; check format_ok before accuracy; keep the decontamination step (Section 4.2) and the contamination check (Section 7.1); compare at least two independent metrics; run the significance test (Section 7.2) before claiming a win.

PART 6 - ADVANCED: DPO, multi-LoRA serving, production API and cloud deployment

6.0 Concept

What: build preference pairs from your own SFT model's answers, align the model with DPO, serve base + all adapters from one vLLM process with hot-swapping, put a FastAPI service with LangSmith tracing in front, package it with Docker, push it to ECR and run it on an AWS GPU machine (rehearsed first on FakeCloud).

Why: SFT teaches the model to imitate; DPO teaches it to prefer better answers, which improves correctness, safety and style without a separate reward model or RL loop. Serving and deployment turn a notebook experiment into a system other people can call.

When: DPO after SFT has converged and you can produce comparisons; deployment once the evaluation says the model is worth serving.

Harder than Part 5 because: you now generate data with your own model, train against a reference model, run two GPU servers plus an API, and operate cloud resources that cost real money. Budget: about 5 GPU-hours (about US$7) + about US$3-5 on AWS if you do the blue column.

SECTION 6.1 - Build DPO preference pairs with distilabel

WHERE: VS Code Terminal, tmux session, folder ~/-MEDSCRIPTAI/scripts, (.venv-synth) active. Both servers from Section 5.4 must be running (student on 8000, teacher on 8001).

GOAL: data/pref/qa_pairs.jsonl and data/pref/notes_pairs.jsonl in TRL's conversational preference format.

Step 1. Check both servers:

```
curl -s localhost:8000/v1/models | jq -r '.data[].id' && curl -s localhost:8001/v1/models | jq -r '.data[].id'
```

You should see: base, qa, notes, summary and then teacher.

Step 2. Start tmux and activate:

```
tmux new -s pairs
```

```
cd ~/-MEDSCRIPTAI && source .venv-synth/bin/activate && cd scripts
```

Step 3. Build QA pairs from 1,500 unseen prompts (about 20-40 minutes, estimate):

```
python 21_build_dpo_pairs.py --task qa --limit 1500
```

You should see: wrote ~600-1200 rows → data/pref/qa_pairs.jsonl and N preference pairs from 1500 prompts (many prompts are skipped because all candidates were equally good: that is expected).

Step 4. Build notes pairs from 600 prompts:

```
python 21_build_dpo_pairs.py --task notes --limit 600
```

Step 5. Look at one pair:

```
head -n 1 ../data/pref/qa_pairs.jsonl | python -m json.tool
```

You should see: "prompt" (system + user), "chosen" (one assistant message, usually the correct letter), "rejected" (one assistant message, often a wrong letter), "score_chosen" and "score_rejected".

FILE scripts/21_build_dpo_pairs.py:

```
"""Part 6: build DPO preference pairs (prompt, chosen, rejected) with distilabel.

For each held-out prompt we collect 3 candidate answers:
  2 samples from YOUR SFT adapter (served by vLLM under the adapter name, e.g. "qa")
  1 answer from the teacher model
The teacher then rates all 3 (UltraFeedback, 1-5). For qa we ALSO check the letter against the gold answer,
because a verifiable signal beats any judge. Best candidate = chosen, worst = rejected.
"""
import argparse
import os
from distilabel.models import OpenAILLM
from distilabel.pipeline import Pipeline
```

```
from distilabel.steps import GroupColumns, LoadDataFromDicts
from distilabel.steps.tasks import TextGeneration, UltraFeedback
from common import PREF, SYSTEM, read_jsonl, write_jsonl

p = argparse.ArgumentParser()
p.add_argument("--task", default="qa", choices=["qa", "notes", "summary"])
p.add_argument("--limit", type=int, default=1500)
a = p.parse_args()

STUDENT_URL = os.getenv("STUDENT_URL", "http://localhost:8000/v1")  # vLLM with your SFT adapters
TEACHER_URL = os.getenv("TEACHER_URL", "http://localhost:8001/v1")  # vLLM with the teacher
TEACHER = os.getenv("TEACHER_MODEL", "teacher")

rows = read_jsonl(PREF / f"{a.task}_dpo.jsonl")[: a.limit]
inputs = [{"instruction": r["messages"][1]["content"],                # the user message
           "gold_letter": r.get("answer_letter")} for r in rows]      # None for notes/summary


def llm(model, url, temperature):
    return OpenAILLM(model=model, base_url=url, api_key="EMPTY",
                     generation_kwargs={"temperature": temperature, "max_new_tokens": 700})


with Pipeline(name=f"medscriptai-dpo-{a.task}") as pipeline:
    load = LoadDataFromDicts(data=inputs, batch_size=64)
    student_1 = TextGeneration(name="student_1", llm=llm(a.task, STUDENT_URL, 0.9),
                               system_prompt=SYSTEM[a.task], input_batch_size=32)
    student_2 = TextGeneration(name="student_2", llm=llm(a.task, STUDENT_URL, 0.9),
                               system_prompt=SYSTEM[a.task], input_batch_size=32)
    teacher = TextGeneration(name="teacher", llm=llm(TEACHER, TEACHER_URL, 0.3),
                             system_prompt=SYSTEM[a.task], input_batch_size=32)
    group = GroupColumns(columns=["generation", "model_name"],          # 3 single answers -> 1 list
                         output_columns=["generations", "generation_models"])
    rate = UltraFeedback(llm=llm(TEACHER, TEACHER_URL, 0.0), aspect="overall-rating", input_batch_size=16)
    load >> [student_1, student_2, teacher] >> group >> rate            # fan out, then join

data = pipeline.run(use_cache=False)["default"]["train"]


def letter_of(text):
    """Read the letter after 'Answer:' ('Answer: C) ...' -> 'C')."""
    if text and "Answer:" in text:
        tail = text.split("Answer:", 1)[1].strip()
        return tail[:1].upper() if tail else None
    return None


pairs = []
for row in data:
    scored = []
    for text, rating in zip(row["generations"], row["ratings"]):
        if not text or rating is None:
            continue
        correct = 1 if row["gold_letter"] and letter_of(text) == row["gold_letter"] else 0
        scored.append((correct * 10 + rating, text))    # correctness dominates, rating breaks ties
    if len(scored) < 2:
        continue
    scored.sort(key=lambda s: s[0])
    (low, rejected), (high, chosen) = scored[0], scored[-1]
    if high - low < 2:                                  # too similar = weak training signal, skip
        continue
    system = {"role": "system", "content": SYSTEM[a.task]}
    pairs.append({"prompt": [system, {"role": "user", "content": row["instruction"]}],
                  "chosen": [{"role": "assistant", "content": chosen}],
                  "rejected": [{"role": "assistant", "content": rejected}],
                  "score_chosen": high, "score_rejected": low})

write_jsonl(PREF / f"{a.task}_pairs.jsonl", pairs)
print(f"{len(pairs)} preference pairs from {len(inputs)} prompts")
```

Line	What it does (plain English)
`from distilabel.steps import GroupColumns, LoadDataFromDicts`	GroupColumns merges the outputs of parallel steps into lists
`from distilabel.steps.tasks import TextGeneration, UltraFeedback`	UltraFeedback = the rating task from the UltraFeedback paper (Cui et al., 2023): the judge rates several answers 1-5

`STUDENT_URL` (port 8000)	vLLM with YOUR SFT adapters; asking for model "qa" means base + sft-qa adapter
`rows = read_jsonl(PREF / f"{a.task}_dpo.jsonl")[: a.limit]`	the held-out DPO prompts written in Section 4.2 (never used in SFT)
`"gold_letter": r.get("answer_letter")`	the correct letter for qa; None for notes
`def llm(model, url, temperature)`	a small factory so every step uses the same settings
`student_1`, `student_2` at temperature 0.9	two DIFFERENT samples from your SFT model: high temperature creates variety, including mistakes worth un-learning
`teacher` at temperature 0.3	a third, usually stronger candidate
`GroupColumns(columns=["generation", "model_name"], output_columns=["generations", "generation_models"])`	turns the three single answers into one list of three
`UltraFeedback(llm=..., aspect="overall-rating")`	the teacher rates all three answers together (1-5) with a rationale
`load >> [student_1, student_2, teacher] >> group >> rate`	fan-out to three generators in parallel, join, then rate
`def letter_of(text)`	extracts the letter after "Answer:"
`correct = 1 if row["gold_letter"] and letter_of(text) == row["gold_letter"] else 0`	the verifiable signal: is the letter right?
`scored.append((correct * 10 + rating, text))`	correctness dominates (10 points), the judge's rating breaks ties. Never let a judge outvote ground truth
`if high - low < 2: continue`	skip near-ties: a pair with no real difference teaches noise
`pairs.append({"prompt": [system, user], "chosen": [...], "rejected": [...]})`	exactly the conversational "explicit prompt" format TRL's DPOTrainer expects
`"score_chosen": high, "score_rejected": low`	kept for analysis (TRL ignores extra columns after select_columns in Section 6.2)

Watch out: this is "on-policy" preference data (the rejected answers come from your own model). It is more useful than random pairs because it targets the mistakes YOUR model actually makes (the idea behind iterative/on-policy DPO, e.g. Tulu 3).

CHECKPOINT: wc -l ../data/pref/qa_pairs.jsonl is at least 300 and notes_pairs.jsonl at least 150.

If not: if almost every prompt is skipped, your SFT model and the teacher agree on nearly everything: lower the threshold from 2 to 1, or use more prompts.

SECTION 6.2 - Train DPO on top of the SFT adapter

WHERE: VS Code Terminal, tmux session, folder ~/-MEDSCRIPTAI/scripts, (.venv-train) active

GOAL: outputs/adapters/dpo-qa and dpo-notes.

Step 1. Free the GPU (stop both servers):

```
docker rm -f medscriptai-vllm medscriptai-teacher
```

Step 2. Start tmux and activate:

```
tmux new -s dpo
```

```
cd ~/-MEDSCRIPTAI && source .venv-train/bin/activate && cd scripts
```

Step 3. Train (about 20-40 minutes per task, estimate):

```
python 22_train_dpo.py --task qa && python 22_train_dpo.py --task notes
```

You should see: adapters inside the model: ['default', 'ref'], then log lines with loss starting at about 0.693, and rewards/accuracies rising above 0.5. At the end a dict such as {'eval_rewards/accuracies': 0.7, 'eval_rewards/margins': 0.4, ...}.

FILE scripts/22_train_dpo.py:

```
"""Part 6: DPO preference alignment that continues from your SFT adapter.

python scripts/22_train_dpo.py --task qa     ->  outputs/adapters/dpo-qa
"""
from unsloth import FastLanguageModel, PatchDPOTrainer   # Unsloth first
PatchDPOTrainer()                                         # no-op in current Unsloth; kept for older versions
```

```
import argparse
import json
from datasets import load_dataset
from trl import DPOConfig, DPOTrainer
from common import PREF, OUT

p = argparse.ArgumentParser()
p.add_argument("--task", default="qa", choices=["qa", "notes", "summary"])
p.add_argument("--beta", type=float, default=0.1)       # how strongly to stay close to the SFT model
p.add_argument("--lr", type=float, default=5e-6)        # DPO needs a MUCH smaller LR than SFT
p.add_argument("--epochs", type=float, default=1.0)
a = p.parse_args()
sft_dir = OUT / "adapters" / f"sft-{a.task}"
out_dir = OUT / "adapters" / f"dpo-{a.task}"

# 1) Load base + your SFT adapter as TRAINABLE. DPO will keep improving these same LoRA weights.
model, tokenizer = FastLanguageModel.from_pretrained(str(sft_dir), max_seq_length=2048, load_in_4bit=True)
tokenizer.padding_side = "left"                          # TRL's DPO collator expects left padding

# 2) Preference data: {"prompt": [...], "chosen": [...], "rejected": [...]} (conversational format).
pairs = load_dataset("json", data_files=str(PREF / f"{a.task}_pairs.jsonl"), split="train")
pairs = pairs.select_columns(["prompt", "chosen", "rejected"])
split = pairs.train_test_split(test_size=0.05, seed=42)  # 5% held out to watch reward accuracy

cfg = DPOConfig(
    output_dir=str(OUT / "checkpoints" / f"dpo-{a.task}"),
    beta=a.beta,
    loss_type=["sigmoid"],               # the original DPO loss
    max_length=2048,                     # prompt + answer are truncated to this many tokens
    per_device_train_batch_size=2,       # each example = 2 sequences (chosen + rejected)
    gradient_accumulation_steps=8,       # effective batch 16 pairs
    num_train_epochs=a.epochs,
    learning_rate=a.lr,
    lr_scheduler_type="cosine",
    warmup_steps=10,
    optim="adamw_8bit",
    bf16=True,
    logging_steps=10,
    eval_strategy="steps",
    eval_steps=50,
    save_steps=200,
    save_total_limit=2,
    report_to="none",
    seed=3407,
)

# ref_model=None + a model that already has an adapter -> TRL copies it into a frozen "ref" adapter,
# so the reference policy is your SFT model (textbook DPO) without loading a second 8B model.
trainer = DPOTrainer(model=model, ref_model=None, args=cfg, processing_class=tokenizer,
                     train_dataset=split["train"], eval_dataset=split["test"])
print("adapters inside the model:", list(trainer.model.peft_config.keys()))   # expect ['default', 'ref']

trainer.train()
metrics = trainer.evaluate()
print({k: round(v, 4) for k, v in metrics.items() if "rewards" in k})   # accuracies should be > 0.5

# Save ONLY the trained adapter ("default"), not the frozen "ref" copy.
model.save_pretrained(str(out_dir), selected_adapters=["default"])
tokenizer.save_pretrained(str(out_dir))
(out_dir / "dpo_eval.json").write_text(json.dumps(metrics, indent=1))
print(f"DONE adapter={out_dir}")
```

Line	What it does (plain English)
`from unsloth import FastLanguageModel, PatchDPOTrainer`	Unsloth first. In Unsloth 2026.10 PatchDPOTrainer() is a no-op (the source simply returns); it is kept so the script also works with older Unsloth versions
`--beta 0.1`	strength of the "stay close to the reference" pull. Higher beta = smaller changes
`--lr 5e-6`	40x smaller than SFT. DPO with a large LR quickly breaks fluency
`sft_dir = OUT / "adapters" / f"sft-{a.task}"`	start from the SFT adapter, not from the raw base model
`FastLanguageModel.from_pretrained(str(sft_dir), ...)`	loads 4-bit base + SFT LoRA; Unsloth loads adapters as trainable (is_trainable=True in its loader)

`tokenizer.padding_side = "left"`	TRL's DPO collator pads on the left
`load_dataset("json", data_files=...)` then `select_columns(["prompt", "chosen", "rejected"])`	keep only the three columns DPO needs
`train_test_split(test_size=0.05, seed=42)`	5% of pairs held out to measure reward accuracy on unseen pairs
`beta=a.beta`	see above
`loss_type=["sigmoid"]`	the original DPO loss: -log sigmoid(beta x (chosen log-ratio - rejected log-ratio)). TRL 1.x also offers IPO, SimPO-style sigmoid_norm, APO, etc.
`max_length=2048`	prompt + answer truncated to 2048 tokens
`per_device_train_batch_size=2`, `gradient_accumulation_steps=8`	each pair = 2 sequences (chosen + rejected), so batch 2 pairs; x8 = effective 16 pairs
`learning_rate`, `lr_scheduler_type="cosine"`, `warmup_steps=10`, `optim="adamw_8bit"`, `bf16=True`	same meaning as in SFT (Section 4.4)
`eval_steps=50`	reward accuracy on held-out pairs every 50 steps
`DPOTrainer(model=model, ref_model=None, ...)`	ref_model=None + a model that already has an adapter: TRL 1.15 COPIES the SFT adapter into a frozen adapter named "ref" and uses it as the reference policy. Textbook DPO (reference = SFT model) without loading a second 8B model
`print("adapters inside the model:", ...)`	proof: you must see ['default', 'ref']. If you only see ['default'], the reference is the base model (still works, different maths)
`trainer.train()`	at step 0 the policy equals the reference, so every log-ratio is 0 and loss = -log sigmoid(0) = ln 2 = 0.693
`metrics = trainer.evaluate()`	eval loss and rewards on the 5% split
`rewards/accuracies`	fraction of pairs where the model now gives the chosen answer a higher implicit reward; above 0.5 = it learned the preference
`rewards/margins`	average reward gap chosen - rejected; should be positive and growing
`model.save_pretrained(str(out_dir), selected_adapters=["default"])`	save only the trained adapter, not the frozen "ref" copy
`(out_dir / "dpo_eval.json").write_text(...)`	keep the DPO metrics for your report

Watch out: if rewards/chosen AND rewards/rejected both fall fast and generated answers get strange, the LR is too high or beta too low (a known DPO failure: it pushes everything down). Use --lr 2e-6 or --beta 0.2.

CHECKPOINT: ls ../outputs/adapters shows dpo-qa and dpo-notes; eval_rewards/accuracies is above 0.5 for both.

If not: fewer than ~200 pairs often gives noisy results; build more pairs (Section 6.1) and retrain.

SECTION 6.3 - vLLM multi-LoRA: one base model, five adapters, hot-swapping

WHERE: VS Code Terminal, folder ~/-MEDSCRIPTAI

GOAL: serve base + sft-qa + sft-notes + sft-summary + dpo-qa + dpo-notes from one GPU process, and add/remove adapters without restarting.

Step 1. Start the student server alone with most of the GPU:

```
GPU_UTIL=0.60 bash serve/start_vllm.sh
```

You should see: Adapters: qa=/adapters/sft-qa notes=/adapters/sft-notes summary=/adapters/sft-summary qa-dpo=/adapters/dpo-qa notes-dpo=/adapters/dpo-notes

Step 2. Wait and list the models:

```
until curl -s localhost:8000/v1/models > /dev/null; do sleep 10; done; curl -s localhost:8000/v1/models | jq -r
'.data[].id'
```

You should see: base, qa, notes, summary, qa-dpo, notes-dpo.

Step 3. Send the SAME question to two adapters:

```
for m in base qa-dpo; do curl -s localhost:8000/v1/chat/completions -H "Content-Type: application/json" -d "
{\"model\":\"$m\",\"max_tokens\":120,\"messages\":[{\"role\":\"user\",\"content\":\"A pregnant woman has cystitis. Which
is safest?\n\nOptions:\nA) Doxycycline\nB) Nitrofurantoin\nC) Ciprofloxacin\nD) Tetracycline\"}]}" | jq -r
'.choices[0].message.content'; echo ---; done
```

You should see: two different answers from the same server; the qa-dpo one starts with "Answer: B)".

Step 4. Hot-unload an adapter at runtime:

```
curl -s -X POST localhost:8000/v1/unload_lora_adapter -H "Content-Type: application/json" -d '{"lora_name": "notes-
dpo"}'
```

Step 5. Hot-load it again under a new name (for example a new version):

```
curl -s -X POST localhost:8000/v1/load_lora_adapter -H "Content-Type: application/json" -d '{"lora_name": "notes-dpo-
v2", "lora_path": "/adapters/dpo-notes"}'
```

Step 6. List again:

```
curl -s localhost:8000/v1/models | jq -r '.data[].id'
```

You should see: notes-dpo is gone and notes-dpo-v2 is present, with no restart and no downtime.

FILE serve/start_vllm.sh:

```
#!/usr/bin/env bash
# Serve Llama-3.1-8B-Instruct + all MedScriptAI LoRA adapters from ONE GPU process (multi-LoRA).
# Usage:  bash serve/start_vllm.sh            (from the repo folder)
# Model names clients can use: base, qa, notes, summary, qa-dpo, notes-dpo
set -euo pipefail                                   # stop on the first error

VLLM_IMAGE="${VLLM_IMAGE:-vllm/vllm-openai:v0.32.0-cu129}"   # cu129 build + compat for R570 drivers
GPU_UTIL="${GPU_UTIL:-0.85}"                        # share of GPU memory vLLM may use (0.35 when the teacher also runs)
ADAPTERS="$(pwd)/outputs/adapters"                  # trained adapters on the VM

# Only register adapters that exist, so the script works after Part 4 (qa only) and after Part 6 (all).
LORA_ARGS=()
for pair in qa=sft-qa notes=sft-notes summary=sft-summary qa-dpo=dpo-qa notes-dpo=dpo-notes; do
  name="${pair%%=*}"; dir="${pair#*=}"
  if [ -f "$ADAPTERS/$dir/adapter_config.json" ]; then LORA_ARGS+=("$name=/adapters/$dir"); fi
done
echo "Adapters: ${LORA_ARGS[*]:-none}"
EXTRA=()                                            # becomes: --lora-modules qa=/adapters/sft-qa ...
if [ "${#LORA_ARGS[@]}" -gt 0 ]; then EXTRA=(--lora-modules "${LORA_ARGS[@]}"); fi

docker rm -f medscriptai-vllm 2>/dev/null || true   # remove an old container with the same name
docker run -d --name medscriptai-vllm \
  --gpus all --ipc=host \
  -p 8000:8000 \
  -e HF_TOKEN="${HF_TOKEN:-}" \
  -e VLLM_ENABLE_CUDA_COMPATIBILITY="${VLLM_ENABLE_CUDA_COMPATIBILITY:-1}" \
  -e VLLM_ALLOW_RUNTIME_LORA_UPDATING=True \
  -v "${HF_HOME:-$HOME/.cache/huggingface}":/root/.cache/huggingface \
  -v "$ADAPTERS":/adapters:ro \
  "$VLLM_IMAGE" \
  --model unsloth/Llama-3.1-8B-Instruct \
  --served-model-name base \
  --dtype bfloat16 \
  --max-model-len 4096 \
  --gpu-memory-utilization "$GPU_UTIL" \
  --enable-lora \
  --max-loras 4 \
  --max-lora-rank 16 \
  --max-cpu-loras 8 \
  "${EXTRA[@]}"

echo "Started. Follow the logs with:  docker logs -f medscriptai-vllm"
```

Line	What it does (plain English)
`VLLM_IMAGE=...v0.32.0-cu129`	official vLLM server image (CUDA 12.9 build, works on R570 with the compatibility flag)
`ADAPTERS="$(pwd)/outputs/adapters"`	adapters on the VM disk

`for pair in qa=sft-qa notes=sft-notes ... ; do ... done`	for each name=folder pair: if the folder has adapter_config.json, register it. So the script works after Part 4 (only qa) and after Part 6 (all five)
`name="${pair%%=*}"; dir="${pair#*=}"`	bash string cutting: text before "=" and text after "="
`EXTRA=(--lora-modules "${LORA_ARGS[@]}")`	builds the argument list only when at least one adapter exists
`-e VLLM_ALLOW_RUNTIME_LORA_UPDATING=True`	enables /v1/load_lora_adapter and /v1/unload_lora_adapter. vLLM's docs warn this is risky in production: anyone who can reach port 8000 could load files. That is why docker-compose.yml never publishes vLLM's port
`-v "$ADAPTERS":/adapters:ro`	the adapters folder appears inside the container at /adapters, read-only
`--model unsloth/Llama-3.1-8B-Instruct --served-model-name base`	the bf16 base model, called "base" by clients
`--max-model-len 4096`	longest context per request (our longest is about 1,600 tokens + answer)
`--enable-lora`	turns on LoRA support (vLLM's batched LoRA kernels, the Punica/S-LoRA idea: many adapters in one batch)
`--max-loras 4`	how many DIFFERENT adapters can be active in the same GPU batch. The default is 1: with the default, requests for different adapters queue behind each other. A classic production gotcha
`--max-lora-rank 16`	must be at least the largest adapter rank (ours is 16); allowed values are 1, 8, 16, 32, 64, 128, 256, 320, 512
`--max-cpu-loras 8`	how many adapters are cached in CPU RAM for fast swapping into the GPU
`--lora-modules qa=/adapters/sft-qa ...`	name=path pairs: the name is what clients put in the "model" field

How hot-swapping works: the 16 GB base weights stay on the GPU once. Each adapter is about 80-170 MB. For each request vLLM looks at the "model" name and applies that adapter's small matrices. Three adapters cost almost the same memory as one model, and one GPU can serve many specialised "models".

> **Watch out:** an adapter trained against the 4-bit base is served here on the bf16 base. This is standard practice and works well; the small mismatch is one reason evaluation must run on the SERVED setup (this server), not inside the training script.

> **CHECKPOINT:** Step 6 shows notes-dpo-v2.

> **If not:** "Loading lora ... failed" with a rank error means --max-lora-rank is smaller than the adapter rank.

SECTION 6.4 - Evaluate base vs SFT vs DPO

> **WHERE:** VS Code Terminal, folder ~/-MEDSCRIPTAI/scripts, (.venv-eval) active; student server on 8000

> **GOAL:** numbers that show whether DPO helped.

Step 1. Load the adapter back under its original name (Section 6.3 unloaded it):

```
curl -s -X POST localhost:8000/v1/load_lora_adapter -H "Content-Type: application/json" -d '{"lora_name": "notes-dpo", "lora_path": "/adapters/dpo-notes"}'
```

Step 2. Start the judge next to it:

```
GPU_UTIL=0.30 bash ../serve/start_teacher.sh
```

Step 3. Activate the evaluation environment:

```
cd ~/-MEDSCRIPTAI && source .venv-eval/bin/activate && cd scripts
```

Step 4. Generate predictions for the DPO adapters:

```
python 30_generate_preds.py --task qa --models qa-dpo && python 30_generate_preds.py --task notes --models notes-dpo
```

Step 5. Recompute all metrics:

```
python 31_eval_metrics.py
```

Step 6. Wait for the judge:

```
until curl -s localhost:8001/v1/models > /dev/null; do sleep 10; done; echo ready
```

Step 7. Pairwise judge with position swap (DPO vs SFT):

```
python 32_judge.py --task qa --mode pairwise --models qa-dpo qa --n 200 && python 32_judge.py --task notes --mode
pairwise --models notes-dpo notes --n 100
```

You should see: JSON like {"qa-dpo_wins": ..., "qa_wins": ..., "ties": ..., "n": 200}.

How to read the result:

If you see...	It means...	Do this next
qa-dpo accuracy higher and qa-dpo_wins > qa_wins	DPO helped	run the significance test (Section 7.2) before you claim it
accuracy equal, win rate higher	better explanations/style, same correctness	report both honestly
accuracy lower	DPO hurt correctness	lower LR, raise beta, check pair quality (Section 6.1 Step 5)
ties above 60%	the judge flips with order: weak preference signal	look at 10 duels yourself

Watch out: the student server was started with GPU_UTIL=0.60 in Section 6.3 and the judge uses 0.30, so together 0.90 of the GPU. If the judge does not start, restart the student with GPU_UTIL=0.45 bash serve/start_vllm.sh.

CHECKPOINT: metrics.json has qa/qa-dpo and notes/notes-dpo; two pairwise judge files exist.

SECTION 6.5 - The FastAPI service

WHERE: VS Code Terminal, folder ~/-MEDSCRIPTAI, (.venv-eval) active; student vLLM on 8000

GOAL: one HTTP endpoint that routes each task to the right adapter.

Step 1. Start the API in the foreground (replace dev-key with any string):

```
cd ~/-MEDSCRIPTAI/api && VLLM_URL=http://localhost:8000/v1 API_KEY=dev-key PYTHONPATH=../scripts uvicorn app:app --host
0.0.0.0 --port 8080
```

You should see: Uvicorn running on http://0.0.0.0:8080

Step 2. In a SECOND VS Code Terminal, check readiness:

```
curl -s localhost:8080/ready | jq
```

You should see: {"status": "ready", "models": ["base", "notes", "notes-dpo", "qa", "qa-dpo", "summary"]}

Step 3. Call the API:

```
curl -s -X POST localhost:8080/v1/generate -H "Content-Type: application/json" -H "X-API-Key: dev-key" -d
'{"task":"qa","input":"A pregnant woman has cystitis. Which is safest?\n\nOptions:\nA) Doxycycline\nB)
Nitrofurantoin\nC) Ciprofloxacin\nD) Tetracycline"}' | jq
```

You should see: request_id, task "qa", model "qa-dpo", output starting with "Answer: B)", latency_ms and the disclaimer.

Step 4. Open the interactive docs: in VS Code click the PORTS tab next to TERMINAL, click Forward a Port, type 8080, then open http://localhost:8080/docs in your laptop browser.

You should see: the Swagger UI with GET /health, GET /ready and POST /v1/generate.

Step 5. Stop the API with Ctrl+C in the first terminal.

FILE api/app.py:

```
"""MedScriptAI API: one endpoint, three tasks, each routed to its own LoRA adapter inside vLLM."""
import json
import os
import time
import uuid

from fastapi import FastAPI, Header, HTTPException
from langsmith import traceable                  # turns a function call into a LangSmith trace
from langsmith.wrappers import wrap_openai       # also traces every OpenAI-style call inside it
from openai import OpenAI
from pydantic import BaseModel, Field

from common import SYSTEM                        # SAME system prompts as training (no train/serve skew)

VLLM_URL = os.getenv("VLLM_URL", "http://localhost:8000/v1")
```

```
API_KEY = os.getenv("API_KEY", "")                # empty = no auth (local only); set it in production
TASK_TO_MODEL = {                                 # which adapter answers which task (change without code edits)
    "qa": os.getenv("MODEL_QA", "qa-dpo"),
    "notes": os.getenv("MODEL_NOTES", "notes"),
    "summary": os.getenv("MODEL_SUMMARY", "summary"),
}

llm = wrap_openai(OpenAI(base_url=VLLM_URL, api_key="EMPTY", timeout=120))
app = FastAPI(title="MedScriptAI", version="1.0.0")


class GenerateRequest(BaseModel):
    task: str = Field(pattern="^(qa|notes|summary)$", examples=["qa"])
    input: str = Field(min_length=1, max_length=20000)
    max_tokens: int = Field(default=700, ge=1, le=2000)
    temperature: float = Field(default=0.0, ge=0.0, le=1.5)


class GenerateResponse(BaseModel):
    request_id: str
    task: str
    model: str
    output: str
    parsed_json: dict | None = None               # filled for task="summary" when the output is valid JSON
    latency_ms: int
    disclaimer: str = "AI-generated. For education/documentation support only. Not medical advice."


@traceable(name="medscriptai.generate", run_type="chain")
def run_generation(task: str, text: str, max_tokens: int, temperature: float) -> dict:
    model = TASK_TO_MODEL[task]
    resp = llm.chat.completions.create(
        model=model,
        messages=[{"role": "system", "content": SYSTEM[task]}, {"role": "user", "content": text}],
        max_tokens=max_tokens,
        temperature=temperature,
    )
    return {"model": model, "output": resp.choices[0].message.content or ""}


@app.get("/health")
def health():
    """Liveness: the API process is up."""
    return {"status": "ok"}


@app.get("/ready")
def ready():
    """Readiness: vLLM answers and every adapter we route to is loaded."""
    try:
        served = {m.id for m in llm.models.list().data}
    except Exception as exc:
        raise HTTPException(503, f"vLLM not reachable: {exc.__class__.__name__}")
    missing = sorted(set(TASK_TO_MODEL.values()) - served)
    if missing:
        raise HTTPException(503, f"adapters not loaded in vLLM: {missing}")
    return {"status": "ready", "models": sorted(served)}


@app.post("/v1/generate", response_model=GenerateResponse)
def generate(req: GenerateRequest, x_api_key: str = Header(default="")):
    if API_KEY and x_api_key != API_KEY:
        raise HTTPException(401, "invalid or missing X-API-Key header")
    start = time.perf_counter()
    try:
        result = run_generation(req.task, req.input, req.max_tokens, req.temperature)
    except Exception as exc:
        raise HTTPException(502, f"model backend error: {exc.__class__.__name__}")
    parsed = None
    if req.task == "summary":
        try:
            parsed = json.loads(result["output"])
        except json.JSONDecodeError:
            parsed = None
    return GenerateResponse(request_id=str(uuid.uuid4()), task=req.task, model=result["model"],
```

```
                        output=result["output"], parsed_json=parsed,
                        latency_ms=int((time.perf_counter() - start) * 1000))
```

Line	What it does (plain English)
`from fastapi import FastAPI, Header, HTTPException`	FastAPI app object, header reading, HTTP errors
`from langsmith import traceable`	a decorator that records a function call as a LangSmith "run"
`from langsmith.wrappers import wrap_openai`	wraps the OpenAI client so every LLM call inside a trace becomes a child run with tokens and latency
`from pydantic import BaseModel, Field`	typed request/response models with automatic validation and docs
`from common import SYSTEM`	the SAME system prompts used in training (no train/serve skew)
`VLLM_URL = os.getenv(...)`	where vLLM is; in Docker Compose this becomes http://vllm:8000/v1
`API_KEY = os.getenv("API_KEY", "")`	shared secret for clients; empty disables auth (only for local tests)
`TASK_TO_MODEL = {"qa": os.getenv("MODEL_QA", "qa-dpo"), ...}`	routing table task → adapter, configurable without code changes (canary: point MODEL_QA to a new adapter, watch metrics, roll back by changing one variable)
`llm = wrap_openai(OpenAI(base_url=VLLM_URL, api_key="EMPTY", timeout=120))`	one shared client for all requests; 120 s timeout
`app = FastAPI(title="MedScriptAI", version="1.0.0")`	the web app; title/version appear in /docs
`class GenerateRequest(BaseModel)`	the request body schema
`task: str = Field(pattern="^(qa|notes|summary)$")`	only these three values pass; anything else gets HTTP 422 automatically
`input: str = Field(min_length=1, max_length=20000)`	rejects empty and huge inputs (cost and abuse protection)
`max_tokens ... ge=1, le=2000`, `temperature ... ge=0.0, le=1.5`	numeric bounds checked by Pydantic
`class GenerateResponse(BaseModel)`	the response schema shown in /docs
`parsed_json: dict | None = None`	for task summary, the parsed JSON object (or null if the model produced invalid JSON)
`disclaimer: str = "AI-generated ..."`	always tell users this is not medical advice
`@traceable(name="medscriptai.generate", run_type="chain")`	each call of run_generation becomes a LangSmith trace named medscriptai.generate
`def run_generation(task, text, max_tokens, temperature)`	picks the adapter and calls vLLM with system + user messages
`@app.get("/health")`	liveness probe: the process is alive (Docker HEALTHCHECK and load balancers use it)
`@app.get("/ready")`	readiness probe: vLLM answers AND every routed adapter is loaded; returns 503 otherwise, so traffic is not sent too early
`served = {m.id for m in llm.models.list().data}`	the model names vLLM currently serves
`missing = sorted(set(TASK_TO_MODEL.values()) - served)`	adapters we route to but vLLM does not have
`@app.post("/v1/generate", response_model=GenerateResponse)`	the main endpoint; response_model enforces the output schema
`x_api_key: str = Header(default="")`	FastAPI reads the X-API-Key header into this argument
`if API_KEY and x_api_key != API_KEY: raise HTTPException(401, ...)`	reject wrong or missing keys
`start = time.perf_counter()`	high-resolution timer for latency
`except Exception: raise HTTPException(502, ...)`	if vLLM fails, return 502 Bad Gateway (an upstream error), never a stack trace
`if req.task == "summary": parsed = json.loads(...)`	gives clients a ready JSON object; invalid JSON becomes null instead of an error
`request_id=str(uuid.uuid4())`	a unique ID per request for logs and support tickets

`latency_ms=int(...)`	measured server-side latency

Step 6. Run the API tests without a GPU, against the mock server (useful on any laptop and in CI):

```
cd ~/-MEDSCRIPTAI && (uvicorn tests.mock_openai:app --port 8009 > /tmp/mock.log 2>&1 &) && sleep 2 &&
VLLM_URL=http://localhost:8009/v1 python -m pytest -q tests/test_api.py
```

You should see: 5 passed.

FILE tests/test_api.py:

```
"""API tests that run on any laptop/CPU: start the mock first (uvicorn tests.mock_openai:app --port 8000)."""
import os
import sys

sys.path[:0] = ["api", "scripts"]                # import app.py and common.py
os.environ.setdefault("VLLM_URL", "http://localhost:8000/v1")
os.environ["API_KEY"] = "test-key"

from fastapi.testclient import TestClient          # noqa: E402
import app as api                                  # noqa: E402

client = TestClient(api.app)
HEADERS = {"X-API-Key": "test-key"}


def test_health():
    assert client.get("/health").json() == {"status": "ok"}


def test_ready_lists_adapters():
    body = client.get("/ready").json()
    assert body["status"] == "ready" and "qa-dpo" in body["models"]


def test_qa_routes_to_dpo_adapter():
    r = client.post("/v1/generate", headers=HEADERS, json={"task": "qa", "input": "Q?\n\nOptions:\nA) x\nB) y"})
    assert r.status_code == 200
    assert r.json()["model"] == "qa-dpo" and r.json()["output"].startswith("Answer:")


def test_summary_returns_parsed_json():
    r = client.post("/v1/generate", headers=HEADERS, json={"task": "summary", "input": "A 40-year-old ..."})
    assert r.json()["parsed_json"]["patient information"]["age"] == "40"


def test_auth_and_validation():
    assert client.post("/v1/generate", json={"task": "qa", "input": "x"}).status_code == 401
    assert client.post("/v1/generate", headers=HEADERS, json={"task": "bad", "input": "x"}).status_code == 422
```

Line	What it does (plain English)
`sys.path[:0] = ["api", "scripts"]`	lets the test import app.py and common.py
`os.environ["API_KEY"] = "test-key"`	set BEFORE importing app, because app reads it at import time
`client = TestClient(api.app)`	calls the app in-process, no real network needed
`test_health`	/health returns {"status": "ok"}
`test_ready_lists_adapters`	/ready says ready and lists qa-dpo
`test_qa_routes_to_dpo_adapter`	task qa is routed to qa-dpo and the output follows the Answer format
`test_summary_returns_parsed_json`	summary output is parsed into parsed_json
`test_auth_and_validation`	missing key → 401; invalid task → 422

FILE tests/mock_openai.py (a fake vLLM for tests, no GPU):

```
"""A fake OpenAI-compatible server (like vLLM) for testing pipelines and the API WITHOUT a GPU.

Run:  uvicorn tests.mock_openai:app --port 8000
It answers /v1/models and /v1/chat/completions with small, predictable replies.
"""
import re
import time
from fastapi import FastAPI, Request
```

```
app = FastAPI()
MODELS = ["base", "qa", "notes", "summary", "qa-dpo", "notes-dpo", "teacher"]


def reply_for(model, text):
    """Pick a fake answer that has the same SHAPE as a real one."""
    n_texts = len(re.findall(r"<text \d+>", text))
    if n_texts:                                          # UltraFeedback judge request
        blocks = [f"#### Output for Text {i + 1}\nRating: {5 - i if i < 4 else 1}\nRationale: mock rationale."
                  for i in range(n_texts)]
        return "\n\n".join(blocks)
    if "The correct answer is:" in text:                 # teacher explanation request (Part 5)
        gold = text.split("The correct answer is:", 1)[1].splitlines()[0].strip()
        return (f"{gold} is correct because it best fits the key findings in the stem. "
                "The other options do not explain the presentation as well. (mock teacher)")
    if '"score": <1-5>' in text:                         # 32_judge.py --mode score
        return '{"score": 4, "reason": "mock grade"}'
    if '"winner"' in text:                               # 32_judge.py --mode pairwise
        answer_a = text.split("ANSWER A:", 1)[1].split("ANSWER B:", 1)[0]
        return '{"winner": "%s", "reason": "mock"}' % ("A" if "dpo" in answer_a else "B")
    if "Return ONLY valid JSON" in text:                 # summary adapter
        return '{"visit motivation": "mock", "patient information": {"age": "40"}}'
    if "clinical case note" in text:                     # notes adapter
        return "A 40-year-old patient presented with mock symptoms."
    letter = "B" if model.startswith("qa") else "A"      # qa adapters / base / teacher
    return f"Answer: {letter}) mock option\nExplanation: mock explanation from {model}."


@app.get("/v1/models")
def models():
    return {"object": "list", "data": [{"id": m, "object": "model"} for m in MODELS]}


@app.get("/health")
def health():
    return {"status": "ok"}


@app.post("/v1/chat/completions")
async def chat(request: Request):
    body = await request.json()
    text = "\n".join(str(m.get("content", "")) for m in body.get("messages", []))
    content = reply_for(body.get("model", "base"), text)
    return {
        "id": "mock-1", "object": "chat.completion", "created": int(time.time()), "model": body.get("model"),
        "choices": [{"index": 0, "finish_reason": "stop",
                     "message": {"role": "assistant", "content": content}}],
        "usage": {"prompt_tokens": len(text) // 4, "completion_tokens": len(content) // 4,
                  "total_tokens": (len(text) + len(content)) // 4},
    }
```

Line	What it does (plain English)
`MODELS = [...]`	the names /v1/models reports, like a real vLLM with adapters
`def reply_for(model, text)`	returns a reply with the same SHAPE as the real one for each kind of request (judge ratings, explanations, JSON, notes, Answer: X)
`re.findall(r"<text \d+>", text)`	detects UltraFeedback rating prompts, which number the candidate texts
`@app.post("/v1/chat/completions")`	the OpenAI chat endpoint, returning choices[0].message.content and usage

Watch out: every request loads the shared client; never create an OpenAI client per request (slow, exhausts connections).

CHECKPOINT: Step 3 returns model "qa-dpo" and Step 6 shows 5 passed.

If not: 503 from /ready = an adapter in TASK_TO_MODEL is not loaded in vLLM; check curl localhost:8000/v1/models.

SECTION 6.6 - LangSmith tracing

WHERE: web browser (smith.langchain.com) and VS Code Terminal

GOAL: every API request visible as a trace with input, output, adapter and latency.

Step 1. Open https://smith.langchain.com and sign in.

Step 2. Click Settings (gear icon) > API Keys > Create API Key.

Step 3. Copy the key (starts with lsv2_).

Step 4. In the VS Code Terminal save it:

```
echo 'export LANGSMITH_API_KEY=lsv2_YOUR_KEY' >> ~/.bashrc && source ~/.bashrc
```

Step 5. Start the API with tracing on:

```
cd ~/-MEDSCRIPTAI/api && LANGSMITH_TRACING=true LANGSMITH_PROJECT=medscriptai VLLM_URL=http://localhost:8000/v1
API_KEY=dev-key PYTHONPATH=../scripts uvicorn app:app --port 8080
```

Step 6. In the second terminal, repeat the curl from Section 6.5 Step 3 three times.

Step 7. In LangSmith click Projects (left menu) > medscriptai.

You should see: three runs named medscriptai.generate; click one to see the input, the output, a child ChatOpenAI run with the model name (qa-dpo), token counts and latency.

Variable	What it does
LANGSMITH_TRACING=true	turns tracing on (the SDK checks LANGSMITH_TRACING, or the older LANGCHAIN_TRACING_V2)
LANGSMITH_API_KEY	authenticates your SDK
LANGSMITH_PROJECT	which project the traces go to (created automatically)
LANGSMITH_ENDPOINT	only if your account is in the EU region: https://eu.api.smith.langchain.com

Watch out: traces contain the full input text. With real patient data that is a privacy problem (PHI). For this project the data is public and synthetic-derived; in a real hospital you would self-host or redact inputs.

CHECKPOINT: the medscriptai project shows your runs.

If not: no project appears = tracing is off; check echo $LANGSMITH_API_KEY and that LANGSMITH_TRACING=true is on the same command line.

SECTION 6.7 - Package the API with Docker and run the whole stack with Compose

WHERE: VS Code Terminal, folder ~/-MEDSCRIPTAI

GOAL: a medscriptai-api:v1 image, then vLLM + API started together by one command.

Step 1. Build the image (from the repo root):

```
docker build -f api/Dockerfile -t medscriptai-api:v1 .
```

You should see: "naming to docker.io/library/medscriptai-api:v1" (about 290 MB).

Step 2. Run it against the vLLM that is already running on the VM:

```
docker run -d --name api-test --network host -e VLLM_URL=http://localhost:8000/v1 -e API_KEY=dev-key medscriptai-api:v1
```

Step 3. Test it:

```
sleep 5 && curl -s localhost:8080/ready | jq -r .status
```

You should see: ready

Step 4. Remove the test container:

```
docker rm -f api-test
```

FILE api/Dockerfile:

```
# Small CPU-only image for the API. The GPU work happens in the separate vLLM container.
# Build from the REPO ROOT:  docker build -f api/Dockerfile -t medscriptai-api:v1 .
# Official Python image via the AWS ECR Public mirror (avoids Docker Hub pull rate limits).
# Switch registry if one is rate-limited:  --build-arg BASE_IMAGE=python:3.12-slim
ARG BASE_IMAGE=public.ecr.aws/docker/library/python:3.12-slim
FROM ${BASE_IMAGE}

ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1

WORKDIR /app
```

```
COPY api/requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY scripts/common.py api/app.py ./

RUN useradd --create-home appuser
USER appuser

EXPOSE 8080
HEALTHCHECK --interval=30s --timeout=5s --retries=3 \
  CMD python -c "import urllib.request; urllib.request.urlopen('http://localhost:8080/health')" || exit 1

CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "8080", "--workers", "2"]
```

Line	What it does (plain English)
`ARG BASE_IMAGE=public.ecr.aws/docker/library/python:3.12-slim`	the official Python image through AWS's ECR Public mirror (avoids Docker Hub's anonymous pull limits; we hit a 429 while testing). Switch with --build-arg BASE_IMAGE=python:3.12-slim
`FROM ${BASE_IMAGE}`	start from that small Debian image with Python 3.12
`ENV PYTHONDONTWRITEBYTECODE=1 PYTHONUNBUFFERED=1`	no .pyc files; logs appear immediately (important for docker logs and CloudWatch)
`WORKDIR /app`	all following commands run in /app
`COPY api/requirements.txt .` then `RUN pip install --no-cache-dir -r requirements.txt`	install pinned dependencies first, as a separate layer: code changes do not re-install packages (faster rebuilds)
`COPY scripts/common.py api/app.py ./`	only the two files the API needs; that is why you build from the repo root
`RUN useradd --create-home appuser` and `USER appuser`	do not run as root inside the container (security best practice)
`EXPOSE 8080`	documents the port
`HEALTHCHECK ... urllib.request.urlopen('http://localhost:8080/health')`	Docker marks the container healthy/unhealthy (no curl needed in the slim image)
`CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "8080", "--workers", "2"]`	start 2 worker processes listening on all interfaces

Step 5. Stop the standalone servers (Compose will start its own vLLM):

```
docker rm -f medscriptai-vllm medscriptai-teacher
```

Step 6. Create your settings file:

```
cp deploy/.env.example deploy/.env && nano deploy/.env
```

Step 7. In nano set HF_TOKEN, a long random API_KEY (run openssl rand -hex 24 in another terminal to make one) and LANGSMITH_API_KEY; keep the other lines; press Ctrl+O, Enter, Ctrl+X.

Step 8. Start the stack:

```
cd deploy && docker compose --env-file .env up -d && cd ..
```

Step 9. Watch until both are healthy (2-4 minutes):

```
watch -n 10 docker compose -f deploy/docker-compose.yml ps
```

You should see: vllm "healthy" and api "running"/"healthy". Press Ctrl+C.

Step 10. Test through the stack:

```
curl -s -X POST localhost:8080/v1/generate -H "Content-Type: application/json" -H "X-API-Key: $(grep ^API_KEY
deploy/.env | cut -d= -f2)" -d '{"task":"summary","input":"A 54-year-old man with type 2 diabetes presented with chest
pain for 2 hours. ECG showed ST elevation. He underwent primary PCI."}' | jq
```

You should see: model "summary" and parsed_json with fields such as "patient information".

FILE deploy/docker-compose.yml:

```
# The full MedScriptAI stack on ONE GPU machine (Hyperstack VM or AWS EC2 g6).
# Start:  docker compose --env-file .env up -d        (run inside the deploy/ folder)
```

```
services:
  vllm:
    image: ${VLLM_IMAGE:-vllm/vllm-openai:v0.32.0}
    command: >
      --model unsloth/Llama-3.1-8B-Instruct --served-model-name base --dtype bfloat16
      --max-model-len 4096 --gpu-memory-utilization 0.90
      --enable-lora --max-loras 4 --max-lora-rank 16 --max-cpu-loras 8
      --lora-modules qa=/adapters/sft-qa notes=/adapters/sft-notes summary=/adapters/sft-summary
      qa-dpo=/adapters/dpo-qa
    environment:
      HF_TOKEN: ${HF_TOKEN:-}
      VLLM_ENABLE_CUDA_COMPATIBILITY: ${VLLM_ENABLE_CUDA_COMPATIBILITY:-0}
    volumes:
      - ${ADAPTERS_DIR:?set ADAPTERS_DIR in .env}:/adapters:ro
      - ${HF_HOME:?set HF_HOME in .env}:/root/.cache/huggingface
    ipc: host
    deploy:
      resources:
        reservations:
          devices:
            - driver: nvidia
              count: all
              capabilities: [gpu]
    healthcheck:
      test: ["CMD", "python3", "-c", "import urllib.request; urllib.request.urlopen('http://localhost:8000/health')"]
      interval: 30s
      timeout: 5s
      retries: 40
      start_period: 120s
    restart: unless-stopped
    # No "ports:" on purpose: vLLM is reachable ONLY by the api container, never from the internet.

  api:
    image: ${API_IMAGE:-medscriptai-api:v1}
    environment:
      VLLM_URL: http://vllm:8000/v1
      API_KEY: ${API_KEY:?set API_KEY in .env}
      MODEL_QA: ${MODEL_QA:-qa-dpo}
      MODEL_NOTES: ${MODEL_NOTES:-notes}
      MODEL_SUMMARY: ${MODEL_SUMMARY:-summary}
      LANGSMITH_TRACING: ${LANGSMITH_TRACING:-false}
      LANGSMITH_API_KEY: ${LANGSMITH_API_KEY:-}
      LANGSMITH_PROJECT: ${LANGSMITH_PROJECT:-medscriptai}
      LANGSMITH_ENDPOINT: ${LANGSMITH_ENDPOINT:-https://api.smith.langchain.com}
    ports:
      - "8080:8080"
    depends_on:
      vllm:
        condition: service_healthy
    restart: unless-stopped
```

Line	What it does (plain English)
`services: vllm: ... api: ...`	two containers that Compose starts, networks and restarts together
`image: ${VLLM_IMAGE:-vllm/vllm-openai:v0.32.0}`	the vLLM image; .env sets the -cu129 build for your R570 driver
`command: > --model ... --lora-modules ...`	the same server arguments as start_vllm.sh, written in YAML ("> " folds the lines into one)
`environment: HF_TOKEN, VLLM_ENABLE_CUDA_COMPATIBILITY`	from .env
`volumes: ${ADAPTERS_DIR:?...}:/adapters:ro`	`:?message` makes Compose STOP with that message if the variable is missing (no silent empty paths)
`ipc: host`	same as --ipc=host
`deploy.resources.reservations.devices: driver nvidia, count all, capabilities [gpu]`	Compose's way to give a container the GPU
`healthcheck: test: python3 -c "...urlopen('http://localhost:8000/health')"`	vLLM is healthy when its /health answers; start_period 120s gives it time to load
`restart: unless-stopped`	restart after crashes and reboots
no `ports:` under vllm	vLLM is reachable only inside the Compose network (by the name vllm), never from the internet
`api: image: ${API_IMAGE:-medscriptai-api:v1}`	on EC2 this becomes the ECR image address

`VLLM_URL: http://vllm:8000/v1`	Compose's internal DNS: the service name is the host name
`ports: "8080:8080"`	only the API is published
`depends_on: vllm: condition: service_healthy`	start the API only after vLLM is healthy

FILE deploy/compose.aws.yml (extra settings used only on AWS):

```
# Extra settings used ONLY on AWS EC2: send container logs to CloudWatch Logs.
# docker compose -f docker-compose.yml -f compose.aws.yml --env-file .env up -d
services:
  api:
    logging:
      driver: awslogs
      options:
        awslogs-region: ${AWS_REGION:-us-east-1}
        awslogs-group: /medscriptai/api
        awslogs-create-group: "true"
  vllm:
    logging:
      driver: awslogs
      options:
        awslogs-region: ${AWS_REGION:-us-east-1}
        awslogs-group: /medscriptai/vllm
        awslogs-create-group: "true"
```

Line	What it does (plain English)
`logging: driver: awslogs`	Docker sends container logs straight to CloudWatch Logs
`awslogs-group: /medscriptai/api`	the log group name
`awslogs-create-group: "true"`	create the group if it does not exist (the EC2 role allows logs:CreateLogGroup)
`docker compose -f docker-compose.yml -f compose.aws.yml`	the second file is merged on top of the first

Watch out: never commit deploy/.env (it is in .gitignore).

CHECKPOINT: docker compose ps shows both services healthy and Step 10 returns parsed JSON.

If not: docker compose -f deploy/docker-compose.yml logs vllm | tail -50 shows vLLM's error.

SECTION 6.8 - Deploy step by step: registry, storage, secrets, IAM, logs, network, GPU machine

WHERE: VS Code Terminal on the Hyperstack VM, folder ~/-MEDSCRIPTAI

GOAL: the same production deployment done twice: first free on FakeCloud, then for real on AWS.

Before you start (both columns): export API_KEY=$(openssl rand -hex 24) and make sure LANGSMITH_API_KEY and HF_TOKEN are set (echo them). For the blue column your GPU quota request (Section 2.11) must be approved.

Step	FakeCloud (free practice)	Real AWS (same step)
1 Target	`export AWS_ENDPOINT_URL=http://localhost:4566 AWS_ACCESS_KEY_ID=test AWS_SECRET_ACCESS_KEY=test AWS_DEFAULT_REGION=us-east-1`	`unset AWS_ENDPOINT_URL && export AWS_DEFAULT_REGION=us-east-1`
2 Who am I	`export ACCOUNT_ID=$(aws sts get-caller-identity --query Account --output text)`	same (your real 12-digit ID)
3 Registry address	`export REGISTRY=localhost:4566`	`export REGISTRY=$ACCOUNT_ID.dkr.ecr.us-east-1.amazonaws.com`
4 Create the image repository	`aws ecr create-repository --repository-name medscriptai-api --image-scanning-configuration scanOnPush=true`	same
5 Log Docker in	`aws ecr get-login-password | docker login --username AWS --password-stdin $REGISTRY`	same
6 Tag the image	`docker tag medscriptai-api:v1 $REGISTRY/medscriptai-api:v1`	same
7 Push	`docker push $REGISTRY/medscriptai-api:v1`	same (about 290 MB upload)

8 Verify	`aws ecr describe-images --repository-name medscriptai-api --query 'imageDetails[].imageTags'`	same; console: ECR > Repositories > medscriptai-api
9 Bucket	`export BUCKET=medscriptai-artifacts-$ACCOUNT_ID-us-east-1 && aws s3 mb s3://$BUCKET`	same (skip mb if you created it in Part 4)
10 Adapters + compose files to S3	`aws s3 sync outputs/adapters s3://$BUCKET/adapters --exclude "*/checkpoint-*" && aws s3 cp deploy/docker-compose.yml s3://$BUCKET/deploy/ && aws s3 cp deploy/compose.aws.yml s3://$BUCKET/deploy/`	same
11 Secrets	`for n in HF_TOKEN LANGSMITH_API_KEY API_KEY; do aws secretsmanager create-secret --name medscriptai/$n --secret-string "${!n}"; done`	same
12 IAM role for the machine	`aws iam create-role --role-name medscriptai-ec2-role --assume-role-policy-document file:///tmp/medscriptai-trust.json` (write the trust JSON shown below first)	same
13 Least-privilege policy	`aws iam put-role-policy --role-name medscriptai-ec2-role --policy-name medscriptai-runtime --policy-document file:///tmp/medscriptai-policy.json`	same
14 Instance profile	`aws iam create-instance-profile --instance-profile-name medscriptai-ec2-profile && aws iam add-role-to-instance-profile --instance-profile-name medscriptai-ec2-profile --role-name medscriptai-ec2-role`	same, then `sleep 10` (real IAM needs seconds to propagate)
15 Logs	`aws logs create-log-group --log-group-name /medscriptai/api && aws logs put-retention-policy --log-group-name /medscriptai/api --retention-in-days 14`	same
16 Alarm	`aws cloudwatch put-metric-alarm --alarm-name medscriptai-cpu-high --namespace AWS/EC2 --metric-name CPUUtilization --statistic Average --period 300 --evaluation-periods 2 --threshold 90 --comparison-operator GreaterThanThreshold`	same (add --alarm-actions with an SNS topic to get e-mails)
17 Firewall (security group)	`export SG_ID=$(aws ec2 create-security-group --group-name medscriptai-sg --description "MedScriptAI API" --query GroupId --output text)`	same (uses your default VPC)
18 Allow only your IP	`export MY_IP=$(curl -s https://checkip.amazonaws.com) && for p in 22 8080; do aws ec2 authorize-security-group-ingress --group-id $SG_ID --protocol tcp --port $p --cidr $MY_IP/32; done`	same. Note: MY_IP here is the Hyperstack VM's IP; to call the API from your laptop, add your laptop's IP as well
19 SSH key pair	`aws ec2 create-key-pair --key-name medscriptai-key --query KeyMaterial --output text > ~/.ssh/medscriptai-key.pem && chmod 400 ~/.ssh/medscriptai-key.pem`	same
20 Machine image (AMI)	`export AMI=$(aws ec2 describe-images --owners amazon --query 'Images[0].ImageId' --output text)` (FakeCloud ships demo AMIs)	`export AMI=$(aws ec2 describe-images --owners amazon --filters "Name=name,Values=Deep Learning Base OSS Nvidia Driver GPU AMI (Ubuntu 24.04)*" "Name=state,Values=available" --query 'sort_by(Images,&CreationDate)[-1].ImageId' --output text)` (driver 595, CUDA 13.2, Docker included [13])
21 Boot script	`sed -e "s|__REGION__|us-east-1|g" -e "s|__REGISTRY__|$REGISTRY\|g" -e "s\|__BUCKET__\|$BUCKET|g" -e "s|__TAG__|v1|g" deploy/ec2_user_data.sh > /tmp/user-data.sh`	same
22 Launch the GPU machine	`aws ec2 run-instances --image-id $AMI --instance-type g6.xlarge --key-name medscriptai-key --security-group-ids $SG_ID --iam-instance-profile Name=medscriptai-ec2-profile --block-device-mappings '[{"DeviceName":"/dev/sda1","Ebs":{"VolumeSize":150,"VolumeType":"gp3"}}]' --user-data file:///tmp/user-data.sh --tag-specifications 'ResourceType=instance,Tags=[{Key=Name,Value=medscriptai-gpu}]'`	same. BILLING STARTS NOW (about US$0.805/h + EBS)
23 Get its IP	`aws ec2 describe-instances --filters Name=tag:Name,Values=medscriptai-gpu --query 'Reservations[].Instances[].[InstanceId,State.Name,PublicIpAddress]' --output text`	same

You should see	an i-... id, "running" or "pending", and a fake IP such as 52.0.0.10. FakeCloud starts a small container to represent the instance; there is no GPU and the boot script does not serve the model	a real public IP. The boot script needs 10-15 minutes (downloads about 30 GB)
24 Watch the boot	not applicable	`ssh -i ~/.ssh/medscriptai-key.pem ubuntu@EC2_IP` then `sudo tail -f /var/log/cloud-init-output.log` (wait for docker compose to report started)
25 Test the API	not applicable (no model behind FakeCloud)	`curl -s http://EC2_IP:8080/ready` then the /v1/generate curl from Section 6.5 with your API_KEY
26 Read the logs	`aws logs describe-log-groups --query 'logGroups[].logGroupName'`	console: CloudWatch > Log groups > /medscriptai/api, or `aws logs tail /medscriptai/api --follow`
27 TEAR DOWN	`TARGET=fakecloud bash deploy/teardown.sh`	`TARGET=aws bash deploy/teardown.sh` (then check EC2 > Instances shows terminated and Billing shows no running resources)

Trust policy for Step 12 (write it with this command first):

```
cat > /tmp/medscriptai-trust.json <<'EOF'
{"Version":"2012-10-17","Statement":[{"Effect":"Allow","Principal":
{"Service":"ec2.amazonaws.com"},"Action":"sts:AssumeRole"}]}
EOF
```

Permissions policy for Step 13 (least privilege: only our repo, our bucket, our secrets, logs):

```
cat > /tmp/medscriptai-policy.json <<EOF
{"Version":"2012-10-17","Statement":[
 {"Effect":"Allow","Action":["ecr:GetAuthorizationToken"],"Resource":"*"},
 {"Effect":"Allow","Action":["ecr:BatchGetImage","ecr:GetDownloadUrlForLayer"],"Resource":"arn:aws:ecr:us-east-
1:$ACCOUNT_ID:repository/medscriptai-api"},
 {"Effect":"Allow","Action":["s3:GetObject","s3:ListBucket"],"Resource":
["arn:aws:s3:::$BUCKET","arn:aws:s3:::$BUCKET/*"]},
 {"Effect":"Allow","Action":["secretsmanager:GetSecretValue"],"Resource":"arn:aws:secretsmanager:us-east-
1:$ACCOUNT_ID:secret:medscriptai/*"},
 {"Effect":"Allow","Action":["logs:CreateLogGroup","logs:CreateLogStream","logs:PutLogEvents"],"Resource":"*"}]}
EOF
```

Command pieces that are new here:

Piece	What it does
`aws ecr get-login-password | docker login --password-stdin`	ECR gives a 12-hour token; piping it avoids typing it or saving it in shell history
`docker tag LOCAL REGISTRY/REPO:TAG`	a second name for the same image that says where it will be pushed
`"${!n}"`	bash indirect expansion: the value of the variable whose NAME is in n (HF_TOKEN, ...)
trust policy (Principal ec2.amazonaws.com)	WHO may use the role: EC2 machines
permissions policy	WHAT the role may do; every Resource is limited to MedScriptAI's own repo, bucket and secrets
instance profile	the container that attaches a role to an EC2 machine; programs on the machine get temporary credentials automatically, no keys on disk
`--cidr $MY_IP/32`	/32 = exactly one IP address
`--block-device-mappings ... VolumeSize 150, gp3`	a 150 GB SSD root disk: the vLLM image (about 11 GB compressed) + the model (16 GB) need room
`--user-data file://...`	a script cloud-init runs once at first boot, as root
g6.xlarge	1x NVIDIA L4 (24 GB), 4 vCPU, 16 GB RAM: the cheapest current AWS GPU that fits an 8B bf16 model with LoRA

Placeholders:

Placeholder	Replace with	Example
EC2_IP	the PublicIpAddress from Step 23	3.91.20.14

FILE deploy/ec2_user_data.sh (what the AWS machine does on first boot):

```
#!/bin/bash
# Runs ONCE, as root, the first time the EC2 machine boots (cloud-init "user data").
# deploy.sh replaces the __PLACEHOLDERS__ before sending it to AWS.
set -euxo pipefail                                    # -x prints every command into the boot log
```

```
REGION=__REGION__
REGISTRY=__REGISTRY__
BUCKET=__BUCKET__
TAG=__TAG__
APP_DIR=/opt/medscriptai
mkdir -p $APP_DIR/adapters $APP_DIR/hf-cache

# 1) Adapters from S3 (the machine's IAM role gives read access; no keys on the box).
aws s3 sync "s3://$BUCKET/adapters" $APP_DIR/adapters --region "$REGION"

# 2) Secrets from Secrets Manager into a root-only .env file.
secret() { aws secretsmanager get-secret-value --region "$REGION" --secret-id "medscriptai/$1" \
           --query SecretString --output text 2>/dev/null || true; }
cat > $APP_DIR/.env <<EOF
ADAPTERS_DIR=$APP_DIR/adapters
HF_HOME=$APP_DIR/hf-cache
HF_TOKEN=$(secret HF_TOKEN)
API_KEY=$(secret API_KEY)
LANGSMITH_API_KEY=$(secret LANGSMITH_API_KEY)
LANGSMITH_TRACING=true
VLLM_IMAGE=vllm/vllm-openai:v0.32.0
VLLM_ENABLE_CUDA_COMPATIBILITY=0
API_IMAGE=$REGISTRY/medscriptai-api:$TAG
AWS_REGION=$REGION
EOF
chmod 600 $APP_DIR/.env

# 3) Log Docker in to ECR and pull our API image.
aws ecr get-login-password --region "$REGION" | docker login --username AWS --password-stdin "$REGISTRY"

# 4) The compose files that deploy.sh uploaded to S3.
aws s3 cp "s3://$BUCKET/deploy/docker-compose.yml" $APP_DIR/ --region "$REGION"
aws s3 cp "s3://$BUCKET/deploy/compose.aws.yml" $APP_DIR/ --region "$REGION"

# 5) Start vLLM + API. vLLM downloads the 16 GB base model on first start.
cd $APP_DIR
docker compose -f docker-compose.yml -f compose.aws.yml --env-file .env up -d
```

Line	What it does (plain English)
`set -euxo pipefail`	stop on errors, and print every command into /var/log/cloud-init-output.log (-x) so you can debug
`REGION=__REGION__` etc.	placeholders that Step 21 (or deploy.sh) replaced with real values
`aws s3 sync "s3://$BUCKET/adapters" $APP_DIR/adapters`	downloads the adapters using the machine's role (no keys)
`secret() { aws secretsmanager get-secret-value ... }`	a small function that reads one secret by name
`cat > $APP_DIR/.env <<EOF ... EOF` and `chmod 600`	writes the settings file readable only by root
`VLLM_IMAGE=vllm/vllm-openai:v0.32.0` and `VLLM_ENABLE_CUDA_COMPATIBILITY=0`	the Deep Learning AMI has driver 595 / CUDA 13.2, so the default CUDA 13 image runs natively, no compatibility layer
`API_IMAGE=$REGISTRY/medscriptai-api:$TAG`	Compose pulls YOUR image from ECR
`aws ecr get-login-password ... | docker login ...`	lets Docker pull from your private registry
`aws s3 cp "s3://$BUCKET/deploy/docker-compose.yml" ...`	the same compose files you tested on Hyperstack
`docker compose -f docker-compose.yml -f compose.aws.yml --env-file .env up -d`	starts vLLM + API with CloudWatch logging

The same deployment in one command (after you have done it by hand once, this is how you work):

```
TARGET=fakecloud bash deploy/deploy.sh
```

FILE deploy/deploy.sh:

```
#!/usr/bin/env bash
# One script, two targets:
#   TARGET=fakecloud bash deploy/deploy.sh   -> free local AWS emulator (localhost:4566)
#   TARGET=aws       bash deploy/deploy.sh   -> your real AWS account (costs money!)
# Run from the repo folder after: the API image is built and outputs/adapters/ exists.
set -euo pipefail
```

```
TARGET="${TARGET:-fakecloud}"
export AWS_REGION="${AWS_REGION:-us-east-1}"
export AWS_DEFAULT_REGION="$AWS_REGION"
APP=medscriptai
TAG="${TAG:-v1}"
MY_IP="${MY_IP:-$(curl -s https://checkip.amazonaws.com)}"   # only YOUR IP may reach SSH and the API

if [ "$TARGET" = "fakecloud" ]; then
  export AWS_ENDPOINT_URL=http://localhost:4566             # every aws command now talks to FakeCloud
  export AWS_ACCESS_KEY_ID=test AWS_SECRET_ACCESS_KEY=test  # FakeCloud accepts any keys
fi

ACCOUNT_ID=$(aws sts get-caller-identity --query Account --output text)
if [ "$TARGET" = "fakecloud" ]; then
  REGISTRY=localhost:4566
else
  REGISTRY="$ACCOUNT_ID.dkr.ecr.$AWS_REGION.amazonaws.com"
fi
BUCKET="$APP-artifacts-$ACCOUNT_ID-$AWS_REGION"               # bucket names must be globally unique
echo "== target=$TARGET account=$ACCOUNT_ID region=$AWS_REGION registry=$REGISTRY"

echo "== 1/7 ECR repository + push the API image"
aws ecr describe-repositories --repository-names "$APP-api" >/dev/null 2>&1 \
  || aws ecr create-repository --repository-name "$APP-api" --image-scanning-configuration scanOnPush=true >/dev/null
aws ecr get-login-password | docker login --username AWS --password-stdin "$REGISTRY"
docker tag "$APP-api:$TAG" "$REGISTRY/$APP-api:$TAG"
docker push "$REGISTRY/$APP-api:$TAG"

echo "== 2/7 S3 bucket + upload adapters and reports"
aws s3 mb "s3://$BUCKET" 2>/dev/null || true
aws s3 sync outputs/adapters "s3://$BUCKET/adapters" --exclude "*/checkpoint-*" --only-show-errors
[ -d outputs/reports ] && aws s3 sync outputs/reports "s3://$BUCKET/reports" --only-show-errors
aws s3 cp deploy/docker-compose.yml "s3://$BUCKET/deploy/docker-compose.yml" --only-show-errors
aws s3 cp deploy/compose.aws.yml "s3://$BUCKET/deploy/compose.aws.yml" --only-show-errors

echo "== 3/7 Secrets Manager (HF token, LangSmith key, API key)"
for name in HF_TOKEN LANGSMITH_API_KEY API_KEY; do
  value="${!name:-}"
  [ -z "$value" ] && { echo "   skip $name (not set in your shell)"; continue; }
  aws secretsmanager create-secret --name "$APP/$name" --secret-string "$value" >/dev/null 2>&1 \
    || aws secretsmanager put-secret-value --secret-id "$APP/$name" --secret-string "$value" >/dev/null
  echo "   stored $APP/$name"
done

echo "== 4/7 IAM role the EC2 machine will use (least privilege)"
cat > /tmp/$APP-trust.json <<'EOF'
{"Version":"2012-10-17","Statement":[{"Effect":"Allow","Principal":
{"Service":"ec2.amazonaws.com"},"Action":"sts:AssumeRole"}]}
EOF
cat > /tmp/$APP-policy.json <<EOF
{"Version":"2012-10-17","Statement":[
 {"Effect":"Allow","Action":["ecr:GetAuthorizationToken"],"Resource":"*"},
 {"Effect":"Allow","Action":
["ecr:BatchGetImage","ecr:GetDownloadUrlForLayer"],"Resource":"arn:aws:ecr:$AWS_REGION:$ACCOUNT_ID:repository/$APP-
api"},
 {"Effect":"Allow","Action":["s3:GetObject","s3:ListBucket"],"Resource":
["arn:aws:s3:::$BUCKET","arn:aws:s3:::$BUCKET/*"]},
 {"Effect":"Allow","Action":
["secretsmanager:GetSecretValue"],"Resource":"arn:aws:secretsmanager:$AWS_REGION:$ACCOUNT_ID:secret:$APP/*"},
 {"Effect":"Allow","Action":["logs:CreateLogGroup","logs:CreateLogStream","logs:PutLogEvents"],"Resource":"*"}]}
EOF
aws iam get-role --role-name "$APP-ec2-role" >/dev/null 2>&1 \
  || aws iam create-role --role-name "$APP-ec2-role" --assume-role-policy-document "file:///tmp/$APP-trust.json"
>/dev/null
aws iam put-role-policy --role-name "$APP-ec2-role" --policy-name "$APP-runtime" --policy-document "file:///tmp/$APP-
policy.json"
aws iam get-instance-profile --instance-profile-name "$APP-ec2-profile" >/dev/null 2>&1 || {
  aws iam create-instance-profile --instance-profile-name "$APP-ec2-profile" >/dev/null
  aws iam add-role-to-instance-profile --instance-profile-name "$APP-ec2-profile" --role-name "$APP-ec2-role"
  sleep 10                                                  # real IAM needs a few seconds to propagate
}

echo "== 5/7 CloudWatch: log group + CPU alarm"
aws logs create-log-group --log-group-name "/$APP/api" 2>/dev/null || true
aws logs put-retention-policy --log-group-name "/$APP/api" --retention-in-days 14
aws cloudwatch put-metric-alarm --alarm-name "$APP-cpu-high" --namespace AWS/EC2 --metric-name CPUUtilization \
```

```
  --statistic Average --period 300 --evaluation-periods 2 --threshold 90 --comparison-operator GreaterThanThreshold

echo "== 6/7 Security group: port 8080 (API) and 22 (SSH) only from $MY_IP"
SG_ID=$(aws ec2 describe-security-groups --filters "Name=group-name,Values=$APP-sg" \
        --query 'SecurityGroups[0].GroupId' --output text 2>/dev/null || echo None)
if [ "$SG_ID" = "None" ] || [ -z "$SG_ID" ]; then
  SG_ID=$(aws ec2 create-security-group --group-name "$APP-sg" --description "MedScriptAI API" \
          --query GroupId --output text)
  for port in 22 8080; do
    aws ec2 authorize-security-group-ingress --group-id "$SG_ID" --protocol tcp --port $port --cidr "$MY_IP/32"
>/dev/null
  done
fi

echo "== 7/7 Launch the GPU machine (g6.xlarge = 1x NVIDIA L4 24 GB)"
if [ "$TARGET" = "fakecloud" ]; then
  AMI=$(aws ec2 describe-images --owners amazon --query 'Images[0].ImageId' --output text)
else
  AMI=$(aws ec2 describe-images --owners amazon \
        --filters "Name=name,Values=Deep Learning Base OSS Nvidia Driver GPU AMI (Ubuntu 24.04)*"
"Name=state,Values=available" \
        --query 'sort_by(Images,&CreationDate)[-1].ImageId' --output text)
fi
sed -e "s|__REGION__|$AWS_REGION|g" -e "s|__REGISTRY__|$REGISTRY|g" -e "s|__BUCKET__|$BUCKET|g" -e "s|__TAG__|$TAG|g" \
  deploy/ec2_user_data.sh > /tmp/$APP-user-data.sh
aws ec2 describe-key-pairs --key-names "$APP-key" >/dev/null 2>&1 || {
  aws ec2 create-key-pair --key-name "$APP-key" --query KeyMaterial --output text > "$HOME/.ssh/$APP-key.pem"
  chmod 400 "$HOME/.ssh/$APP-key.pem"
}
INSTANCE_ID=$(aws ec2 run-instances --image-id "$AMI" --instance-type "${INSTANCE_TYPE:-g6.xlarge}" \
  --key-name "$APP-key" --security-group-ids "$SG_ID" \
  --iam-instance-profile Name="$APP-ec2-profile" \
  --block-device-mappings '[{"DeviceName":"/dev/sda1","Ebs":{"VolumeSize":150,"VolumeType":"gp3"}}]' \
  --user-data "file:///tmp/$APP-user-data.sh" \
  --tag-specifications "ResourceType=instance,Tags=[{Key=Name,Value=$APP-gpu}]" \
  --query 'Instances[0].InstanceId' --output text)
echo "   instance $INSTANCE_ID starting (AMI $AMI)"
timeout 300 aws ec2 wait instance-running --instance-ids "$INSTANCE_ID" 2>/dev/null || sleep 5
IP=$(aws ec2 describe-instances --instance-ids "$INSTANCE_ID" --query 'Reservations[0].Instances[0].PublicIpAddress' --
output text)
echo "DONE  ->  API will be at http://$IP:8080 in ~10-15 min (first boot downloads ~30 GB)."
echo "      SSH: ssh -i ~/.ssh/$APP-key.pem ubuntu@$IP   |   boot log: sudo tail -f /var/log/cloud-init-output.log"
```

Line	What it does (plain English)
`TARGET="${TARGET:-fakecloud}"`	the default target is the free emulator; you must type TARGET=aws on purpose
`MY_IP=...checkip.amazonaws.com`	your public IP, used to lock the firewall
`if [ "$TARGET" = "fakecloud" ]; then export AWS_ENDPOINT_URL=...`	the ONLY difference between the two targets
`aws ecr describe-repositories ... || aws ecr create-repository ...`	"create if missing": the script is idempotent (we ran it twice in testing; the second run succeeds)
`aws s3 mb ... 2>/dev/null || true`	ignore "bucket already exists"
`value="${!name:-}"` and `continue`	skip secrets you have not exported
`create-secret ... || put-secret-value ...`	create the secret, or update it if it exists
`aws iam get-role ... || aws iam create-role ...`	create the role only once
`SG_ID=$(aws ec2 describe-security-groups --filters ...)`	re-use the security group on a second run
`sort_by(Images,&CreationDate)[-1].ImageId`	JMESPath: sort the matching AMIs by date and take the newest
`sed -e "s|__REGION__|$AWS_REGION|g" ...`	fills the placeholders in the boot script
`aws ec2 describe-key-pairs ... || { aws ec2 create-key-pair ... }`	create and save the SSH key once
`timeout 300 aws ec2 wait instance-running`	wait (at most 5 minutes) until the machine runs

FILE deploy/teardown.sh:

```
#!/usr/bin/env bash
# Delete everything deploy.sh created, so the AWS bill stops.  TARGET=aws bash deploy/teardown.sh
set -uo pipefail                                          # no -e: keep going if one item is already gone
```

```
TARGET="${TARGET:-fakecloud}"
export AWS_REGION="${AWS_REGION:-us-east-1}" AWS_DEFAULT_REGION="${AWS_REGION:-us-east-1}"
APP=medscriptai
if [ "$TARGET" = "fakecloud" ]; then
  export AWS_ENDPOINT_URL=http://localhost:4566 AWS_ACCESS_KEY_ID=test AWS_SECRET_ACCESS_KEY=test
fi
ACCOUNT_ID=$(aws sts get-caller-identity --query Account --output text)
BUCKET="$APP-artifacts-$ACCOUNT_ID-$AWS_REGION"

IDS=$(aws ec2 describe-instances --filters "Name=tag:Name,Values=$APP-gpu" "Name=instance-state-
name,Values=pending,running,stopped" \
      --query 'Reservations[].Instances[].InstanceId' --output text)
[ -n "$IDS" ] && aws ec2 terminate-instances --instance-ids $IDS >/dev/null && echo "terminated $IDS"
[ -n "$IDS" ] && timeout 600 aws ec2 wait instance-terminated --instance-ids $IDS 2>/dev/null
for try in 1 2 3 4 5 6; do                                  # the SG can only go once the instance is gone
  aws ec2 delete-security-group --group-name "$APP-sg" >/dev/null 2>&1 && { echo "deleted security group"; break; }
  sleep 10
done
aws ec2 delete-key-pair --key-name "$APP-key" >/dev/null 2>&1 && echo "deleted key pair"
aws ecr delete-repository --repository-name "$APP-api" --force >/dev/null 2>&1 && echo "deleted ECR repo"
aws s3 rb "s3://$BUCKET" --force >/dev/null 2>&1 && echo "deleted bucket"
for name in HF_TOKEN LANGSMITH_API_KEY API_KEY; do
  aws secretsmanager delete-secret --secret-id "$APP/$name" --force-delete-without-recovery >/dev/null 2>&1 && echo
"deleted secret $name"
done
aws cloudwatch delete-alarms --alarm-names "$APP-cpu-high" 2>/dev/null && echo "deleted alarm"
aws logs delete-log-group --log-group-name "/$APP/api" 2>/dev/null && echo "deleted log group"
aws iam remove-role-from-instance-profile --instance-profile-name "$APP-ec2-profile" --role-name "$APP-ec2-role"
2>/dev/null
aws iam delete-instance-profile --instance-profile-name "$APP-ec2-profile" 2>/dev/null && echo "deleted instance
profile"
aws iam delete-role-policy --role-name "$APP-ec2-role" --policy-name "$APP-runtime" 2>/dev/null
aws iam delete-role --role-name "$APP-ec2-role" 2>/dev/null && echo "deleted role"
echo "TEARDOWN DONE ($TARGET)"
```

Line	What it does (plain English)
`set -uo pipefail` (no -e)	keep going if one resource is already gone
`describe-instances --filters "Name=tag:Name,Values=$APP-gpu"`	find our machines by their Name tag
`terminate-instances` then `wait instance-terminated`	delete the machine (stops compute billing) and wait
`for try in 1 2 3 4 5 6; do delete-security-group ...`	a security group can only be deleted after the instance is fully gone; we retry (found while testing)
`delete-repository --force`	delete the repo including its images
`aws s3 rb ... --force`	delete the bucket and everything in it
`delete-secret --force-delete-without-recovery`	delete secrets immediately (no 7-30 day recovery window that is still billed)
`remove-role-from-instance-profile`, `delete-instance-profile`, `delete-role-policy`, `delete-role`	IAM pieces must be removed in this order

> **Watch out:** the most expensive mistake in this whole manual is forgetting Step 27 on real AWS. Set a phone reminder when you launch the instance.

CHECKPOINT (FakeCloud): deploy.sh ends with DONE and teardown.sh ends with TEARDOWN DONE (fakecloud).

CHECKPOINT (AWS): curl http://EC2_IP:8080/v1/generate returns an answer from your adapter; after teardown, EC2 shows the instance terminated.

> **If not:** on AWS, "VcpuLimitExceeded" = GPU quota not approved yet (Section 2.11); "InvalidAMIID" = run Step 20 in us-east-1; API not answering after 20 minutes = read /var/log/cloud-init-output.log on the machine.

SECTION 6.9 - (Optional) SageMaker endpoint: the managed alternative

WHERE: VS Code Terminal, folder ~/-MEDSCRIPTAI

GOAL: understand the SageMaker deployment objects; practise their API calls for free.

Step	FakeCloud (free practice)	Real AWS (same step)

1 Model	`aws sagemaker create-model --model-name medscriptai-model --primary-container Image=$REGISTRY/medscriptai-api:v1 --execution-role-arn arn:aws:iam::$ACCOUNT_ID:role/medscriptai-ec2-role`	needs a SageMaker-compatible container (answers GET /ping and POST /invocations on port 8080) and a role trusted by sagemaker.amazonaws.com. Our API image is NOT that; AWS's Large Model Inference (LMI) containers with a vLLM backend are the usual choice (verify current names in the SageMaker docs)
2 Endpoint config	`aws sagemaker create-endpoint-config --endpoint-config-name medscriptai-cfg --production-variants VariantName=AllTraffic,ModelName=medscriptai-model,InstanceType=ml.g6.xlarge,InitialInstanceCount=1`	same shape
3 Endpoint	`aws sagemaker create-endpoint --endpoint-name medscriptai-endpoint --endpoint-config-name medscriptai-cfg`	same; billing per hour while InService
4 Status	`aws sagemaker describe-endpoint --endpoint-name medscriptai-endpoint --query EndpointStatus`	same
You should see	"InService" (FakeCloud emulates the control plane only; invoke-endpoint returns NotFound, we tested it)	"Creating" for 5-15 minutes, then "InService"
5 Delete	`aws sagemaker delete-endpoint --endpoint-name medscriptai-endpoint`	same, plus delete-endpoint-config and delete-model

Watch out: honest comparison for interviews: EC2 + Docker Compose (what you built) gives full control and is cheapest for one always-on model; SageMaker adds managed autoscaling, deployment guardrails and monitoring, at a higher per-hour price and with more container rules.

CHECKPOINT: on FakeCloud, describe-endpoint returned InService and delete-endpoint succeeded; on AWS, every SageMaker endpoint you created is deleted (console: SageMaker > Inference > Endpoints is empty).

6.10 Common mistake at this level

The mistake: exposing the model server directly (port 8000 open with runtime LoRA loading on), keeping secrets in .env files baked into images or Git, and leaving GPU machines running.

How to spot it: from your laptop, curl http://EC2_IP:8000/v1/models must FAIL (only 8080 is published, and only to your IP); docker history medscriptai-api:v1 must show no secrets; aws ec2 describe-instances after teardown must show nothing running.

PART 7 - ELITE: how top labs and practitioners do it

7.0 Concept

What: the habits that separate a demo from work a frontier lab, a health-AI company or a strong reviewer would trust: proving there is no test leakage, proving a gain is not noise, proving the judge agrees with humans, scoring against explicit criteria, rolling out model versions without downtime, and documenting everything in a model card.

Why: in interviews and in real teams, "accuracy went from 61% to 64%" is worthless until you can answer: was the test set contaminated, is 3 points significant on 500 questions, who graded it and how reliable is the grader, and can you roll it back?

When: before you claim any result anywhere (CV, README, interview, paper).

Harder than Part 6 because: the work is statistical and methodological, not just engineering, and it can (and should) prove your own earlier claims wrong.

7.1 Who defines the top level today

Who / what	Contribution you should know	Where
Tim Dettmers et al. (University of Washington), 2023	QLoRA: 4-bit NF4 + LoRA, double quantisation, paged optimizers; fine-tuning 65B models on one GPU	https://arxiv.org/abs/2305.14314
Edward Hu et al. (Microsoft), 2021	LoRA: low-rank adapters	https://arxiv.org/abs/2106.09685
Rafael Rafailov, Archit Sharma, Eric Mitchell et al. (Stanford), 2023	DPO: preference optimisation without a reward model or RL	https://arxiv.org/abs/2305.18290
Lewis Tunstall et al. (Hugging Face H4), 2023	Zephyr: distilled SFT + DPO on AI-feedback (UltraFeedback) for a 7B model; the recipe this project follows	https://arxiv.org/abs/2310.16944
Nathan Lambert et al. (AI2), 2024	Tulu 3: open post-training recipe (SFT, DPO with on-policy data, RL with verifiable rewards), with decontamination	https://arxiv.org/abs/2411.15124
Karan Singhal et al. (Google), Nature Medicine 2025	Med-PaLM 2: expert-level MedQA (86.5%) and physician pairwise evaluation on many axes	https://doi.org/10.1038/s41591-024-03423-7
Rahul K. Arora et al. (OpenAI), 2025	HealthBench: 5,000 health conversations graded with 48,562 physician-written rubric criteria	https://arxiv.org/abs/2505.08775
Stanford CRFM / Stanford Medicine, 2025	MedHELM: holistic evaluation across 121 real clinical tasks (notes, decision support, communication, admin)	https://arxiv.org/abs/2505.23802
Lianmin Zheng et al. (LMSYS), 2023	LLM-as-a-judge: position, verbosity and self-preference bias, and the swap trick you used	https://arxiv.org/abs/2306.05685
Seungone Kim et al., 2024	Prometheus 2: open evaluator models for rubric and pairwise grading	https://arxiv.org/abs/2405.01535
Woosuk Kwon et al. (vLLM, UC Berkeley), 2023	PagedAttention: the KV-cache memory manager behind vLLM	https://arxiv.org/abs/2309.06180
Ying Sheng et al., 2023 (S-LoRA) and Lequn Chen et al., 2023 (Punica)	serving thousands of LoRA adapters on one base model	https://arxiv.org/abs/2311.03285
Daniel and Michael Han (Unsloth)	hand-written Triton kernels that make LoRA/QLoRA about 2x faster with less VRAM	https://github.com/unslothai/unsloth
Brown et al. (OpenAI), 2020, GPT-3 paper appendix	13-gram overlap contamination analysis, the method in Section 7.2	https://arxiv.org/abs/2005.14165

SECTION 7.2 - Prove there is no test leakage (near-duplicate contamination check)

WHERE: VS Code Terminal, folder ~/-MEDSCRIPTAI/scripts, (.venv-eval) active

GOAL: the number of test questions that are near-copies of training questions (should be 0).

Step 1. Run:

```
python 04_contamination_check.py
```

You should see: qa test rows= 500 near-duplicates of training rows= 0 (0.0%) and the same for notes and summary.

FILE scripts/04_contamination_check.py:

```
"""Elite check: is any TEST question a near-copy of a TRAINING question? (13-gram overlap, GPT-3 paper style)

Exact-match de-duplication (step 2) misses near-copies: the same question with one word changed.
Medical exams share stock phrases ("is brought to the emergency department because of a 2-day history of",
lab panels), so ONE shared 13-gram proves nothing. We flag a test question only when at least
50% of its 13-grams come from a SINGLE training question.
"""
import re
from collections import Counter, defaultdict
from common import SFT, read_jsonl

N = 13                       # n-gram length in words
THRESHOLD = 0.5              # share of the test question's n-grams found in one training question


def grams(text):
    text = text.split("\n\nOptions:")[0]                # compare the question stem, not the option labels
    words = re.findall(r"[a-z0-9]+", text.lower())
    return {" ".join(words[i:i + N]) for i in range(len(words) - N + 1)}


for task in ["qa", "notes", "summary"]:
    index = defaultdict(set)                            # n-gram -> ids of training rows containing it
    path = SFT / f"{task}_train.jsonl"
    if path.exists():
        for row_id, row in enumerate(read_jsonl(path)):
            for g in grams(row["messages"][1]["content"]):
                index[g].add(row_id)
    test = read_jsonl(SFT / f"{task}_test.jsonl")
    flagged = []
    for row in test:
        test_grams = grams(row["messages"][1]["content"])
        per_train_row = Counter(i for g in test_grams for i in index.get(g, ()))
        best = per_train_row.most_common(1)[0][1] if per_train_row else 0
        if test_grams and best / len(test_grams) >= THRESHOLD:
            flagged.append(row["messages"][1]["content"][:110].replace("\n", " "))
    print(f"{task:8} test rows={len(test):4}  near-duplicates of training rows={len(flagged):3} "
          f"({len(flagged) / max(len(test), 1):.1%})")
    for text in flagged[:3]:
        print("   e.g.", text)
```

Line	What it does (plain English)
`N = 13`	n-gram length, as in the GPT-3 contamination analysis
`THRESHOLD = 0.5`	flag a test question if at least half of its 13-grams come from ONE training question
`def grams(text)`	lower-case words, cut the "Options:" block, return the set of all 13-word windows
`index = defaultdict(set)`	inverted index: 13-gram → ids of training rows that contain it
`per_train_row = Counter(i for g in test_grams for i in index.get(g, ()))`	for one test question, how many of its 13-grams each training row shares
`best / len(test_grams) >= THRESHOLD`	the share coming from the single most similar training row

What we learned while building this (a real elite lesson): the first version flagged 13% of QA test questions as contaminated after exact de-duplication. Inspection showed false positives: exam boilerplate ("which of the following is the most appropriate next step in management", standard lab panels, "is brought to the emergency department because of a 2-day history of"). Counting a single shared 13-gram over-reports leakage in templated domains; requiring that half of a question comes from one training row fixed it, and a planted near-copy (one word changed) was still caught.

Watch out: this checks YOUR training data. The base model's pre-training data may already contain MedQA (it is public). That is why Med-PaLM 2, HealthBench and MedHELM also evaluate on new, private or expert-written items. Say this in interviews.

CHECKPOINT: 0 near-duplicates (or a short list you then remove from training and re-train).

SECTION 7.3 - Prove a gain is real (paired bootstrap confidence interval)

WHERE: VS Code Terminal, folder ~/-MEDSCRIPTAI/scripts, (.venv-eval) active

GOAL: a 95% confidence interval for "DPO accuracy minus SFT accuracy".

Step 1. Run:

```
python 33_significance.py --a qa --b qa-dpo
```

Step 2. Also compare base vs SFT:

```
python 33_significance.py --a base --b qa
```

You should see: accuracy qa=... qa-dpo=... diff=+0.0xx, a 95% CI such as [-0.010, +0.052] and either SIGNIFICANT or NOT significant.

FILE scripts/33_significance.py:

```
"""Elite check: is model B REALLY better than model A, or is it noise? Paired bootstrap on QA accuracy.

python scripts/33_significance.py --a qa --b qa-dpo
"""
import argparse
import random
from common import OUT, read_jsonl

p = argparse.ArgumentParser()
p.add_argument("--a", required=True)
p.add_argument("--b", required=True)
p.add_argument("--rounds", type=int, default=10000)
args = p.parse_args()


def correct(row):
    pred = row["prediction"] or ""
    letter = pred.split("Answer:", 1)[1].strip()[:1].upper() if "Answer:" in pred else None
    return int(letter == row["answer_letter"])


a = [correct(r) for r in read_jsonl(OUT / "preds" / "qa" / f"{args.a}.jsonl")]
b = [correct(r) for r in read_jsonl(OUT / "preds" / "qa" / f"{args.b}.jsonl")]
n = min(len(a), len(b))
diffs = []
rng = random.Random(0)
for _ in range(args.rounds):
    idx = [rng.randrange(n) for _ in range(n)]          # resample the SAME questions for both models
    diffs.append(sum(b[i] - a[i] for i in idx) / n)
diffs.sort()
low, high = diffs[int(0.025 * args.rounds)], diffs[int(0.975 * args.rounds)]
observed = (sum(b[:n]) - sum(a[:n])) / n
print(f"accuracy {args.a}={sum(a[:n]) / n:.3f}  {args.b}={sum(b[:n]) / n:.3f}  diff={observed:+.3f}")
print(f"95% CI of the difference: [{low:+.3f}, {high:+.3f}]")
print("SIGNIFICANT" if low > 0 or high < 0 else "NOT significant: the CI contains 0, do not claim an improvement")
```

Line	What it does (plain English)
`def correct(row)`	1 if the predicted letter equals the gold letter, else 0
`a = [...], b = [...]`	per-question correctness for the two models, on the SAME 500 questions
`idx = [rng.randrange(n) for _ in range(n)]`	a bootstrap sample: draw 500 question indices with replacement
`diffs.append(sum(b[i] - a[i] for i in idx) / n)`	the accuracy difference on that resampled test set; PAIRED because both models are scored on the same resampled questions
`rounds = 10000`	repeat 10,000 times to get the distribution of the difference
`low, high = diffs[2.5%], diffs[97.5%]`	the middle 95% of the differences = the confidence interval
`SIGNIFICANT if low > 0 or high < 0`	if the interval does not contain 0, the difference is unlikely to be noise

Rule of thumb you should be able to say in an interview: on 500 multiple-choice questions, the 95% interval of a single accuracy is about +/- 4 points, so a 2-point gain is usually NOT significant; paired tests are tighter because both models share the hard questions.

CHECKPOINT: you can state each comparison as "diff X, 95% CI [L, H]".

SECTION 7.4 - Prove the judge is trustworthy (agreement with your own labels)

WHERE: VS Code Terminal and WinSCP (or VS Code editor), folder ~/-MEDSCRIPTAI/scripts, (.venv-eval) active

GOAL: Cohen's kappa between the LLM judge and you on 30 pairwise comparisons.

Step 1. Create the label sheet:

```
python 35_judge_agreement.py --judge-file ../outputs/reports/judge_qa_pairwise_qa-dpo_vs_qa.json
```

You should see: fill the 'human' column in .../outputs/reports/human_labels.csv, then run this script again

Step 2. Show the two answers for item i (repeat for i = 0..29; change I_NUMBER):

```
python -c "import json;a=[json.loads(l) for l in open('../outputs/preds/qa/qa-dpo.jsonl')];b=[json.loads(l) for l in
open('../outputs/preds/qa/qa.jsonl')];i=I_NUMBER;print(a[i]['prompt'],'\n\nREFERENCE:',a[i]['reference'],'\n\nqa-
dpo:',a[i]['prediction'],'\n\nqa:',b[i]['prediction'])"
```

Step 3. Open outputs/reports/human_labels.csv in VS Code (Explorer panel) and type qa-dpo, qa or tie in the human column for each row you judged. Save with Ctrl+S.

Step 4. Run again:

```
python 35_judge_agreement.py --judge-file ../outputs/reports/judge_qa_pairwise_qa-dpo_vs_qa.json
```

You should see: items=30 agreement=0.xx Cohen's kappa=0.xx and the interpretation line.

FILE scripts/35_judge_agreement.py:

```
"""Elite check: can you trust the LLM judge? Compare its pairwise verdicts with YOUR labels (Cohen's kappa).

1) Open outputs/reports/human_labels.csv (created on first run) and fill the 'human' column
   with the model name you prefer, or 'tie', for each row (read the two answers in the preds files).
2) Run again: python scripts/35_judge_agreement.py --judge-file outputs/reports/judge_qa_pairwise_qa-dpo_vs_qa.json
"""
import argparse
import csv
import json
from collections import Counter
from pathlib import Path
from common import OUT

p = argparse.ArgumentParser()
p.add_argument("--judge-file", required=True)
p.add_argument("--n", type=int, default=30, help="how many items you will label by hand")
a = p.parse_args()

judge = json.loads(Path(a.judge_file).read_text())["per_item"][: a.n]
labels = OUT / "reports" / "human_labels.csv"
if not labels.exists():                                    # first run: make the sheet for you to fill
    with labels.open("w", newline="") as f:
        w = csv.writer(f)
        w.writerow(["i", "human"])
        w.writerows([[i, ""] for i in range(len(judge))])
    print(f"fill the 'human' column in {labels}, then run this script again")
    raise SystemExit

human = {int(r["i"]): r["human"].strip() for r in csv.DictReader(labels.open()) if r["human"].strip()}
pairs = [(judge[i], h) for i, h in human.items() if i < len(judge)]
n = len(pairs)
observed = sum(j == h for j, h in pairs) / n                # raw agreement
jc, hc = Counter(j for j, _ in pairs), Counter(h for _, h in pairs)
expected = sum(jc[k] * hc[k] for k in set(jc) | set(hc)) / (n * n)   # agreement by pure chance
kappa = (observed - expected) / (1 - expected) if expected < 1 else 1.0
print(f"items={n}  agreement={observed:.2f}  Cohen's kappa={kappa:.2f}")
print("kappa > 0.6 = judge is usable; 0.4-0.6 = use with care; < 0.4 = do not trust this judge")
```

Line	What it does (plain English)
`judge = json.loads(...)["per_item"][: a.n]`	the judge's verdict per item, saved by 32_judge.py
`if not labels.exists(): ... raise SystemExit`	first run only creates the sheet for you
`human = {int(r["i"]): r["human"].strip() ...}`	your labels; empty rows are ignored
`observed = sum(j == h ...) / n`	raw agreement: how often you and the judge said the same
`expected = sum(jc[k] * hc[k] ...) / (n * n)`	agreement you would get by chance given how often each of you uses each label

`kappa = (observed - expected) / (1 - expected)`	Cohen's kappa: agreement beyond chance. 1 = perfect, 0 = no better than chance

A lesson from testing: with a judge that always answered "qa-dpo", raw agreement was 0.80 but kappa was 0.00. High agreement can be meaningless when one label dominates; that is exactly why kappa exists.

Watch out: you are not a physician. For a real medical product the labels must come from clinicians (HealthBench used 262 physicians). For this project, be transparent: "judge-human agreement measured on 30 items by the author".

CHECKPOINT: you can report kappa with its interpretation.

SECTION 7.5 - Criterion-level (rubric) scoring for structured summaries

WHERE: VS Code Terminal, folder ~/-MEDSCRIPTAI/scripts, (.venv-eval) active

GOAL: precision/recall over the individual fields of the clinical summary, the HealthBench rubric idea made verifiable.

Step 1. Run:

```
python 36_summary_field_scores.py --models base summary
```

You should see: one line per model with field precision, recall, F1 and invalid_json counts. The fine-tuned summary adapter should have far fewer invalid JSON outputs and higher recall than base.

FILE scripts/36_summary_field_scores.py:

```
"""Elite check: criterion-level scoring for the summary task (rubric idea from HealthBench, made verifiable).

Every field path in the REFERENCE JSON is one criterion, e.g. "patient information.age".
recall    = reference fields the model filled       (missing facts lower it)
precision = model fields that exist in the reference (invented fields lower it = hallucination signal)
"""
import argparse
import json
from common import OUT, read_jsonl

p = argparse.ArgumentParser()
p.add_argument("--models", nargs="+", default=["base", "summary"])
a = p.parse_args()


def paths(value, prefix=""):
    """{"a": {"b": 1}, "c": [{"d": 2}]} -> {"a.b", "c.d"} (list items are merged)."""
    out = set()
    if isinstance(value, dict):
        for k, v in value.items():
            out |= paths(v, f"{prefix}{k.lower().strip()}.")
    elif isinstance(value, list):
        for v in value:
            out |= paths(v, prefix)
    elif value not in (None, "", "None"):
        out.add(prefix.rstrip("."))
    return out


def load(text):
    try:
        return json.loads((text or "").strip().removeprefix("```json").removesuffix("```").strip())
    except json.JSONDecodeError:
        return None


for model in a.models:
    rows = read_jsonl(OUT / "preds" / "summary" / f"{model}.jsonl")
    tp = fp = fn = invalid = 0
    for r in rows:
        ref, pred = paths(load(r["reference"]) or {}), load(r["prediction"])
        if pred is None:
            invalid += 1
            fn += len(ref)                    # an unreadable answer misses every criterion
            continue
        got = paths(pred)
        tp += len(ref & got)
        fp += len(got - ref)
        fn += len(ref - got)
    precision = tp / max(tp + fp, 1)
```

```
    recall = tp / max(tp + fn, 1)
    f1 = 2 * precision * recall / max(precision + recall, 1e-9)
    print(f"{model:10} field precision={precision:.3f} recall={recall:.3f} F1={f1:.3f} invalid_json=
{invalid}/{len(rows)}")
```

Line	What it does (plain English)
`def paths(value, prefix="")`	flattens nested JSON into field paths such as "patient information.age"; list items are merged
`elif value not in (None, "", "None"): out.add(...)`	a field only counts if it has a real value
`def load(text)`	parses JSON, also inside a ```json fence; None if invalid
`if pred is None: invalid += 1; fn += len(ref)`	an unreadable answer misses every criterion
`tp += len(ref & got)`	fields present in both = criteria met
`fp += len(got - ref)`	fields the model invented that the reference does not have = hallucination signal
`fn += len(ref - got)`	facts the model missed
`precision`, `recall`, `f1`	standard definitions over all fields of all notes

Watch out: this checks that the right FIELDS are filled, not that each VALUE is correct. For values, combine it with the judge (Section 5.7) or exact match on simple fields (age, sex). Saying this limitation out loud is part of being elite.

CHECKPOINT: you have field precision/recall for base and summary.

SECTION 7.6 - Zero-downtime adapter rollout and rollback (canary)

WHERE: VS Code Terminal, folder ~/-MEDSCRIPTAI, Compose stack from Section 6.7 running

GOAL: switch task qa from qa-dpo to a new adapter version without restarting vLLM, then roll back in seconds.

Step 1. Ask vLLM (from inside the api container, because vLLM's port is not published) which adapters it serves:

```
docker compose -f deploy/docker-compose.yml exec api python -c "import urllib.request,json; print([m['id'] for m in
json.load(urllib.request.urlopen('http://vllm:8000/v1/models'))['data']])"
```

You should see: ['base', 'qa', 'notes', 'summary', 'qa-dpo'].

Step 2. Change the route for qa in deploy/.env (open the file and add this line, then save):

```
MODEL_QA=qa
```

Step 3. Recreate ONLY the api container (vLLM keeps running and keeps its GPU memory):

```
cd deploy && docker compose --env-file .env up -d api && cd ..
```

Step 4. Check the route:

```
curl -s -X POST localhost:8080/v1/generate -H "Content-Type: application/json" -H "X-API-Key: $(grep ^API_KEY
deploy/.env | cut -d= -f2)" -d '{"task":"qa","input":"Which vitamin deficiency causes scurvy?\n\nOptions:\nA) A\nB)
B12\nC) C\nD) D"}' | jq -r .model
```

You should see: qa (the SFT adapter now answers). Rollback = set MODEL_QA=qa-dpo and repeat Step 3.

Concept	What it means
canary	send a small share of traffic to the new version and watch metrics before switching everyone (with two API deployments behind a load balancer, or a percentage router in the API)
rollback	returning to the previous version; here one variable and a 2-second container restart
why it is cheap	adapters are tiny and vLLM can host many at once; the expensive base model never reloads

Watch out: the Compose file does not enable VLLM_ALLOW_RUNTIME_LORA_UPDATING (production safety). To add a brand-new adapter in Compose, add it to --lora-modules and recreate vllm, or enable runtime updates only on a private network.

CHECKPOINT: you switched and rolled back without vLLM restarting (docker compose ps shows vllm Up for the same duration).

SECTION 7.7 - Report and model card

WHERE: VS Code Terminal, folder ~/-MEDSCRIPTAI/scripts, (.venv-eval) active

GOAL: outputs/reports/REPORT.md ready for your Hugging Face model card, README and interviews.

Step 1. Run:

```
python 34_report.py
```

Step 2. Download outputs/reports with WinSCP (drag the folder to your laptop).

Step 3. On your Hub adapter page click Edit model card and paste REPORT.md, then add: base model, data sources and licences, training settings (r=16, alpha=16, LR 2e-4, 1 epoch, DPO beta 0.1, LR 5e-6), contamination result, significance results, judge kappa, limitations, "Built with Llama", and "not for clinical use".

FILE scripts/34_report.py:

```
"""Build outputs/reports/REPORT.md (paste it into your Hugging Face model card and your CV/portfolio)."""
import json
from pathlib import Path
from common import OUT

reports = OUT / "reports"
metrics = json.loads((reports / "metrics.json").read_text())
cols = ["n", "accuracy", "format_ok", "json_valid", "rougeL", "bertscore_f1"]
lines = ["# MedScriptAI evaluation report", "", "## Automatic metrics", "",
         "| task/model | " + " | ".join(cols) + " |", "|" + "---|" * (len(cols) + 1)]
for name, m in metrics.items():
    lines.append(f"| {name} | " + " | ".join(str(m.get(c, "-")) for c in cols) + " |")
lines += ["", "## LLM-as-a-judge", ""]
for path in sorted(reports.glob("judge_*.json")):
    lines.append(f"- **{path.stem.removeprefix('judge_')}**: `{json.dumps(json.loads(path.read_text()))}`")
lines += ["", "Judge: Qwen2.5-14B-Instruct. Pairwise results use position swapping (a win must win in both orders).",
          "", "Built with Llama. Educational use only; not a medical device and not medical advice."]
(reports / "REPORT.md").write_text("\n".join(lines) + "\n")
print("\n".join(lines))
```

Line	What it does (plain English)
`metrics = json.loads((reports / "metrics.json").read_text())`	all automatic metrics from Section 5.6/6.4
`cols = [...]`	the columns of the Markdown table
`lines.append(f"| {name} | ...")`	one table row per task/model; "-" where a metric does not apply
`for path in sorted(reports.glob("judge_*.json"))`	every judge result file
final lines	judge description, "Built with Llama", and the not-for-clinical-use statement

CHECKPOINT: REPORT.md opens and shows your numbers.

7.8 What elite teams would do next (pick one as your stretch goal)

- **RL with verifiable rewards (GRPO).** For multiple-choice QA the reward is checkable (correct letter = 1). TRL's GRPOTrainer and Unsloth support it; this is the DeepSeek-R1 / Tulu 3 style of post-training (verify current TRL/Unsloth GRPO docs).
- **Other preference losses.** Compare DPO with SimPO (loss_type="sigmoid_norm" in TRL), IPO, KTO or ORPO on the same pairs; report the significance test for each.
- **Iterative on-policy DPO.** Regenerate pairs with the DPO model and train a second round; stop when the win rate stops moving.
- **Serving efficiency.** FP8 or AWQ-quantised base model for serving, prefix caching for the long shared system prompt, speculative decoding; measure tokens/second and cost per 1,000 requests.
- **Safety evaluation.** A red-team set (self-harm, dosage requests, emergencies) with expected refusals or escalation messages; track refusal rate next to helpfulness.
- **Privacy and regulation.** Real clinical notes contain PHI: de-identify, keep traces private (self-hosted LangSmith or redaction), and know that software making clinical decisions is regulated (for example FDA "software as a medical device").

7.9 Common mistake at this level

The mistake: running elite-looking analyses that are not tied to a decision: a kappa nobody reads, a CI computed on a different test set than the headline number, a contamination check run after the paper is written.

How to spot it: every analysis answers one question that would change what you ship or what you claim. Write the question first ("is DPO better than SFT on QA accuracy?"), then the analysis, then the decision ("ship qa-dpo" or "keep qa").

PART 8 - Tools and Resources Map (checked October 2026)

GitHub stars were read on 21 October 2026 and change daily. Prices marked (verify) could not be confirmed on the official page.

Tool	What it does	Free / Open-source / Paid	Exact cost + free-tier limits	Who uses it	Platform	Official link (+ stars)	Best for	My verdict
Unsloth 2026.10.7	2x faster, lower-VRAM LoRA/QLoRA/GRPO training	open source (Apache-2.0) + paid Pro	free	individuals, startups, Kaggle winners	Linux/Windows, NVIDIA (AMD/Apple partial)	https://github.com/unslothai/unsloth (78k)	single-GPU QLoRA	default choice for one GPU
TRL 1.15/1.16	SFT, DPO, GRPO, reward trainers	open source (Apache-2.0)	free	Hugging Face, AI2, most open post-training papers	Python	https://github.com/huggingface/trl (19.6k)	the standard post-training API	learn its configs deeply
PEFT 0.21	LoRA and other adapters	open source	free	industry-wide	Python	https://github.com/huggingface/peft (21.8k)	adapters	always used under TRL/Unsloth
Transformers 5.x	model and tokenizer library	open source	free	everyone	Python	https://github.com/huggingface/transformers (168k)	loading any model	core skill
bitsandbytes 0.50	4-bit/8-bit quantisation, 8-bit optimizers	open source	free	QLoRA users	Linux/Windows	https://github.com/bitsandbytes-foundation/bitsandbytes (8.5k)	QLoRA	needed, rarely touched
distilabel 1.5.3	synthetic data and AI-feedback pipelines	open source (Apache-2.0)	free	Argilla/HF community, Zephyr-style datasets	Python	https://github.com/argilla-io/distilabel (3.4k)	generate + rate pipelines	last release Jan 2025: pin versions or write plain async OpenAI calls
vLLM 0.32	high-throughput inference, multi-LoRA	open source (Apache-2.0)	free	Meta, Mistral, AWS, many startups	Linux, NVIDIA/AMD/TPU	https://github.com/vllm-project/vllm (94.6k)	serving open models	industry default

SGLang	inference engine (RadixAttention, structured output)	open source	free	xAI, LMSYS, cloud providers	Linux	https://github.com/sgl-project/sglang (37.6k)	very high throughput, agents	strong alternative to vLLM
Axolotl / LLaMA-Factory	config-driven fine-tuning frameworks	open source	free	researchers, startups	Linux	https://github.com/axolotl-ai-cloud/axolotl (12.6k), https://github.com/hiyouga/LLaMA-Factory (75.6k)	multi-GPU / no-code training	good once you understand TRL
FastAPI 0.156	Python web API framework	open source (MIT)	free	Microsoft, Netflix, Uber, countless startups	Python	https://github.com/fastapi/fastapi (103k)	ML APIs	default for Python services
LangSmith	tracing, evaluation, prompt management	paid SaaS with free tier	Developer plan US$0: 1 seat, about 5k base traces/month; overage about US$2.50-5.00 per 1k traces (sources disagree, verify) [11]	LangChain users, many enterprises	web + SDK	https://smith.langchain.com (SDK 1.1k)	quick tracing	fine for demos; check data-privacy terms
Langfuse	open-source LLM tracing/evals	open source (MIT) + cloud	self-host free; cloud free tier (verify)	startups wanting self-hosting	web + SDK	https://github.com/langfuse/langfuse (36.1k)	self-hosted tracing	best when data must stay inside
Docker / Compose	containers	free (Docker Desktop paid for large companies)	Engine free	everyone	Linux/Win/Mac	https://docs.docker.com	packaging	must-know
FakeCloud 0.51	local AWS emulator, 106 services incl. ECR push/pull, EC2, SageMaker control plane	open source (AGPL-3.0)	free, no account [5]	developers testing AWS code	Linux/macOS/Docker	https://github.com/faiscadev/fakecloud (761)	free AWS practice and integration tests	perfect for learning; not a GPU

LocalStack	the best-known AWS emulator	open source core + paid tiers	free tier needs an account/auth token (verify)	many companies' CI	Docker	https://github.com/localstack/localstack (65k)	team CI	industry standard alternative
moto	Python library that mocks AWS	open source	free	Python unit tests	Python	https://github.com/getmoto/moto (8.7k)	unit tests	lightweight
AWS (ECR, S3, EC2, SageMaker, CloudWatch)	production cloud	paid	g6.xlarge about US$0.805/h on demand in us-east-1 [12]; ECR storage about US$0.10/GB-month (verify)	most enterprises	web/CLI	https://aws.amazon.com	real deployment	learn the core 6 services
Hyperstack	GPU cloud VMs	paid	A100-80G-PCIe about US$1.35/h, L40 about US$1.00/h, RTX A6000 about US$0.50/h on demand [1] (verify)	startups, researchers	web/API	https://www.hyperstack.cloud	cheap single-GPU training	good value; remember hibernate
lm-evaluation-harness	standard benchmark runner (MedQA, MMLU, ...)	open source (MIT)	free	EleutherAI, HF Open LLM Leaderboard	Python	https://github.com/EleutherAI/lm-evaluation-harness (14.3k)	comparable benchmark numbers	add it after this project
Stanford HELM / MedHELM	holistic evaluation framework	open source	free	Stanford, research	Python	https://github.com/stanford-crfm/helm (3k)	rigorous medical evaluation	read the MedHELM paper
Prometheus-eval	open judge models	open source	free	researchers	Python	https://github.com/prometheus-eval/prometheus-eval (1.1k)	judging without paid APIs	good second judge
DeepEval	LLM test framework (pytest-style)	open source + cloud	free core	startups	Python	https://github.com/confident-ai/deepeval (19k)	CI tests for LLM apps	nice for API regression tests

uv	fast Python package/env manager	open source	free	fast-growing industry default	all	https://github.com/astral-sh/uv (91k)	environments	use everywhere
VS Code + Remote-SSH	editor on remote machines	free	free	everyone	all	https://code.visualstudio.com (194k)	working on GPU VMs	default
WinSCP	SFTP/SCP file manager	free (GPL)	free	Windows users	Windows	https://winscp.net (3.9k)	moving files	simple and reliable

Best free setup: Google Colab or Kaggle free GPU (T4, 15-16 GB; no bf16, slower) + Unsloth + TRL + FakeCloud + LangSmith Developer + Hugging Face Hub. You can do Parts 4-5 on a T4 with --batch 2 --grad-accum 8, but not the 14B teacher.

Best paid setup: 1x H100 80 GB (Hyperstack about US$1.90-3.00/h [1]) for everything; AWS or GCP for deployment with Terraform; Langfuse self-hosted or LangSmith Plus for tracing; W&B or MLflow for experiment tracking.

Best setup within this manual's budget (about US$30-40): Hyperstack 1x A100-80G-PCIe only while working + hibernate + FakeCloud for every AWS rehearsal + one 2-3 hour real AWS deployment with immediate teardown.

Five open-source projects worth studying:

- **huggingface/alignment-handbook** (5.7k stars): the exact Zephyr SFT + DPO recipes and configs; compare them with yours. https://github.com/huggingface/alignment-handbook
- **allenai/open-instruct** (3.9k stars): AI2's Tulu 3 post-training code (SFT, DPO, RLVR) with decontamination tooling. https://github.com/allenai/open-instruct
- **epfLLM/meditron** (2.2k stars): an open medical LLM with documented data and evaluation. https://github.com/epfLLM/meditron
- **openai/simple-evals** (4.7k stars): includes the HealthBench grading code and rubric approach. https://github.com/openai/simple-evals
- **vllm-project/vllm** (94.6k stars): read docs/features/lora.md and the LoRA code path to understand multi-adapter batching. https://github.com/vllm-project/vllm

PART 9 - Keep It Working

Task	How
Save your work	push adapters and datasets to the Hub (Sections 4.6, 5.8); git add/commit/push your code changes; aws s3 sync outputs/reports to S3
Back up the VM	your important files are on the Hub and in Git; the VM is disposable. To be extra safe, Hyperstack > Volumes > create a Shared Storage Volume and copy outputs/ there (see Hyperstack docs)
Stop costs	Hibernate the Hyperstack VM (Section 2.12); TARGET=aws bash deploy/teardown.sh; delete unused Hub repos is optional (free)
Restart after hibernate	Restore the VM; update the IP in ~/.ssh/config and WinSCP; on the VM: mkdir -p /ephemeral/hf-cache; restart FakeCloud (Section 2.10 Step 4); cd deploy && docker compose --env-file .env up -d
Resume an interrupted training run	checkpoints are in outputs/checkpoints/RUN; change trainer.train() to trainer.train(resume_from_checkpoint=True) in 10_train_sft.py and run the same command again
Update libraries safely	never upgrade in place: create .venv-train2, install the new versions, run 03_validate_jsonl.py and a --max-steps 20 training test, compare, then switch. Unsloth releases often pin new transformers/trl versions
Update vLLM	change VLLM_IMAGE to the new tag, restart, re-run 30_generate_preds.py on 50 rows and compare outputs with the old version
Undo a code change	git diff (see changes), git restore FILE (undo one file), git log + git revert COMMIT (undo a pushed commit)
Free disk space	docker system prune (removes stopped containers and dangling images); du -sh $HF_HOME/hub/* to find big cached models, then delete only the folders you no longer need
Rotate secrets	create a new HF/LangSmith key, aws secretsmanager put-secret-value, restart the API; delete the old key on the website

PART 10 - Troubleshooting

Symptom	Likely cause	Exact fix (steps)
ssh: Connection timed out	SSH access/firewall not enabled or wrong IP after restore	1) Hyperstack > VM > Firewall: Enable SSH access. 2) Copy the current PUBLIC IP. 3) Update HostName in ~/.ssh/config
torch.cuda.is_available() is False	CPU build of PyTorch installed	1) source .venv-train/bin/activate. 2) uv pip install -r requirements/train.txt --torch-backend cu126 --reinstall
CUDA out of memory during training	batch too big, or vLLM/teacher still running on the GPU	1) nvidia-smi to see who uses memory. 2) docker rm -f medscriptai-vllm medscriptai-teacher. 3) or add --batch 4 --grad-accum 4
Two <|begin_of_text|> tokens / quality worse than expected	chat template applied twice	use 10_train_sft.py as given (removeprefix(tokenizer.bos_token)); print train[0]["text"] and check it starts with <|start_header_id|>
vLLM: "CUDA driver version is insufficient for CUDA runtime version"	image CUDA newer than the driver	1) keep -e VLLM_ENABLE_CUDA_COMPATIBILITY=1 with the -cu129 image (data-centre GPU). 2) or deploy with the R595 / CUDA 13.2 OS image and VLLM_IMAGE=vllm/vllm-openai:v0.32.0
vLLM returns 404 "The model `qa` does not exist"	adapter not registered or wrong name	1) curl localhost:8000/v1/models. 2) check outputs/adapters/sft-qa/adapter_config.json exists. 3) restart serve/start_vllm.sh
Requests for different adapters are slow / queue	--max-loras left at the default 1	start vLLM with --max-loras 4 (already in start_vllm.sh and docker-compose.yml)
distilabel: No module named 'requests' / datasets errors	wrong environment or unpinned versions	1) source .venv-synth/bin/activate. 2) uv pip install -r requirements/synth.txt
Docker build: 429 Too Many Requests pulling the base image	anonymous pull rate limit	1) wait 10 minutes, or 2) docker build --build-arg BASE_IMAGE=python:3.12-slim ... (other registry), or 3) docker login to Docker Hub
AWS: VcpuLimitExceeded when launching g6.xlarge	GPU quota still 0	Service Quotas > Amazon EC2 > Running On-Demand G and VT instances > request 8 vCPUs (Section 2.11); wait for approval
aws CLI: Could not connect to the endpoint URL http://localhost:4566	FakeCloud not running (e.g. after hibernate)	1) nohup fakecloud --storage-mode persistent --data-path ~/fakecloud-data > ~/fakecloud.log 2>&1 &. 2) curl localhost:4566/_fakecloud/health
Real AWS command accidentally went to FakeCloud (or the opposite)	AWS_ENDPOINT_URL still set / not set	echo $AWS_ENDPOINT_URL before every blue-column step; unset AWS_ENDPOINT_URL for real AWS
teardown: DependencyViolation deleting the security group	instance still shutting down	run TARGET=aws bash deploy/teardown.sh again after 1-2 minutes (the script already retries 6 times)

PART 11 - Practice Projects (goals and success criteria only, no solutions)

11.1 Easy - a fourth adapter: patient-friendly explanations

Goal: an adapter "explain" that rewrites a clinical note in plain language for patients (reading age about 12).

Requirements: generate 2,000 training pairs with distilabel from the notes data (teacher writes the plain version); add a SYSTEM["explain"] prompt; train with 10_train_sft.py; serve it as a fifth adapter; add task "explain" to the API.

Success criteria: the API returns explanations for 20 unseen notes; a readability score (for example Flesch-Kincaid grade) is at least 3 grades lower than the source note; the judge rates faithfulness at least 4/5 on 80% of items.

11.2 Medium - RL with verifiable rewards for QA

Goal: improve qa-dpo with GRPO using the correct letter as the reward.

Requirements: use TRL's GRPOTrainer (or Unsloth's GRPO support) on the 3,000 DPO prompts; reward = 1 for the correct letter, 0 otherwise, plus a small format reward; evaluate exactly as in Section 6.4 and 7.3.

Success criteria: a QA accuracy gain over qa-dpo whose 95% paired bootstrap interval excludes 0, with format_ok still above 99%.

11.3 Hard - production hardening on AWS

Goal: a deployment a security reviewer would accept.

Requirements: infrastructure as code (Terraform or AWS CDK) for everything in deploy.sh; HTTPS with an Application Load Balancer and an ACM certificate; the API in a private subnet; secrets only from Secrets Manager; CloudWatch alarm on 5xx rate and p95 latency with e-mail alerts; a canary that sends 10% of qa traffic to a new adapter; a load test (for example locust) report; teardown with one command; everything rehearsed on FakeCloud first.

Success criteria: terraform apply and destroy both succeed on FakeCloud and AWS; p95 latency under 3 s at 5 requests/second on g6.xlarge for qa; zero secrets in Git, images or user data; the canary can be rolled back in under 1 minute.

PART 12 - Cheat Sheet

Part 2 - workspace

```
ssh-keygen -t ed25519 -f $env:USERPROFILE\.ssh\hyperstack_ed25519 -C "medscriptai"
ssh -i $env:USERPROFILE\.ssh\hyperstack_ed25519 ubuntu@VM_IP
nvidia-smi
echo 'export HF_HOME=/ephemeral/hf-cache' >> ~/.bashrc
curl -LsSf https://astral.sh/uv/install.sh | sh
git clone -b claude/determined-carson-9p7115 https://github.com/kashyapphassan2000-netizen/-MEDSCRIPTAI.git
uv venv .venv-train --python 3.12 && source .venv-train/bin/activate
uv pip install -r requirements/train.txt --torch-backend cu126
uv venv .venv-synth --python 3.12 && uv pip install --python .venv-synth -r requirements/synth.txt
uv venv .venv-eval --python 3.12 && uv pip install --python .venv-eval -r requirements/eval.txt --torch-backend cu126
curl -fsSL https://fakecloud.dev/install.sh | bash -s -- --version v0.51.0
nohup fakecloud --storage-mode persistent --data-path ~/fakecloud-data > ~/fakecloud.log 2>&1 &
export AWS_ENDPOINT_URL=http://localhost:4566 AWS_ACCESS_KEY_ID=test AWS_SECRET_ACCESS_KEY=test AWS_DEFAULT_REGION=us-
east-1
unset AWS_ENDPOINT_URL        # back to real AWS
tmux new -s NAME   |   Ctrl+B then D (detach)   |   tmux attach -t NAME
```

Part 4 - data and first SFT (.venv-train, folder scripts)

```
python 01_download_data.py
python 02_build_sft_jsonl.py
python 03_validate_jsonl.py
python 10_train_sft.py --task qa --max-train 2000
python 11_chat_test.py --task qa --n 3
python 12_push_to_hub.py --folder ../outputs/adapters/sft-qa --repo YOUR_HF_USERNAME/Llama-3.1-8B-MedScriptAI-qa-lora
aws s3 sync outputs/adapters/sft-qa s3://$BUCKET/adapters/sft-qa
```

Part 5 - synthetic data, three adapters, evaluation

```
bash serve/start_teacher.sh                              # teacher/judge on :8001
python 20_synth_explanations.py --limit 50               # (.venv-synth)
python 10_train_sft.py --task qa --train-file ../data/sft/qa_train_synth.jsonl --max-train 12000
python 10_train_sft.py --task notes && python 10_train_sft.py --task summary
GPU_UTIL=0.35 bash serve/start_vllm.sh                   # student + adapters on :8000
python 30_generate_preds.py --task qa --models base qa   # (.venv-eval)
python 31_eval_metrics.py
python 32_judge.py --task notes --mode score --models base notes --n 100
```

Part 6 - DPO, serving, API, Docker, deployment

```
python 21_build_dpo_pairs.py --task qa --limit 1500      # (.venv-synth)
python 22_train_dpo.py --task qa                         # (.venv-train)
curl -s -X POST localhost:8000/v1/load_lora_adapter -H "Content-Type: application/json" -d
'{"lora_name":"NAME","lora_path":"/adapters/FOLDER"}'
curl -s -X POST localhost:8000/v1/unload_lora_adapter -H "Content-Type: application/json" -d '{"lora_name":"NAME"}'
python 32_judge.py --task qa --mode pairwise --models qa-dpo qa --n 200
VLLM_URL=http://localhost:8000/v1 API_KEY=dev-key PYTHONPATH=../scripts uvicorn app:app --port 8080
docker build -f api/Dockerfile -t medscriptai-api:v1 .
cd deploy && docker compose --env-file .env up -d
aws ecr get-login-password | docker login --username AWS --password-stdin $REGISTRY
docker tag medscriptai-api:v1 $REGISTRY/medscriptai-api:v1 && docker push $REGISTRY/medscriptai-api:v1
TARGET=fakecloud bash deploy/deploy.sh      |   TARGET=aws bash deploy/deploy.sh
TARGET=fakecloud bash deploy/teardown.sh    |   TARGET=aws bash deploy/teardown.sh
```

Part 7 - elite checks

```
python 04_contamination_check.py
python 33_significance.py --a qa --b qa-dpo
python 35_judge_agreement.py --judge-file ../outputs/reports/judge_qa_pairwise_qa-dpo_vs_qa.json
python 36_summary_field_scores.py --models base summary
python 34_report.py
```

Key numbers and settings

Setting	Value	Why
LoRA rank / alpha / dropout	16 / 16 / 0	scale 1.0; fast in Unsloth
target modules	q,k,v,o,gate,up,down	all linear layers
SFT LR / epochs / batch	2e-4 / 1 / 16 effective	QLoRA standard
DPO beta / LR / batch	0.1 / 5e-6 / 16 pairs	stable DPO
max sequence length	2048 (train), 4096 (serve)	data max about 1,600 tokens
DPO loss at step 0	0.693 = ln 2	policy equals reference
vLLM LoRA flags	--enable-lora --max-loras 4 --max-lora-rank 16 --max-cpu-loras 8	default max-loras is 1
Memory rule of thumb	2 bytes/param (bf16), ~0.5-0.7 bytes/param (4-bit)	8B: 16 GB vs ~6 GB
Hyperstack states	SHUTOFF = still billed; HIBERNATED = GPU not billed, /ephemeral wiped	Section 2.12
FakeCloud	http://localhost:4566, account 123456789012, any keys	Section 2.10

PART 13 - Self-Test

1. What happens if you remove .removeprefix(tokenizer.bos_token) from 10_train_sft.py?
2. Do this: compute how many optimizer steps one epoch takes with 7,040 notes examples, --batch 8 and --grad-accum 2.
3. What happens if you start vLLM without --max-loras and three users call qa, notes and summary at the same moment?
4. What happens to the DPO loss at step 0, and why?
5. Do this: write the aws command that lists the images in the medscriptai-api ECR repository, for FakeCloud.
6. What happens if you hibernate the Hyperstack VM with the Hugging Face cache on /ephemeral, and what must you do after Restore?
7. Your QA accuracy goes from 0.612 to 0.628 on 500 questions and the paired 95% CI is [-0.010, +0.042]. What may you claim?
8. What happens if the LLM judge always answers "A" in pairwise mode with position swapping?
9. Do this: change the API so task qa is answered by the SFT adapter instead of the DPO adapter, without editing Python code.
10. Why does the EC2 instance profile policy list specific ARNs instead of "Resource": "*" for ECR images, S3 and secrets?

PART 14 - Sources

[1] Hyperstack GPU pricing: https://www.hyperstack.cloud/gpu-pricing (and https://computeprices.com/providers/hyperstack/gpus/a100pcie)

[2] Hyperstack getting started (billing starts when ACTIVE; console flow): https://docs.hyperstack.cloud/docs/getting-started/

[3] Hyperstack VM states and billing; hibernation: https://docs.hyperstack.cloud/docs/billing/states-and-billing and https://docs.hyperstack.cloud/docs/virtual-machines/hibernation

[4] Hyperstack operating system images: https://docs.hyperstack.cloud/docs/virtual-machines/images

[5] FakeCloud: https://fakecloud.dev and https://github.com/faiscadev/fakecloud

[6] Hyperstack ephemeral storage (/ephemeral, wiped on hibernate): https://docs.hyperstack.cloud/docs/storage/ephemeral

[7] FakeCloud install and ECR docs: https://fakecloud.dev/docs/getting-started/install and https://fakecloud.dev/docs/services/ecr

[8] vLLM LoRA adapters (flags, runtime load/unload): https://docs.vllm.ai/en/latest/features/lora.html

[9] TRL SFTTrainer: https://huggingface.co/docs/trl/main/en/sft_trainer

[10] TRL DPOTrainer: https://huggingface.co/docs/trl/main/en/dpo_trainer

[11] LangSmith pricing (secondary sources, verify on the official page): https://www.langchain.com/pricing, https://www.zenml.io/blog/langsmith-pricing, https://www.metacto.com/blogs/the-true-cost-of-langsmith-a-comprehensive-pricing-integration-guide

[12] AWS g6.xlarge / g5.xlarge pricing (aggregators): https://calculator.holori.com/aws/ec2/g6.xlarge/us-east-1, https://www.doit.com/compute/spot/us-east-1/g6.xlarge

[13] AWS Deep Learning Base OSS Nvidia Driver GPU AMI (Ubuntu 24.04) release notes: https://docs.aws.amazon.com/dlami/latest/devguide/aws-deep-learning-base-gpu-ami-ubuntu-24-04.html

[14] Unsloth documentation: https://unsloth.ai/docs and https://github.com/unslothai/unsloth

[15] Llama 3.1 Community License: https://huggingface.co/meta-llama/Llama-3.1-8B-Instruct and https://www.llama.com/llama3_1/license/

[16] distilabel on PyPI (1.5.3, January 2025): https://pypi.org/project/distilabel/

[17] Datasets: https://huggingface.co/datasets/GBaker/MedQA-USMLE-4-options, https://huggingface.co/datasets/openlifescienceai/medmcqa, https://huggingface.co/datasets/qiaojin/PubMedQA, https://huggingface.co/datasets/AGBonnet/augmented-clinical-notes

[18] Qwen2.5-14B-Instruct: https://huggingface.co/Qwen/Qwen2.5-14B-Instruct

[19] LangSmith SDK: https://github.com/langchain-ai/langsmith-sdk and https://docs.smith.langchain.com

[20] Papers: QLoRA https://arxiv.org/abs/2305.14314, LoRA https://arxiv.org/abs/2106.09685, DPO https://arxiv.org/abs/2305.18290, Zephyr https://arxiv.org/abs/2310.16944, Tulu 3 https://arxiv.org/abs/2411.15124, UltraFeedback https://arxiv.org/abs/2310.01377, LLM-as-a-judge https://arxiv.org/abs/2306.05685, Prometheus 2 https://arxiv.org/abs/2405.01535, PagedAttention https://arxiv.org/abs/2309.06180, S-LoRA https://arxiv.org/abs/2311.03285, GPT-3 https://arxiv.org/abs/2005.14165, BERTScore https://arxiv.org/abs/1904.09675

[21] Medical evaluation: Med-PaLM 2 https://doi.org/10.1038/s41591-024-03423-7, HealthBench https://arxiv.org/abs/2505.08775, MedHELM https://arxiv.org/abs/2505.23802

[22] AWS documentation: ECR https://docs.aws.amazon.com/ecr/, EC2 user data https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/user-data.html, Service Quotas https://docs.aws.amazon.com/servicequotas/, Secrets Manager https://docs.aws.amazon.com/secretsmanager/

[23] uv: https://docs.astral.sh/uv/ · VS Code Remote-SSH: https://code.visualstudio.com/docs/remote/ssh · WinSCP: https://winscp.net/eng/docs/start

ANSWER KEY

1. The trainer adds <|begin_of_text|> again, so every example starts with two BOS tokens. Training still runs, but the model learns from a sequence it never sees at inference (vLLM sends one BOS), which can hurt quality. The CPU smoke test caught exactly this.

2. 7,040 / (8 × 2) = 440 optimizer steps.

3. The default --max-loras is 1, so only one adapter can be active in a GPU batch; requests for the other two adapters wait for the next batches. Latency rises even though the GPU has capacity. With --max-loras 4 all three run in the same batch.

4. About 0.693 (= ln 2). At step 0 the policy equals the reference (the frozen "ref" copy of the SFT adapter), so both log-ratios are 0, the margin is 0 and -log(sigmoid(0)) = ln 2.

5. aws --endpoint-url http://localhost:4566 ecr describe-images --repository-name medscriptai-api (or set AWS_ENDPOINT_URL and drop the flag).

6. GPU billing stops, the root disk is kept, but /ephemeral is wiped, so the cache is lost (it re-downloads automatically). After Restore: run mkdir -p /ephemeral/hf-cache, update the IP if it changed, and restart FakeCloud.

7. Only that the observed difference is +1.6 points and NOT statistically significant: the interval contains 0. Do not claim an improvement; collect more test data or a bigger effect.

8. With swapping, A-then-B gives "A" (model 1) and B-then-A gives "A" (model 2), so the orders disagree and every item becomes a tie. The win rates are 0/0 and ties 100%: a clear signal of a broken or position-biased judge.

9. Set MODEL_QA=qa in deploy/.env and run docker compose --env-file .env up -d api in the deploy folder (only the API container restarts).

10. Least privilege: if the machine is compromised, the attacker can only read MedScriptAI's own image, bucket and secrets, not every image, bucket and secret in the account. "Resource": "*" is kept only for ecr:GetAuthorizationToken (it has no resource) and logs.